

Agentic AI untuk Bisnis dengan Open Source

Membangun Agen Kerja Lokal untuk Operasi Pengetahuan,
Manajemen Proses, dan Workflow Bisnis

Onno W. Purbo
onno@itts.ac.id
onno@indo.net.id
Twitter onnowpurbo



Asosiasi Manajemen Indonesia (AMA)
Institut Teknologi Tangerang Selatan (ITTS)
DISKOMINFO Tangerang Selatan
OnnoCenter
2026

Lisensi: Karya ini dilisensikan di bawah **Creative Commons Attribution (CC BY) 4.0 International**. Anda bebas menggunakan, membagikan, dan mengadaptasi karya ini untuk tujuan apa pun, termasuk komersial, dengan tetap mencantumkan atribusi yang layak kepada penulis.

Disclaimer: Buku ini disusun untuk tujuan edukasi, dokumentasi, dan refleksi publik berdasarkan sumber-sumber yang dianggap kredibel pada saat penulisan. Penulis telah berupaya menjaga akurasi, namun sangat mungkin masih terdapat kekurangan atau kekeliruan; karena itu, buku ini tidak dimaksudkan sebagai satu-satunya rujukan yang final, dan penulis terbuka terhadap koreksi serta masukan yang membangun

Kata Pengantar

Alhamdulillah, puji syukur ke hadirat Allah SWT, atas rahmat-Nya buku **Agentic AI untuk Bisnis dengan Open Source, 2026** ini dapat diselesaikan. Terima kasih kepada semua pihak, sahabat, rekan, dan komunitas yang terus memberi semangat, gagasan, dan inspirasi.

Buku ini ditulis sekarang karena AI sedang bergerak dari sekadar alat bantu menulis menjadi **alat kerja** yang dapat membantu membaca konteks, mencari pengetahuan, menyiapkan langkah, dan mendukung *workflow* bisnis. Bagi Indonesia, **Agentic AI** penting karena dapat membantu membuat proses kerja lebih cepat, lebih rapi, lebih hemat, dan lebih bisa diaudit.

Buku ini juga menekankan pentingnya **open source**. Alasannya jelas: **biaya lebih terjangkau, privasi lebih terjaga, kontrol lebih besar, audit lebih mungkin dilakukan, dan kedaulatan *workflow* lebih kuat**. Kita tidak cukup hanya menjadi pengguna teknologi; kita perlu memahami, mengendalikan, dan membangunnya sesuai kebutuhan kita sendiri.

Saya juga perlu menyampaikan bahwa buku ini ditulis **dengan bantuan AI**, terutama untuk membantu mempercepat proses kerja penulisan. Namun demikian, **seluruh tanggung jawab atas isi buku ini tetap berada pada penulis**. Saya mohon maaf jika masih ada kekurangan atau kesalahan.

Semoga buku ini bermanfaat bagi mahasiswa, praktisi, manajer, dan siapa pun yang ingin membangun AI yang lebih berguna, terbuka, dan relevan untuk kebutuhan bisnis Indonesia.

Jakarta, April 2026

Penulis

Executive Summary

Buku ini menegaskan bahwa **Agentic AI** bukan sekadar alat untuk membuat teks atau menjawab pertanyaan, melainkan alat kerja untuk membantu organisasi membaca informasi, mempercepat alur keputusan, menertibkan proses, dan mengurangi beban administratif yang selama ini menyita waktu manusia. Bagi manajer, pesan utamanya sangat jelas: jangan tertarik pada AI karena *hype*, tetapi karena ia bisa dipakai untuk menyelesaikan masalah yang benar-benar terasa di lapangan—*approval* yang lambat, dokumen yang sulit dicari, tiket yang menumpuk, rapat yang tidak berubah menjadi aksi, dan laporan yang memakan terlalu banyak tenaga. Buku ini juga bersikap kritis: **AI tidak akan memperbaiki organisasi yang prosesnya sendiri berantakan**; kalau SOP kacau, data buruk, tanggung jawab kabur, dan jalur eskalasi tidak jelas, maka AI justru bisa mempercepat kekacauan itu. Karena itu, pendekatan yang ditawarkan bukan membeli sistem besar lalu berharap keajaiban, tetapi **memulai dari satu proses, satu pemilik, satu target manfaat, lalu membangun secara bertahap**. Keunggulan pendekatan *open source* yang ditekankan disini juga sangat relevan untuk Indonesia: **biaya lebih terkendali, privasi lebih terjaga, kontrol lebih kuat, audit lebih mungkin dilakukan, dan organisasi tidak terlalu tergantung pada platform tertutup**. Manfaat yang dijanjikan bukan sesuatu yang abstrak, tetapi sangat manajerial: **waktu kerja lebih cepat, biaya koordinasi lebih rendah, kualitas keputusan awal lebih baik, pengetahuan internal lebih mudah ditemukan, dan proses bisnis lebih bisa dipantau serta dipertanggungjawabkan**. Intinya, buku ini mendorong manajer untuk melihat AI bukan sebagai proyek teknologi, tetapi sebagai **alat perbaikan proses dan penguatan kapasitas organisasi**. Yang paling penting, **AI seharusnya tidak menggantikan penilaian manusia, melainkan membuat manusia bekerja dengan konteks yang lebih jelas, keputusan yang lebih tertib, dan energi yang lebih banyak untuk hal-hal yang memang bernilai strategis**.

Cara Menggunakan Buku Ini

Buku ini ditulis untuk **manajer, analis proses, operator, dan praktisi teknis** yang ingin memahami *agentic AI* bukan sebagai jargon, tetapi sebagai **alat kerja yang nyata**. Fokus buku ini bukan pada perlombaan model paling besar, melainkan pada bagaimana membangun **agen kerja lokal** yang bisa membantu membaca konteks, mencari pengetahuan, menyiapkan langkah, menggerakkan *workflow*, dan tetap berada di dalam pagar kontrol yang sehat. Arah itu konsisten dengan naskah buku ini sendiri, yang sejak awal menempatkan hasil akhir sebagai **asisten pengetahuan internal berbasis RAG, agent workflow** untuk laporan, tiket, approval, dan tindak lanjut, serta **agent analisis proses ringan berbasis event log dan process mining**.

Secara praktis, buku ini juga mengikuti ekosistem yang memang realistis untuk pembaca Indonesia yang ingin membangun sistem lokal berbasis *open source*: **Ollama** untuk menjalankan model lokal di Linux, **Open WebUI** sebagai antarmuka *self-hosted* dan *offline-friendly*, serta **n8n** untuk *workflow orchestration* dan automasi AI yang bisa di-*self-host*. Dokumentasi resmi ketiga komponen ini menunjukkan bahwa jalur lokal dan *self-hosted* memang sudah cukup matang untuk dipakai sebagai fondasi belajar maupun pilot bisnis ringan. ([Open WebUI](#))

Untuk siapa buku ini

Manajer dapat memakai buku ini untuk memahami **dimana AI benar-benar memberi nilai bisnis**, bagaimana memilih *use case* yang waras, bagaimana menilai manfaatnya dengan KPI yang masuk akal, dan bagaimana menjaga agar agent tidak bergerak liar tanpa kontrol. Buku ini tidak mengharuskan manajer menjadi programmer; yang dibutuhkan adalah kemampuan melihat proses, bottleneck, risiko, dan keputusan yang perlu dijaga manusia.

Analis proses akan menemukan bagian yang sangat relevan pada pembahasan *workflow, event log, process mining, bottleneck, rework*, dan pemetaan proses sebelum agent dipasang. Bagi analis proses, buku ini bisa dibaca sebagai jembatan antara **proses bisnis** dan **sistem AI yang bisa membantu proses itu bergerak lebih baik**. PM4Py sendiri memang didokumentasikan sebagai pustaka *open source* untuk *process mining* yang mendukung *process discovery, conformance checking*, dan analisis terkait, sehingga sangat cocok untuk pembaca yang ingin menghubungkan jejak proses ke desain agent bisnis. ([Open WebUI](#))

Operator akan paling terbantu oleh bagian-bagian yang bersifat *hands-on*: instalasi host, Docker, Ollama, Open WebUI, OpenClaw, n8n, RAG, *knowledge base, checklist, prompt sistem*, dan lampiran-lampiran praktis. Untuk pembaca jenis ini, buku ini sebaiknya tidak dibaca seperti teori panjang, tetapi seperti **buku kerja**: baca satu bagian, coba, gagal sedikit, perbaiki, lalu lanjut. Dokumentasi resmi Ollama, Open WebUI, dan n8n memang mendukung pola seperti itu karena semuanya menyediakan jalur cepat instalasi, konfigurasi, dan *self-hosting*. ([Open WebUI](#))

Praktisi teknis akan mendapatkan nilai dari bagian arsitektur, RAG, *dataset engineering*, *synthetic data*, *process mining*, *guardrails*, *observability*, dan roadmap produksi ringan. Untuk pembaca teknis, buku ini cukup kaya untuk dijadikan fondasi eksperimen lokal, tetapi tetap sengaja dijaga agar **tidak berubah menjadi buku yang terlalu akademik atau terlalu berat secara infrastruktur**.

Urutan membaca untuk pembaca umum

Untuk pembaca umum, manajer, atau siapa pun yang ingin cepat paham **gambaran besarnya**, urutan membaca yang paling sehat adalah sebagai berikut:

- **Bab 1–2** untuk memahami apa itu *agentic AI*, bedanya dengan chatbot, dan nilai bisnis yang nyata.
- **Bab 3–4** untuk memahami cara memilih *use case* dan arsitektur minimum yang masuk akal.
- **Bab 11–14** untuk melihat bagaimana agent masuk ke proses, bagaimana proses dibaca, bagaimana *governance* dijaga, dan bagaimana bergerak dari pilot ke produksi ringan.
- Setelah itu, baru kembali ke **Bab 5–10** bila ingin melihat detail teknis, RAG, dataset, dan *hands-on*.

Urutan ini cocok untuk pembaca yang ingin menjawab dulu pertanyaan besar seperti: “**Mengapa ini penting?**”, “**Di mana gunanya?**”, dan “**Bagaimana saya memulainya tanpa tersesat di teknologi?**”

Urutan membaca untuk pembaca teknis

Untuk pembaca teknis, admin sistem, atau praktisi yang ingin **langsung membangun stack lokal**, urutan yang lebih efisien adalah:

- **Bab 5–7** untuk menyiapkan host, GPU, Docker, Ollama, Open WebUI, dan runtime agent / orkestrasi workflow.
- **Bab 8–10** untuk memahami RAG, *knowledge base*, *instruction data*, dan *synthetic data*.
- **Bab 11–13** untuk memasukkan agent ke proses nyata dan menyiapkan *guardrails*, *governance*, serta *observability*.
- **Bab 14** untuk menutup dengan pola naik kelas dari pilot ke produksi ringan.
- Gunakan **Lampiran A–G** sebagai *cheat sheet* harian.

Urutan ini cocok untuk pembaca yang pertanyaannya lebih praktis, seperti:

“**Saya harus mulai dari host atau dari model?**”,
“**Bagaimana menyusun RAG yang masuk akal?**”,
“**Kapan pakai n8n, kapan pakai OpenClaw?**”,
dan “**Bagaimana membuat agent cukup aman untuk diuji di lingkungan kerja?**”

Cara membaca buku ini agar tidak tersesat

Buku ini paling berguna bila dibaca dengan **tiga lapis cara pandang**.

Pertama, baca sebagai **peta konsep**. Pahami dulu istilah-istilah penting: agent, *agentic system*, RAG, workflow, *event log*, *process mining*, *guardrails*, dan *governance*. Ini akan membuat pembaca tidak mudah terseret hype.

Kedua, baca sebagai **panduan memilih prioritas**. Tidak semua masalah butuh agent. Tidak semua dokumen harus masuk RAG. Tidak semua proses layak diotomasi. Buku ini berulang kali menekankan bahwa langkah yang sehat adalah **mulai dari rasa sakit proses, bukan dari demo**, lalu memilih intervensi yang cukup kecil tetapi nyata.

Ketiga, baca sebagai **buku kerja**. Jangan tunggu selesai membaca semuanya baru mulai mencoba. Coba satu bagian kecil, misalnya:

- menyalakan Ollama,
- membuka Open WebUI,
- membuat satu *knowledge base*,
- menyusun satu *prompt sistem*,
- atau membuat satu *event log* sederhana.

Lalu belajar dari hasilnya. Open WebUI memang dirancang sebagai platform *self-hosted* yang dapat berjalan *offline*, sedangkan n8n juga menyediakan *self-hosted AI starter kit* dan ekosistem workflow yang cocok untuk eksperimen bertahap. ([Open WebUI](#))

Apa yang dibangun pembaca dalam buku ini

Pada akhir buku ini, pembaca idealnya tidak hanya memahami teori, tetapi juga berhasil membangun **tiga hal yang sangat nyata**.

Pertama, **asisten pengetahuan internal berbasis RAG**. Ini berarti pembaca mampu menaruh SOP, kebijakan, FAQ, atau dokumen kerja ke dalam *knowledge base*, lalu membuat model lokal dapat mencari dan menjawab berdasarkan sumber yang relevan. Open WebUI secara resmi mendukung *Knowledge*, *Focused Retrieval*, *Full Context*, dan sitasi sumber untuk tujuan seperti ini. ([Open WebUI](#))

Kedua, **agent workflow untuk operasi bisnis ringan**. Misalnya untuk triase tiket, ringkasan memo, approval awal, tindak lanjut rapat, atau eskalasi kasus yang perlu perhatian manusia. n8n memang dibangun untuk orkestrasi workflow seperti ini, dan Open WebUI juga terus bergerak ke arah *AI workflow builder* yang memadukan model, tools, *knowledge bases*, dan logika langkah-demi-langkah. ([n8n Docs](#))

Ketiga, **fondasi proses dan governance yang lebih matang**. Pembaca akan belajar bahwa agent yang berguna bukan yang paling liar, tetapi yang paling bisa dijelaskan, diukur, diaudit, dan dihentikan bila perlu. Itu sebabnya buku ini tidak berhenti di instalasi, tetapi bergerak sampai *process management*, *process mining*, *guardrails*, *governance*, dan roadmap 90 hari.

Pesan akhir untuk pembaca

Buku ini tidak menuntut pembaca menjadi pakar AI dalam satu malam. Yang diharapkan justru lebih sederhana dan lebih penting: **pembaca menjadi lebih jernih dalam melihat masalah kerja, lebih disiplin dalam memilih use case, lebih berani membangun solusi lokal, dan lebih bijak dalam memberi batas pada AI**. Itulah sebabnya buku ini dimulai dari pertanyaan bisnis, bergerak ke arsitektur, lalu masuk ke ekosistem lokal, RAG, data, proses, *governance*, dan akhirnya ke produksi ringan.

Kalau dibaca dengan urutan yang sehat dan dicoba sedikit demi sedikit, buku ini tidak hanya akan memberi pengetahuan, tetapi juga membantu pembaca membangun **agen kerja digital yang sederhana, terbuka, hemat, dan relevan** untuk kebutuhan organisasi nyata.

Peta Hasil Akhir

Buku ini tidak ditulis agar pembaca sekadar memahami istilah-istilah seperti *agent*, RAG, *workflow*, atau *process mining*. Buku ini ditulis agar, pada akhir perjalanan membaca dan mencoba, pembaca benar-benar memiliki **gambaran utuh tentang apa yang bisa dibangun**, mengapa hal itu berguna, dan bagaimana memulainya secara realistis di lingkungan kerja yang nyata. Karena itu, *peta hasil akhir* buku ini sengaja dibuat sangat praktis: bukan daftar impian yang terlalu tinggi, tetapi **empat keluaran nyata** yang bisa dicapai secara bertahap. Arah ini sejalan dengan naskah buku Anda, yang memang sejak awal memosisikan target akhirnya sebagai **asisten pengetahuan internal berbasis RAG, agent workflow untuk laporan, tiket, approval, dan tindak lanjut, agent analisis proses ringan berbasis event log dan process mining**, serta **mini-stack lokal** yang dapat diuji di organisasi kecil–menengah.

Secara teknis, target ini juga cukup realistis karena fondasinya memang sudah tersedia di ekosistem *open source* yang matang. **Open WebUI** mendukung *knowledge base* yang dapat mencari, membaca, dan bernalar atas dokumen, dengan mode **Focused Retrieval** dan **Full Context**, sitasi sumber, serta akses yang dapat dibatasi sesuai kebutuhan. **Ollama** menyediakan jalur Linux untuk menjalankan model lokal. **n8n** menyediakan *self-hosted AI starter kit* dan pola *workflow* yang cocok untuk orkestrasi proses bisnis. Sementara **PM4Py** menyediakan perangkat *open source* untuk *process discovery*, *conformance checking*, *performance analysis*, dan analisis proses lain yang relevan untuk mencari peluang intervensi agentic AI. ([Open WebUI](#))

1. Asisten pengetahuan internal berbasis RAG

Hasil pertama yang dibangun pembaca melalui buku ini adalah **asisten pengetahuan internal**. Ini adalah agent yang dapat membantu menjawab pertanyaan, mencari SOP, membaca kebijakan, merangkum dokumen, dan menunjukkan sumber yang relevan. Nilai dari keluaran ini sangat besar karena banyak organisasi sebenarnya tidak kekurangan informasi, tetapi kekurangan **cara menemukan informasi yang benar pada saat dibutuhkan**. Open WebUI sendiri menjelaskan bahwa *Knowledge* adalah tempat menyimpan file dan koleksi yang dapat dicari, dibaca, dan dinalar oleh AI, dan bahwa dokumen bisa dicari berdasarkan makna, bukan hanya kata kunci, serta dapat diberi sitasi. Itu membuat RAG sangat cocok untuk fungsi seperti asisten SOP, kebijakan, FAQ internal, kontrak, dan dokumen proyek. ([Open WebUI](#))

Bagi pembaca, keluaran ini berarti:

- mampu menempelkan dokumen kerja ke basis pengetahuan
- mampu membangun model preset untuk kebutuhan tertentu
- mampu membuat AI menjawab dengan dasar dokumen, bukan tebakan
- dan mampu menjaga jawaban tetap bisa ditelusuri

Ini adalah langkah pertama yang sangat penting, karena banyak *agentic AI* bisnis yang baik justru bermula dari **pengetahuan yang tertib**, bukan dari otonomi yang terlalu agresif.

2. Agent workflow untuk laporan, tiket, approval, dan tindak lanjut

Hasil kedua yang dibangun pembaca adalah **agent workflow**, yaitu agent yang bukan hanya bisa menjawab, tetapi juga **ikut masuk ke alur kerja nyata**. Dalam buku ini, itu berarti agent dapat dipakai untuk:

- membantu ringkasan laporan,
- melakukan triase tiket,
- membaca konteks approval,
- menyiapkan langkah awal,
- mengarahkan dokumen,
- dan menjaga tindak lanjut tetap bergerak.

Ini sangat penting karena bisnis tidak hidup dari percakapan saja. Bisnis hidup dari **pekerjaan yang selesai**. Dokumentasi n8n menekankan pendekatan *self-hosted AI* dan *workflow automation* yang dapat dipakai untuk menggabungkan model, tools, trigger, dan proses bisnis yang nyata. Artinya, pembaca tidak berhenti di level “chat dengan AI”, tetapi naik ke level “AI ikut bekerja di dalam proses”. ([n8n Docs](#))

Dalam praktiknya, hasil ini akan tampak seperti:

- **agent triase helpdesk internal**
- **agent ringkasan rapat dan memo**
- **agent pengecekan kelengkapan dokumen**
- **agent routing approval**
- **agent eskalasi awal untuk kasus yang butuh perhatian cepat**

Inilah titik ketika pembaca mulai merasakan bahwa *agentic AI* bukan sekadar fitur tambahan, tetapi benar-benar berubah menjadi **mesin kerja digital** yang membantu organisasi bergerak lebih rapi.

3. Agent analisis proses ringan berbasis event log dan process mining

Hasil ketiga yang dibangun pembaca adalah kemampuan membuat **agent analisis proses ringan**. Ini mungkin terdengar lebih maju, tetapi sebenarnya sangat logis. Setelah pembaca mampu membangun agent untuk pengetahuan dan *workflow*, langkah berikutnya adalah memakai AI dan data proses untuk **membaca alur kerja yang sedang berjalan**: di mana bottleneck muncul, di mana *rework* berulang, di mana approval terlalu lama, dan di mana peluang otomasi atau dukungan agent paling masuk akal.

Di sinilah *event log* dan *process mining* menjadi sangat relevan. Situs PM4Py menjelaskan bahwa pustaka ini mendukung **process discovery**, **conformance checking**, **performance analysis**, **variant analysis**, *organizational mining*, dan teknik terkait. Ini membuat PM4Py sangat cocok sebagai jembatan dari jejak proses sederhana—misalnya CSV hasil tiket atau approval—menuju *insight* yang bisa dipakai untuk memilih intervensi agentic AI yang paling bernilai. ([Process Intelligence Solutions](#))

Dengan kata lain, pembaca tidak hanya membangun agent untuk bekerja, tetapi juga mulai membangun **cara berpikir berbasis proses**:

- melihat jejak kerja sebagai data
- menemukan bottleneck
- membaca *rework* dan deviasi
- lalu menghubungkannya ke use case agent yang lebih tajam

Ini penting karena agentic AI terbaik biasanya lahir bukan dari imajinasi liar, tetapi dari **pengamatan yang jujur terhadap proses yang benar-benar terjadi**.

4. Mini-stack lokal yang bisa diuji di lingkungan bisnis kecil–menengah

Hasil keempat, dan mungkin yang paling praktis, adalah pembaca memiliki **mini-stack lokal** yang benar-benar bisa diuji. Bukan sekadar ide, bukan sekadar arsitektur di slide, tetapi susunan sistem yang cukup untuk bekerja di lingkungan bisnis kecil–menengah. Dalam konteks buku ini, *mini-stack* itu dibangun dari:

- **Ubuntu 26.04** sebagai host sasaran
- **GPU NVIDIA** bila tersedia untuk akselerasi
- **Docker / Docker Compose** untuk pengemasan
- **Ollama** sebagai mesin model lokal
- **Open WebUI** sebagai workspace manusia dan front-end knowledge
- **OpenClaw dan/atau n8n** untuk runtime agent dan orkestrasi workflow
- **PM4Py** untuk eksperimen *process mining* ringan

Dokumentasi Linux Ollama memang menyediakan jalur instalasi Linux untuk model lokal, sementara Open WebUI menjelaskan bahwa platform ini mendukung dokumen, koleksi, dan *agentic retrieval* di atas *knowledge base*. n8n menyediakan jalur *self-hosted* yang realistis untuk *workflow* bisnis, dan PM4Py memberi perangkat analitik proses berbasis Python untuk membaca log yang sederhana sekalipun. Itu membuat stack ini cukup kuat untuk diuji di:

- unit bisnis kecil
- tim operasional
- laboratorium kampus
- organisasi menengah yang ingin belajar sambil bekerja
- atau pilot internal yang belum ingin membeli platform besar ([Open WebUI](#))

Yang sesungguhnya dibangun pembaca

Kalau diringkas lebih tajam, pembaca sebenarnya membangun **empat lapis kemampuan** sekaligus:

- **lapis pengetahuan**
AI dapat mencari dan memakai dokumen kerja yang relevan
- **lapis tindakan**
AI dapat membantu pekerjaan nyata di dalam workflow
- **lapis proses**
AI dan data membantu membaca alur kerja dan bottleneck
- **lapis kemandirian teknis**
semua itu dapat dijalankan secara lokal, terbuka, dan bertahap

Di sinilah letak nilai strategis buku ini. Pembaca tidak hanya diajak “menggunakan AI”, tetapi diajak **membangun pondasi kerja digital yang lebih tertib, lebih murah, lebih transparan, dan lebih bisa dikendalikan.**

Pesan akhir dari peta ini

Peta hasil akhir ini sengaja dibuat sederhana agar pembaca tidak tersesat. Buku ini tidak menjanjikan bahwa dalam beberapa hari semua proses bisnis akan otomatis sempurna. Yang dijanjikan lebih jujur dan lebih berguna: **pembaca akan mampu membangun satu fondasi yang nyata**, memakainya untuk menyelesaikan masalah kerja yang masuk akal, lalu memperluasnya secara bertahap jika hasilnya memang layak. Itulah sebabnya hasil akhir buku ini tidak berhenti pada satu chatbot atau satu model, tetapi bergerak menuju **asisten pengetahuan, agent workflow, analisis proses, dan mini-stack lokal yang bisa diuji.**

Dengan peta seperti ini, pembaca sejak awal tahu bahwa tujuan akhirnya bukan sekadar “punya AI”, melainkan **punya AI yang bisa bekerja, bisa dijelaskan, dan bisa dipakai untuk membuat organisasi bergerak lebih baik.**

Daftar Isi

Kata Pengantar.....	3
Cara Menggunakan Buku Ini.....	4
Untuk siapa buku ini.....	4
Urutan membaca untuk pembaca umum.....	5
Urutan membaca untuk pembaca teknis.....	5
Cara membaca buku ini agar tidak tersesat.....	6
Apa yang dibangun pembaca dalam buku ini.....	6
Pesan akhir untuk pembaca.....	7
Peta Hasil Akhir.....	8
1. Asisten pengetahuan internal berbasis RAG.....	8
2. Agent workflow untuk laporan, tiket, approval, dan tindak lanjut.....	9
3. Agent analisis proses ringan berbasis event log dan process mining.....	9
4. Mini-stack lokal yang bisa diuji di lingkungan bisnis kecil–menengah.....	10
Yang sesungguhnya dibangun pembaca.....	11
Pesan akhir dari peta ini.....	11
Daftar Isi.....	12
Bagian I. Mengapa Agentic AI, dan Mengapa Bisnis Perlu Sekarang.....	22
Bab 1. Agentic AI untuk Bisnis: bukan chatbot, tetapi mesin kerja.....	22
1.1 Apa itu agent, apa itu agentic system.....	22
1.2 Bedanya dengan chatbot, automasi biasa, dan RPA.....	23
1.3 Mengapa bisnis mulai butuh agent sekarang.....	24
1.4 Dimana agentic AI berguna, di mana tidak.....	24
1.5 Autonomy theater, hype, dan jebakan implementasi.....	25
1.6 Prinsip keras: mulai dari masalah bisnis, bukan dari model.....	26
Penutup bab.....	26
Bab 2. Nilai bisnis yang nyata: operasi, pengetahuan, layanan internal, dan keputusan manajerial.....	28
2.1 Pekerjaan yang penuh konteks dan sering berulang.....	28
2.2 Asisten SOP, kebijakan, dan knowledge worker.....	29
2.3 Ringkasan rapat, memo, laporan, dan tindak lanjut.....	30
2.4 Helpdesk internal, triase tiket, dan approval workflow.....	30
2.5 Dukungan keputusan operasional dan manajerial.....	31
2.6 KPI: waktu, biaya, akurasi, auditability, kepuasan pengguna.....	32
Penutup bab.....	33
Bagian II. Memilih Use Case dan Mendesain Arsitektur yang Waras.....	35
Bab 3. Memilih use case bisnis yang masuk akal.....	35
3.1 Mulai dari rasa sakit proses, bukan dari demo.....	35
3.2 Kriteria memilih use case awal.....	36
3.3 Kapan cukup RPA atau skrip biasa.....	37
3.4 Kapan agentic AI lebih unggul.....	37
3.5 Human-in-the-loop: kapan manusia tetap memegang tombol akhir.....	38
3.6 Hands-on: memetakan satu proses bisnis menjadi goal, tools, memory, approval gate.....	39

Langkah 1: tentukan goal.....	39
Langkah 2: tentukan tools.....	40
Langkah 3: tentukan memory.....	40
Langkah 4: tentukan approval gate.....	41
Bentuk peta sederhananya.....	41
Penutup bab.....	41
Bab 4. Arsitektur minimum agentic system untuk bisnis.....	43
4.1 Model, memori, tools, workflow, evaluasi, guardrails.....	43
4.2 Single-agent vs multi-agent.....	44
4.3 OpenClaw vs n8n: pilih alat sesuai gaya kerja.....	45
4.4 Peran Open WebUI sebagai workspace dan knowledge front-end.....	46
4.5 Logging, audit trail, session state, dan observability.....	47
4.6 Arsitektur minimum yang ringan, murah, dan bisa dirawat.....	48
Penutup bab.....	49
Bagian III. Menyiapkan Ekosistem di Ubuntu 26.04.....	50
Bab 5. Menyiapkan host: Ubuntu 26.04, GPU NVIDIA, Docker.....	50
5.1 Kenapa Ubuntu 26.04 dijadikan target platform.....	50
5.2 Catatan realitas: status versi dan kehati-hatian produksi.....	50
5.3 Hardware minimum dan rekomendasi GPU NVIDIA.....	51
Minimum yang masih masuk akal untuk belajar.....	51
Rekomendasi yang lebih sehat untuk praktik serius.....	52
Untuk laboratorium kecil atau pilot tim.....	52
5.4 Instalasi sistem, update, user, SSH, dan firewall.....	52
Tahap 1. Pasang Ubuntu Server atau Desktop sesuai kebutuhan.....	52
Tahap 2. Update sistem segera setelah login pertama.....	53
Tahap 3. Cek identitas host.....	53
Tahap 4. Buat user kerja terpisah bila perlu.....	54
Tahap 5. Pasang dan aktifkan OpenSSH.....	54
Tahap 6. Aktifkan firewall dasar.....	55
Tahap 7. Rapikan kebiasaan dasar.....	55
5.5 Driver NVIDIA, verifikasi nvidia-smi, dan dasar keamanan host.....	55
Tahap 1. Pastikan GPU NVIDIA terdeteksi.....	56
Tahap 2. Pasang driver NVIDIA melalui paket distro.....	56
Tahap 3. Verifikasi dengan nvidia-smi.....	56
Tahap 4. Pahami gejala umum bila belum sehat.....	57
Tahap 5. Dasar keamanan host.....	57
5.6 Instalasi Docker Engine dan Docker Compose.....	57
Tahap 1. Bersihkan paket konflik lama bila ada.....	58
Tahap 2. Pasang prasyarat repo.....	58
Tahap 3. Tambahkan GPG key dan repository Docker resmi.....	58
Tahap 4. Pasang Docker Engine dan Compose plugin.....	59
Tahap 5. Uji container dasar.....	59
Tahap 6. Opsi menjalankan Docker tanpa sudo.....	59
Tahap 7. Catatan menuju GPU container.....	60

Ringkasan langkah cepat bab ini.....	60
Penutup bab.....	60
Bab 6. Menyiapkan fondasi AI lokal: Ollama + Open WebUI.....	62
6.1 Instalasi Ollama di Linux.....	62
6.2 Menjalankan model lokal untuk tugas bisnis.....	63
6.3 Menjalankan Ollama sebagai service dan mengakses API lokal.....	64
6.4 Instalasi Open WebUI dengan Docker.....	66
6.5 Menghubungkan Open WebUI ke Ollama.....	67
Pola 1. Ollama dan Open WebUI di host yang sama.....	67
Pola 2. Satu container Open WebUI yang sudah dibundel dengan Ollama.....	68
6.6 Workspace, model preset, knowledge attachment, dan user access.....	69
a. Workspace sebagai ruang kerja tim.....	69
b. Model preset.....	69
c. Knowledge attachment.....	70
d. User access.....	70
Langkah ringkas dari nol sampai siap dipakai.....	71
Contoh skenario uji sederhana.....	71
Penutup bab.....	72
Bab 7. Menyiapkan runtime agent dan orkestrasi workflow: OpenClaw / n8n.....	73
7.1 Peran OpenClaw sebagai agent runtime.....	73
7.2 Peran n8n sebagai orkestrator workflow bisnis.....	73
7.3 Instalasi OpenClaw via installer resmi.....	74
7.4 Instalasi n8n via Docker Compose.....	76
Tahap 1. Buat direktori kerja.....	76
Tahap 2. Buat file .env.....	76
Tahap 3. Buat compose.yaml.....	77
Tahap 4. Jalankan Docker Compose.....	77
Tahap 5. Verifikasi container.....	78
7.5 Webhook, tools, skill, approval flow, dan scheduling.....	78
a. Webhook.....	78
b. Tools.....	78
c. Skill.....	79
d. Approval flow.....	79
e. Scheduling.....	79
7.6 Kapan memakai OpenClaw, kapan n8n, kapan dipadukan.....	80
Pakai OpenClaw ketika:.....	80
Pakai n8n ketika:.....	80
Padukan keduanya ketika:.....	80
Contoh step by step ringkas: dari nol sampai dua komponen hidup.....	81
Jalur OpenClaw.....	81
Jalur n8n.....	82
Penutup bab.....	82
Bagian IV. RAG, Knowledge, Dataset, dan Data Kerja Agent.....	83
Bab 8. RAG untuk bisnis: membangun otak eksternal yang bisa diaudit.....	83

8.1 Mengapa agent bisnis butuh RAG.....	83
8.2 Knowledge base di Open WebUI.....	84
Cara berpikir yang sehat.....	84
Langkah demi langkah menyiapkan knowledge base.....	84
Tahap 1. Siapkan folder sumber dokumen.....	84
Tahap 2. Masuk ke Open WebUI.....	85
Tahap 3. Buat collection baru.....	85
Tahap 4. Unggah file.....	85
Tahap 5. Tempelkan knowledge base ke model.....	85
Tahap 6. Uji pertanyaan nyata.....	86
Prinsip praktis yang perlu ditanamkan.....	86
8.3 Struktur dokumen yang baik untuk retrieval.....	86
Struktur yang disarankan.....	86
Dokumen yang kurang sehat.....	87
8.4 Chunking, metadata, embedding, reranking.....	87
a. Chunking.....	87
b. Metadata.....	88
c. Embedding.....	88
d. Reranking.....	89
8.5 Focused Retrieval vs Full Context.....	90
Kapan pakai Focused Retrieval.....	90
Kapan pakai Full Context.....	90
Cara berpikir yang sehat.....	91
8.6 Evaluasi RAG: relevance, faithfulness, latency, source traceability.....	91
a. Relevance.....	91
b. Faithfulness.....	92
c. Latency.....	92
d. Source traceability.....	93
Langkah evaluasi praktis.....	93
Tanda sistem RAG Anda sehat.....	93
Tanda sistem RAG Anda bermasalah.....	93
Hands-on lab: membuat knowledge base SOP, kebijakan, dan FAQ internal.....	93
Tahap 1. Siapkan tiga dokumen.....	93
Tahap 2. Buat embedding model tersedia.....	94
Tahap 3. Buka Open WebUI dan unggah dokumen.....	94
Tahap 4. Tempelkan knowledge base ke model preset.....	94
Tahap 5. Uji tiga pertanyaan.....	94
Tahap 6. Bandingkan dengan Full Context.....	95
Penutup bab.....	95
Bab 9. Dataset kerja, instruction data, dan data governance.....	96
9.1 SOP, kebijakan, tiket, email, laporan, notulen, dan katalog.....	96
Langkah demi langkah: inventarisasi bahan kerja.....	96
Tahap 1. Buat folder sumber berdasarkan jenis dokumen.....	96
Tahap 2. Isi contoh dokumen.....	96

Tahap 3. Cek isi folder.....	97
9.2 Membangun instruction dataset dari proses internal.....	97
Langkah demi langkah: membuat instruction dataset kecil.....	98
Tahap 1. Buat direktori dataset.....	98
Tahap 2. Tulis file JSONL.....	98
Tahap 3. Cek isi.....	98
9.3 Dataset evaluasi vs dataset training.....	99
Langkah demi langkah: memisahkan split.....	99
Tahap 1. Buat struktur file.....	99
Tahap 2. Simpan contoh terpisah.....	99
Tahap 3. Verifikasi file.....	100
9.4 Data sensitif, masking, minimisasi, dan hak akses.....	100
Empat prinsip praktis yang harus ditanamkan.....	100
Langkah demi langkah: masking sederhana.....	101
9.5 Kapan RAG cukup, kapan fine-tuning perlu, kapan hybrid masuk akal.....	101
Aturan praktis yang sangat sederhana.....	102
Cara berpikir yang sehat.....	102
9.6 Risiko data buruk: bias, drift, ilusi kualitas.....	103
a. Bias.....	103
b. Drift.....	103
c. Ilusi kualitas.....	103
Langkah sederhana untuk mengurangi risiko.....	103
Hands-on lab: membuat dataset kerja kecil yang rapi.....	104
Tahap 1. Buat direktori proyek data.....	104
Tahap 2. Siapkan contoh dokumen mentah.....	104
Tahap 3. Masking data sensitif.....	104
Tahap 4. Buat train split.....	104
Tahap 5. Buat eval split.....	104
Tahap 6. Verifikasi struktur.....	105
Penutup bab.....	105
Bab 10. Synthetic data bila data nyata belum cukup.....	106
10.1 Mengapa bisnis kadang perlu data sintetis.....	106
10.2 Teknik: self-instruct, pair generation, scenario expansion.....	106
a. Self-instruct.....	106
b. Pair generation.....	107
c. Scenario expansion.....	107
10.3 Membuat variasi kasus untuk helpdesk, triase, approval, dan laporan.....	108
a. Helpdesk.....	108
b. Triase tiket.....	108
c. Approval.....	108
d. Laporan.....	109
10.4 Memisahkan dataset training dari dataset evaluasi.....	109
Langkah praktis.....	109
10.5 Risiko synthetic data: bias, drift, overconfidence, rasa palsu.....	110

a. Bias.....	110
b. Drift.....	111
c. Overconfidence.....	111
d. Rasa palsu.....	111
10.6 Hands-on: membuat dataset kecil untuk satu use case bisnis nyata.....	111
Tahap 1. Siapkan satu benih kasus nyata.....	111
Tahap 2. Gunakan Ollama untuk membuat variasi skenario.....	111
Tahap 3. Ubah variasi menjadi pasangan instruksi.....	112
Tahap 4. Simpan ke file training.....	112
Tahap 5. Buat data evaluasi terpisah.....	113
Tahap 6. Periksa hasil.....	113
Tahap 7. Lakukan pemeriksaan manual.....	113
Pola kerja yang disarankan untuk pembaca.....	113
Penutup bab.....	114
Bagian V. Agentic AI untuk Process Management dan Process Mining.....	115
Bab 11. Dari process management ke agentic workflow.....	115
11.1 Mengapa proses bisnis sering lambat bukan karena orang bodoh, tapi karena alur kerja buruk.....	115
11.2 Event log, bottleneck, rework, exception, dan approval loop.....	116
11.3 Memetakan proses bisnis sebelum memasang agent.....	117
Langkah demi langkah: memetakan proses sederhana.....	118
11.4 Agent untuk membaca tiket, email, dokumen, dan notulen sebagai jejak proses..	119
11.5 Agent untuk rekomendasi langkah awal dan eskalasi.....	120
11.6 Hands-on: membangun workflow agent untuk satu proses manajemen internal...	121
Tahap 1. Siapkan payload contoh.....	122
Tahap 2. Buat workflow dasar di n8n.....	122
Tahap 3. Kirim request ke webhook.....	123
Tahap 4. Tambahkan human review untuk aksi sensitif.....	123
Tahap 5. Simpan hasil sebagai jejak proses.....	123
Ringkasan langkah kerja bab ini.....	124
Penutup bab.....	124
Bab 12. Process mining ringan dengan open source untuk menemukan peluang agentic AI.....	126
12.1 Apa itu process mining dan mengapa relevan.....	126
12.2 Process discovery, conformance checking, dan bottleneck analysis.....	127
a. Process discovery.....	127
b. Conformance checking.....	127
c. Bottleneck analysis.....	127
12.3 Mengubah log operasional menjadi insight proses.....	128
Langkah demi langkah: membuat event log sederhana.....	128
12.4 PM4Py sebagai pustaka open source untuk analisis proses.....	129
Langkah demi langkah: memasang PM4Py.....	129
Membaca log dengan PM4Py.....	130

12.5 Menghubungkan hasil process mining ke prioritas use case agentic AI.....	131
Contoh koneksi praktis.....	131
12.6 Hands-on: membaca event log sederhana dan menemukan kandidat otomasi / agent.....	132
Tahap 1. Siapkan event log.....	132
Tahap 2. Ringkas aktivitas per kasus dengan Python.....	132
Tahap 3. Hitung durasi sederhana antar event.....	132
Tahap 4. Tentukan kandidat intervensi agent.....	133
Tahap 5. Uji agent lokal untuk memberi saran.....	133
Ringkasan langkah kerja bab ini.....	134
Penutup bab.....	134
Bagian VI. Governance, Keamanan, dan Jalan ke Produksi.....	136
Bab 13. Guardrails, governance, dan observability.....	136
13.1 Mengapa agent tidak boleh dibiarkan liar.....	136
13.2 Threshold, escalation, approval workflow.....	136
13.3 Monitoring: task completion, latency, accuracy, cost.....	137
13.4 Logs, replay, audit trail, dan session state.....	139
13.5 Security minimum: secrets, least privilege, sandbox, policy checks.....	140
13.6 Prompt injection, data leakage, dan failure handling.....	141
Ringkasan langkah kerja bab ini.....	142
Penutup bab.....	143
Bab 14. Dari pilot ke produksi ringan: roadmap 90 hari.....	144
14.1 Memulai dari satu proses dan satu owner.....	144
14.2 Metrik keberhasilan: waktu, biaya, kualitas, trust.....	144
14.3 Runbook operasi dan checklist evaluasi.....	145
14.4 Kapan memperluas agent, kapan menahan diri.....	146
14.5 Struktur tim kecil untuk menjalankan agentic AI.....	147
14.6 Roadmap 90 hari: pilot → ukur → revisi → scale.....	148
Fase 1: Hari 1–30 — Pilot terkendali.....	148
Fase 2: Hari 31–60 — Ukur dan revisi.....	148
Fase 3: Hari 61–90 — Scale ringan.....	149
Ringkasan langkah kerja bab ini.....	149
Penutup bab.....	149
Lampiran A. CLI instalasi cepat Ubuntu 26.04 + NVIDIA + Docker + Ollama + Open WebUI + OpenClaw / n8n.....	151
A.1 Persiapan awal host.....	151
A.2 Instal driver NVIDIA dan verifikasi GPU.....	151
A.3 Instal Docker Engine + Compose Plugin.....	152
A.4 Instal Ollama.....	153
A.5 Jadikan Ollama sebagai service.....	154
A.6 Instal Open WebUI dengan Docker.....	155
A.7 Instal OpenClaw via installer resmi.....	156
A.8 Instal n8n via Docker Compose.....	157
A.9 Uji cepat seluruh stack.....	158

A.10 Urutan kerja yang disarankan.....	159
Lampiran B. Contoh docker-compose.yml minimum.....	160
Keterangan singkat.....	161
Cara menjalankan.....	161
Cara mengakses.....	162
Uji cepat.....	162
Uji Ollama.....	162
Uji PM4Py.....	163
Struktur folder yang disarankan.....	163
Catatan praktis.....	163
Versi lebih minimum tanpa GPU.....	164
Penutup singkat.....	164
Lampiran C. Contoh .env minimum yang direkomendasikan.....	165
C.1 Contoh .env minimum.....	165
C.2 Fungsi tiap bagian.....	167
1. Timezone.....	167
2. Port.....	167
3. Nama container.....	167
4. Variabel Ollama.....	167
5. Variabel Open WebUI.....	168
6. Variabel n8n.....	168
7. Direktori kerja.....	168
C.3 Contoh pemakaian di docker-compose.yml.....	169
C.4 Cara memeriksa .env sudah terbaca.....	170
C.5 Versi lebih pendek untuk pembaca yang ingin super-minimum.....	170
C.6 Catatan penting.....	171
Penutup singkat.....	171
Lampiran D. 30 Prompt Sistem Siap Pakai untuk Agent Bisnis.....	172
A. Prompt untuk pengetahuan, SOP, dan kebijakan.....	172
1. Asisten SOP internal.....	172
2. Asisten kebijakan internal.....	172
3. Asisten FAQ operasional.....	173
4. Asisten onboarding pegawai baru.....	173
5. Asisten dokumen proyek.....	173
B. Prompt untuk ringkasan, memo, dan laporan.....	173
6. Ringkasan rapat.....	173
7. Ringkasan notulen menjadi aksi.....	173
8. Penyusun memo internal.....	173
9. Ringkasan laporan manajerial.....	173
10. Pembuat executive summary.....	174
C. Prompt untuk helpdesk, tiket, dan triase.....	174
11. Triase tiket helpdesk.....	174
12. Klasifikasi insiden operasional.....	174
13. Asisten respons awal helpdesk.....	174

14. Penentu eskalasi tiket.....	174
15. Pemeriksa kelengkapan tiket.....	174
D. Prompt untuk approval, pemeriksaan dokumen, dan workflow.....	174
16. Pemeriksa kelengkapan dokumen approval.....	175
17. Ringkasan konteks approval.....	175
18. Agent routing approval.....	175
19. Agent pengecekan risiko dokumen.....	175
20. Agent tindak lanjut revisi.....	175
E. Prompt untuk keputusan operasional dan manajerial.....	175
21. Asisten keputusan operasional.....	175
22. Asisten briefing manajer.....	175
23. Penanda anomali proses.....	176
24. Pemberi rekomendasi langkah awal.....	176
25. Agent eskalasi manajerial.....	176
F. Prompt untuk quality control, keamanan, dan governance.....	176
26. Pemeriksa kesetiaan terhadap sumber.....	176
27. Pemeriksa data sensitif.....	176
28. Agent dengan batas aksi ketat.....	176
29. Pemeriksa kualitas output.....	176
30. Agent fallback ke manusia.....	177
Cara memilih prompt yang tepat.....	177
Cara memodifikasi agar lebih sesuai organisasi.....	177
Pola prompt sistem yang sehat.....	178
Penutup singkat.....	178
Lampiran E. Checklist RAG dan Governance.....	180
E.1 Checklist tujuan dan ruang lingkup.....	180
E.2 Checklist kualitas dokumen sumber.....	180
E.3 Checklist struktur knowledge base.....	181
E.4 Checklist retrieval dan mode konteks.....	181
E.5 Checklist evaluasi kualitas RAG.....	182
E.6 Checklist governance akses dan peran.....	183
E.7 Checklist security minimum.....	183
E.8 Checklist governance dokumen dan data sensitif.....	184
E.9 Checklist observability dan audit.....	184
E.10 Checklist sebelum go-live.....	184
Versi checklist super-singkat.....	185
Penutup singkat.....	185
Lampiran F. Template Event Log Sederhana untuk Process Mining.....	186
F.1 Struktur kolom minimum.....	186
F.2 Template paling sederhana.....	186
F.3 Template yang lebih kaya.....	187
F.4 Cara membuat file log sederhana.....	188
F.5 Template untuk beberapa jenis proses.....	189
1. Approval dokumen.....	189

2. Helpdesk internal.....	189
3. Ringkasan rapat dan tindak lanjut.....	189
F.6 Aturan penamaan aktivitas.....	189
F.7 Aturan timestamp.....	190
F.8 Template log dengan kolom durasi hasil proses.....	190
F.9 Pemeriksaan cepat kualitas event log.....	191
F.10 Contoh cek cepat dengan Python.....	191
F.11 Template JSONL alternatif.....	192
F.12 Penutup singkat.....	193
Lampiran G. Checklist evaluasi agent sebelum go-live.....	193
G.1 Checklist tujuan dan ruang lingkup.....	194
G.2 Checklist kualitas fungsi utama agent.....	194
G.3 Checklist kualitas RAG dan basis pengetahuan.....	195
G.4 Checklist guardrails dan batas aksi.....	195
G.5 Checklist keamanan minimum.....	196
G.6 Checklist logging, audit, dan observability.....	196
G.7 Checklist metrik operasi.....	196
G.8 Checklist kesiapan pengguna dan proses.....	197
G.9 Checklist runbook dan penanganan kegagalan.....	197
G.10 Checklist evaluasi kasus uji sebelum go-live.....	198
Kasus normal.....	198
Kasus ambigu.....	198
Kasus sensitif.....	198
Kasus gagal.....	198
Kasus volume lebih tinggi.....	198
G.11 Skor keputusan sederhana sebelum go-live.....	198
G.12 Versi checklist super-singkat.....	199
Penutup singkat.....	200
Glossary.....	201
Referensi.....	206
Tentang Penulis.....	209

Bagian I. Mengapa Agentic AI, dan Mengapa Bisnis Perlu Sekarang

Bab 1. Agentic AI untuk Bisnis: bukan chatbot, tetapi mesin kerja

1.1 Apa itu agent, apa itu agentic system

Banyak orang masih mengira bahwa *agentic AI* hanyalah chatbot yang dibuat lebih pintar. Pandangan ini terlalu dangkal. **Agent** bukan sekadar sistem yang menerima pertanyaan lalu memberi jawaban. Agent adalah entitas perangkat lunak yang diberi **tujuan**, memahami **konteks**, memilih **langkah**, menggunakan **alat**, lalu berusaha bergerak menuju hasil yang diminta. Ia tidak hanya “menjawab”; ia **bekerja**. Dalam dokumentasi resmi n8n, agent dijelaskan sebagai sistem otonom yang menerima data, membuat keputusan rasional, dan bertindak di lingkungannya untuk mencapai tujuan tertentu. Dokumentasi Open WebUI juga menunjukkan arah yang sama: model tidak hanya dipakai untuk chat, tetapi dapat diberi akses ke dokumen, pengetahuan, dan fungsi agar membantu pekerjaan nyata. ([n8n Docs](#))

Di atas agent, ada konsep yang lebih besar, yaitu **agentic system**. Ini bukan satu komponen tunggal, melainkan **keseluruhan ekosistem kerja** tempat agent beroperasi. Di dalamnya biasanya ada model bahasa, memori atau konteks, sumber pengetahuan, tools, aturan kerja, guardrails, catatan aktivitas, serta pengawasan manusia. Jadi, ketika kita membahas agentic AI dalam buku ini, kita tidak sedang membicarakan “AI yang lebih cerewet”. Kita sedang membahas **arsitektur kerja digital**. Disney ditempatkan model, memori, tools, workflow, evaluasi, dan guardrails sebagai fondasi minimum sebuah sistem agentic untuk bisnis.

Agar mudah dipahami, bayangkan tiga level berikut. Chatbot biasa itu seperti petugas informasi di meja depan: ia menjawab pertanyaan yang masuk. Agent itu seperti staf junior yang diberi mandat: baca dokumen, cari data, buat ringkasan, lalu siapkan langkah awal. Sementara *agentic system* adalah kantor lengkapnya: ada staf, SOP, arsip, jalur persetujuan, alat kerja, dan atasan yang mengawasi. **Perbedaan inilah yang menentukan apakah AI hanya menghibur, atau benar-benar memberi nilai bisnis.**

Ada beberapa ciri yang membuat sebuah sistem layak disebut agentic. Biasanya ia memiliki:

- **tujuan yang jelas**, bukan sekadar respons acak,
- **kemampuan memecah tugas** menjadi langkah-langkah,
- **akses ke tools atau sumber daya lain**,
- **konteks atau memori kerja**,
- **kemampuan menilai hasil** dan menyesuaikan langkah,
- serta **batasan dan pengawasan manusia** agar tidak bertindak liar.

Kalau unsur-unsur itu belum ada, sering kali kita belum berurusan dengan agentic AI, melainkan masih berada di level chatbot yang dibungkus istilah baru.

1.2 Bedanya dengan chatbot, automasi biasa, dan RPA

Di lapangan, empat istilah ini sering dicampuradukkan: chatbot, automasi biasa, RPA, dan agentic AI. Padahal keempatnya punya logika kerja yang berbeda.

Chatbot pada dasarnya bersifat *reactive*. Ia menunggu input, lalu menjawab. Ia berguna untuk FAQ, tanya-jawab dasar, atau bantuan cepat. Namun chatbot tidak otomatis mengerti prioritas kerja, tidak selalu punya akses ke sistem lain, dan biasanya tidak dirancang untuk merencanakan beberapa langkah ke depan. Ia kuat di permukaan interaksi, tetapi sering dangkal dalam eksekusi.

Automasi biasa bekerja dengan aturan eksplisit. Jika A terjadi, lakukan B. Jika formulir masuk, simpan ke folder. Jika email tertentu datang, kirim notifikasi. Logika ini sangat berguna karena sederhana, cepat, dan mudah dijelaskan. Tetapi ia rapuh saat berhadapan dengan variasi bahasa, ambiguitas, atau dokumen yang bentuknya tidak seragam.

RPA atau *robotic process automation* melangkah lebih jauh. Ia meniru langkah-langkah manusia pada aplikasi: klik tombol, salin data, pindah kolom, buka layar berikutnya. Di banyak organisasi, RPA berhasil mempercepat pekerjaan administratif. Tetapi RPA tetap sangat bergantung pada struktur layar dan aturan yang relatif stabil. Saat konteks menjadi kabur, dokumen menjadi tidak rapi, atau kasus perlu penalaran, RPA mulai kewalahan.

Agentic AI berada di level berbeda. Ia tidak menggantikan semua yang lain, tetapi menambahkan lapisan baru: **pemahaman konteks, perencanaan langkah, penggunaan tools, dan penyesuaian berdasarkan hasil**. Karena itu agentic AI cocok untuk pekerjaan yang berada di wilayah antara “murni manual” dan “murni rule-based”. Misalnya: membaca email masuk, mengklasifikasikan urgensi, menarik pengetahuan dari SOP, menyiapkan draf respons, lalu mengirim ke orang yang tepat untuk persetujuan. Di sini agent tidak sekadar memindahkan data. Ia membantu **membaca situasi**. ([n8n Docs](#))

Yang penting dipahami: **agentic AI bukan pengganti otomatis untuk RPA atau automasi biasa**. Untuk tugas yang sangat stabil, berulang, dan deterministik, automasi klasik sering lebih murah dan lebih andal. Justru kesalahan banyak organisasi adalah memaksa agentic AI ke area yang sebenarnya bisa diselesaikan dengan skrip sederhana. Hasilnya mahal, lambat, dan membingungkan.

1.3 Mengapa bisnis mulai butuh agent sekarang

Ada beberapa alasan mengapa kebutuhan terhadap agent mulai terasa nyata sekarang, bukan lima tahun lalu.

Pertama, pekerjaan bisnis modern makin penuh **teks, konteks, dan pengecualian**. Banyak keputusan operasional tidak lahir dari angka saja, tetapi dari email, memo, notulen, tiket, kontrak, kebijakan, percakapan, dan lampiran. Ini adalah wilayah yang sulit ditangani otomatisasi lama, tetapi mulai bisa disentuh oleh model bahasa modern yang dipadukan dengan pengetahuan dan tools.

Kedua, organisasi sekarang menghadapi **banjir informasi**. Dokumen bertambah, SOP berubah, kebijakan direvisi, dan orang sering tidak tahu versi mana yang paling tepat. Di sinilah kombinasi model lokal dan RAG menjadi menarik. Open WebUI, misalnya, mendokumentasikan fitur *Knowledge* dan *RAG* untuk membuat dokumen bisa dicari secara semantik, dengan mode *Focused Retrieval* untuk koleksi besar dan *Full Context* untuk kebutuhan presisi tinggi. Fitur sitasi pada RAG juga membantu transparansi sumber jawaban. ([Open WebUI](#))

Ketiga, bisnis mulai sadar bahwa AI yang hanya berhenti pada “membantu menulis” nilainya terbatas. Nilai yang lebih besar muncul saat AI ikut masuk ke **workflow**: membaca konteks, memilih langkah, memanggil tools, dan mengirim hasil ke jalur kerja yang benar. Dokumentasi n8n jelas menunjukkan arah ini dengan AI Agent yang bekerja bersama tool-node lain di dalam workflow bisnis. Sementara OpenClaw menempatkan dirinya sebagai runtime agent yang bisa dipasang dan di-*onboard* untuk menjalankan asistensi yang lebih aktif. ([n8n Docs](#))

Keempat, kini tersedia jalur *open source* dan *self-hosted* yang lebih realistis. **Ollama** menyediakan model lokal di Linux; **Open WebUI** menyediakan antarmuka *self-hosted* yang dapat beroperasi offline; **n8n** menggabungkan otomasi proses dengan kemampuan AI; dan **OpenClaw** memberi jalur agent runtime yang lebih eksplisit. Ini membuat organisasi kecil dan menengah tidak harus menunggu platform raksasa untuk mulai bereksperimen. Mereka bisa membangun **agen kerja lokal** dengan biaya dan kontrol yang lebih masuk akal. ([Ollama](#))

1.4 Dimana agentic AI berguna, di mana tidak

Agentic AI sangat berguna ketika pekerjaan memiliki empat ciri sekaligus: **banyak konteks, banyak pengecualian, butuh akses ke pengetahuan, dan perlu tindak lanjut yang terstruktur**. Misalnya:

- triase email atau tiket,
- asisten SOP dan kebijakan internal,
- ringkasan rapat dan tindak lanjut,
- klasifikasi dokumen dan antrian persetujuan,
- penyusunan memo awal,
- analisis awal bottleneck proses dari catatan kerja.

Dalam kasus semacam ini, agent membantu manusia bekerja lebih cepat tanpa harus berpura-pura menjadi “pengganti total”.

Namun ada juga wilayah yang **tidak cocok**. Agentic AI kurang tepat untuk:

- proses yang sepenuhnya deterministik dan stabil,
- keputusan yang sangat sensitif tanpa data dan guardrails yang kuat,
- tugas real-time yang menuntut kepastian absolut,
- proses yang buruk tetapi belum dibenahi,
- organisasi yang belum punya data, SOP, atau pemilik proses yang jelas.

Di sinilah banyak implementasi gagal. Orang berharap agent menyelesaikan kekacauan organisasi, padahal ia justru memperbesar kekacauan itu. **AI tidak secara ajaib memperbaiki proses yang memang rusak**. Ia hanya mempercepat apa yang sudah ada—termasuk kesalahan, kebingungan, dan inkonsistensi.

Sikap paling sehat adalah melihat agentic AI sebagai **penguat proses yang sudah mulai masuk akal**, bukan penyelamat universal.

1.5 Autonomy theater, hype, dan jebakan implementasi

Salah satu masalah terbesar dalam gelombang AI hari ini adalah **autonomy theater**: system terlihat canggih, tampak mandiri, tampil meyakinkan di demo, tetapi sebenarnya rapuh, tidak dapat diaudit, dan sulit diandalkan di pekerjaan nyata. Open WebUI sendiri, dalam dokumentasi RAG-nya, mengingatkan bahwa ketika sistem “halusinasi” atau melewati informasi penting, penyebabnya sering bukan pada model semata, melainkan pada masalah konteks dan retrieval. Ini pelajaran penting: banyak demo AI gagal bukan karena modelnya bodoh, tetapi karena arsitekturnya ceroboh. ([Open WebUI](#))

Jebakan implementasi biasanya datang dalam bentuk seperti ini:

- terlalu cepat mengejar **multi-agent** padahal satu agent saja belum stabil,
- terlalu sibuk membahas model terbaru padahal knowledge base berantakan,
- memasang AI pada proses yang tidak punya owner,
- mengejar otonomi penuh tanpa jalur eskalasi manusia,
- mengukur keberhasilan dari “kesan wow”, bukan dari KPI kerja.

Di banyak organisasi, masalah utamanya bukan kurang AI, melainkan **kurang disiplin desain**. Mereka ingin hasil besar tanpa mau mengerjakan hal yang membosankan: membenahi dokumen, mendefinisikan proses, memberi batas aksi, menyiapkan evaluasi, dan menetapkan siapa yang bertanggung jawab ketika agent salah.

Karena itu buku ini perlu bersikap kritis. **Agentic AI memang menjanjikan, tetapi janji itu hanya menjadi nilai bila ditopang disiplin operasional**. Tanpa itu, organisasi hanya membeli ilusi.

1.6 Prinsip keras: mulai dari masalah bisnis, bukan dari model

Ini prinsip paling penting dalam seluruh buku: **mulailah dari masalah bisnis, bukan dari model**. Jangan mulai dengan pertanyaan, “Model apa yang paling hebat?” Mulailah dengan pertanyaan, “Masalah proses apa yang paling menyita waktu, paling sering berulang, paling penuh konteks, dan paling layak dibantu mesin?”

Urutan berpikir yang sehat biasanya seperti ini. Pertama, temukan pekerjaan yang nyata: tiket lambat diprioritaskan, notulen rapat tidak pernah berubah menjadi tindak lanjut, SOP sulit dicari, atau persetujuan dokumen terlalu lama. Kedua, petakan alur kerjanya: siapa yang menerima input, siapa yang memutuskan, dokumen apa yang dipakai, dan titik mana yang paling sering macet. Ketiga, baru tentukan peran agent: apakah sebagai pembaca konteks, peringkas, pengklasifikasi, penyusun draf, pencari pengetahuan, atau penggerak workflow. Baru setelah itu kita bicara model, tools, dan platform.

Dengan cara berpikir ini, organisasi tidak akan terjebak pada perlombaan kosmetik. Mereka akan membangun agent yang sederhana tetapi berguna. Dan itulah sasaran buku ini: bukan memuja model, melainkan **membangun mesin kerja digital yang masuk akal**.

Secara praktis, ada lima pertanyaan sederhana sebelum memulai:

- **masalah apa yang ingin dipercepat atau dikurangi?**
- **siapa pemilik prosesnya?**
- **data dan dokumen apa yang dipakai?**
- **aksi apa yang boleh dilakukan agent, dan apa yang harus tetap disetujui manusia?**
- **bagaimana kita mengukur keberhasilannya?**

Kalau lima pertanyaan ini belum terjawab, biasanya implementasi masih terlalu dini. Tetapi kalau jawabannya sudah cukup jelas, maka agentic AI bisa menjadi alat yang sangat kuat: bukan untuk mengganti manusia, melainkan untuk **mengurangi beban mekanis, mempercepat pemahaman, dan membuat keputusan operasional awal menjadi lebih tertib.**

Penutup bab

Bab ini harus menanamkan satu gagasan utama: **Agentic AI untuk bisnis bukan soal membuat AI terdengar lebih pintar, tetapi membuat pekerjaan menjadi lebih tertata, lebih cepat, dan lebih bisa diaudit.** Chatbot boleh membantu percakapan, tetapi bisnis membutuhkan lebih dari percakapan. Bisnis membutuhkan sistem yang bisa memahami konteks, mengambil pengetahuan yang relevan, menyiapkan langkah, memakai alat, dan tetap berada dalam pagar kontrol yang sehat. Itulah alasan mengapa fondasi bab ini penting, dan itulah sebabnya tepat ketika sejak awal menempatkan agentic AI sebagai **sistem kerja**, bukan pertunjukan kata-kata.

Bila diibaratkan singkat: **chatbot itu bicara, agent itu bekerja, dan agentic system itu mengatur cara kerja agar hasilnya bisa dipercaya.**

Bab 2. Nilai bisnis yang nyata: operasi, pengetahuan, layanan internal, dan keputusan manajerial

2.1 Pekerjaan yang penuh konteks dan sering berulang

Banyak pekerjaan di organisasi tidak sulit karena rumusnya rumit, tetapi karena **konteksnya banyak**. Email harus dibaca dulu sebelum dipahami. Tiket harus dilihat urgensinya. Dokumen harus dicocokkan dengan kebijakan. Laporan harus dibandingkan dengan target, lalu dikaitkan dengan kejadian di lapangan. Semua ini bukan pekerjaan yang sepenuhnya mekanis, tetapi juga bukan pekerjaan strategis tingkat tinggi. Inilah wilayah yang paling cocok untuk agentic AI.

Pekerjaan semacam ini biasanya memiliki pola yang mirip. Ia:

- sering berulang,
- memakan waktu karena harus membaca banyak teks,
- penuh pengecualian,
- membutuhkan akses ke dokumen atau aturan,
- dan tetap memerlukan manusia untuk keputusan akhir.

Di sinilah **nilai bisnis nyata** mulai muncul. Agentic AI tidak harus mengambil alih seluruh proses. Kadang cukup membantu di bagian yang paling melelahkan: membaca, memilah, merangkum, menghubungkan, dan menyiapkan langkah awal. Dalam dokumentasi n8n, *AI Agent* dijelaskan sebagai sistem yang menerima data, membuat keputusan rasional, dan bertindak di lingkungannya untuk mencapai tujuan tertentu, dengan bantuan tools dan API bila diperlukan. Itu sangat cocok untuk pekerjaan kantor yang padat konteks tetapi berulang. ([n8n Docs](#))

Contoh yang mudah dibayangkan adalah penyusunan laporan bulanan. Secara teori, ini tampak sederhana. Namun di lapangan, staf sering harus menarik data dari beberapa sumber, memeriksa ketidaksesuaian, membaca catatan tambahan, menginterpretasi anomali, menyusun narasi, lalu mengirim hasilnya ke pemilik laporan. Literatur yang dilampirkan bahkan memberi contoh agent yang dapat menyiapkan laporan performa bulanan dengan menarik data dari beberapa sumber, menyesuaikan diri bila ada dashboard yang gagal, dan tetap menjaga pemilik laporan tetap berada dalam *loop*.

Pelajaran pentingnya sederhana: **nilai agentic AI paling cepat muncul pada pekerjaan yang “membosankan tetapi tidak sepenuhnya bisa di otomasi secara kaku.”** Di situlah ia lebih berguna daripada chatbot biasa, tetapi juga tidak perlu dipaksa menjadi “AI super” yang mengendalikan semuanya.

2.2 Asisten SOP, kebijakan, dan knowledge worker

Salah satu masalah klasik di organisasi adalah pengetahuan ada, tetapi **sulit dipakai saat dibutuhkan**. SOP tersimpan di folder. Kebijakan ada di PDF. FAQ internal tersebar di email lama. Notulen rapat mengandung keputusan penting, tetapi tidak pernah benar-benar hidup sebagai pengetahuan organisasi. Akibatnya, orang terus bertanya hal yang sama, mengulang kesalahan yang sama, dan mencari file yang sama.

Di sini agentic AI bisa menjadi **asisten pengetahuan internal**. Bukan sekadar menjawab pertanyaan, tetapi membantu pekerja pengetahuan menemukan jawaban yang benar, menelusuri sumbernya, dan menyajikannya dalam bentuk yang langsung berguna. Ini sangat selaras dengan arah bab-bab berikutnya yang menekankan **RAG**, *knowledge base*, dan auditabilitas.

Dokumentasi Open WebUI cukup jelas dalam hal ini. Fitur *Knowledge* membuat dokumen menjadi dapat dicari secara semantik, bukan hanya berdasarkan kata kunci. Pengguna dapat memilih *Focused Retrieval* untuk koleksi dokumen yang besar, atau *Full Context* ketika presisi dokumen utuh lebih penting. Open WebUI juga menekankan bahwa dokumen dapat disitasi, sehingga jawaban tidak melayang tanpa sumber. ([Open WebUI](#))

Secara bisnis, manfaatnya besar. **Asisten SOP dan kebijakan** dapat dipakai untuk:

- menjawab pertanyaan staf tentang prosedur kerja,
- membantu tim baru memahami kebijakan,
- mengurangi waktu pencarian dokumen,
- mengurangi jawaban asal-asalan,
- dan mempercepat *onboarding*.

Untuk *knowledge worker*, nilai tambahnya lebih menarik lagi. Seorang analis, supervisor, atau manajer tidak hanya perlu “jawaban”, tetapi perlu **jawaban yang bisa ditelusuri**. Karena itu pendekatan *RAG* jauh lebih cocok untuk banyak kebutuhan bisnis dibanding sekedar bertanya ke model umum. Literatur enterprise yang dilampirkan menegaskan bahwa *RAG* kuat karena memberi **dynamic knowledge updating**, **factual grounding**, dan **source transparency**. Artinya, pengetahuan dapat diperbarui tanpa melatih ulang model, jawaban lebih tertambat pada dokumen, dan sumber bisa ditelusuri.

Yang perlu ditegaskan kepada pembaca: **asisten pengetahuan bukan mesin kebenaran mutlak**. Ia adalah alat kerja yang baik bila dokumennya rapi, sumbernya jelas, dan penggunaanya disiplin memeriksa konteks. Namun bila dokumennya kacau, versi SOP tumpang tindih, atau kebijakan tidak pernah dibersihkan, agent hanya akan mempercepat kebingungan.

2.3 Ringkasan rapat, memo, laporan, dan tindak lanjut

Banyak organisasi mengira masalah utama mereka adalah kurang rapat. Padahal sering justru sebaliknya: rapat terlalu banyak, tetapi **hasilnya tidak bergerak**. Notulen dibuat, lalu tenggelam. Memo ditulis, tetapi tidak menjadi aksi. Laporan selesai, tetapi tidak ditindaklanjuti. Ini wilayah yang sangat menarik untuk agentic AI, karena masalahnya bukan sekadar menulis, melainkan **mengubah informasi menjadi alur tindak lanjut**.

Di sini agent dapat berperan sebagai:

- peringkas hasil rapat,
- pengelompok keputusan,
- pembuat daftar aksi,
- pengingat penanggung jawab,
- penyusun memo awal,
- dan penghubung antara catatan kerja dengan alur persetujuan.

Inilah titik pembeda antara AI generatif biasa dan agentic AI. Model generatif bisa membuat ringkasan rapat yang tampak bagus. Tetapi agentic AI bisa melangkah lebih jauh: menyusun ringkasan, memetakan tindakan, mengaitkannya ke PIC, menyiapkan draf email tindak lanjut, lalu memasukkannya ke *workflow* yang benar. Literatur yang dilampirkan menekankan bahwa agentic AI cocok untuk lingkungan kerja yang melibatkan banyak aplikasi dan ketergantungan yang tidak selalu dapat diprediksi. Dalam situasi seperti itu, agent tidak hanya menjawab, tetapi **menjaga pekerjaan tetap berjalan menuju tujuan**.

Bagi pembaca mahasiswa dan praktisi, bagian ini penting karena terasa sangat dekat. Hampir semua orang pernah mengalami:

- rapat yang panjang tetapi hasilnya kabur,
- laporan yang isinya baik tetapi terlambat,
- memo yang terlalu normatif,
- atau tindak lanjut yang hilang di tengah jalan.

Agentic AI bisa membantu memecah masalah ini menjadi alur yang lebih rapi. Namun buku ini perlu tetap kritis: **jangan menyamakan ringkasan yang rapi dengan keputusan yang baik**. Ringkasan membantu, tetapi kualitas keputusan tetap bergantung pada manusia, kualitas data, dan kejelasan proses.

2.4 Helpdesk internal, triase tiket, dan approval workflow

Layanan internal adalah salah satu ladang paling subur bagi agentic AI. Mengapa? Karena banyak pekerjaan helpdesk dan layanan internal memiliki campuran yang ideal: **volume cukup tinggi, pola berulang, konteks penting, dan kebutuhan eskalasi yang jelas.**

Pada helpdesk internal, agent dapat membantu:

- membaca tiket masuk,
- mengelompokkan kategori masalah,
- menentukan prioritas awal,
- mengambil referensi dari FAQ atau SOP,
- menyiapkan respons awal,
- dan meneruskan kasus ke pihak yang tepat.

Literatur enterprise yang dilampirkan memberi contoh kuat bahwa agentic AI sudah dipakai untuk layanan internal dan operasi *back-office*. Ada studi kasus yang menyebut agent layanan TI internal mampu menurunkan *support ticket deflection*, meningkatkan akurasi respons, dan memangkas latensi; contoh lain menunjukkan otomasi alur kerja internal pada fungsi HR dan keuangan dengan akurasi proses yang sangat tinggi. Terlepas dari angka masing-masing vendor yang tentu harus dibaca hati-hati, pesan utamanya jelas: **internal service delivery** adalah salah satu arena paling realistis untuk agentic AI.

Untuk *approval workflow*, manfaatnya juga nyata. Banyak organisasi lambat bukan karena orang malas, tetapi karena **alur persetujuannya berkabut**. Dokumen masuk, tetapi tidak jelas siapa yang harus melihat duluan. Email terkirim, tetapi tidak ada batas waktu. Revisi berputar tanpa kontrol versi. Agentic AI bisa membantu menata tahap awal: membaca konteks, memeriksa kelengkapan, mengklasifikasi tingkat risiko, lalu mengarahkan dokumen ke jalur yang benar.

Di sinilah kombinasi dengan n8n menjadi relevan. Dokumentasi resmi n8n menempatkan *AI Agent* sebagai node yang bekerja bersama tools lain di dalam alur kerja, bukan sebagai fitur yang berdiri sendiri. Artinya, agent bisa dipakai bukan hanya untuk memahami isi tiket atau dokumen, tetapi juga untuk **memicu langkah berikutnya** dalam proses. ([n8n Docs](#))

Namun bab ini juga perlu memberi peringatan penting. Untuk helpdesk dan approval, **batas otonomi harus jelas**. Tidak semua tiket boleh diputuskan otomatis. Tidak semua dokumen boleh disetujui oleh agent. Untuk kasus ambigu, sensitif, atau berisiko tinggi, manusia tetap harus memegang tombol akhir. Ini sejalan dengan literatur yang menekankan pentingnya *human-in-the-loop*, jalur eskalasi, dan batas keputusan yang eksplisit.

2.5 Dukungan keputusan operasional dan manajerial

Salah satu godaan terbesar dalam AI adalah menjual mimpi bahwa mesin akan mengambil semua keputusan lebih baik daripada manusia. Pandangan ini berlebihan. Tetapi sebaliknya, terlalu kecil juga bila AI hanya dipandang sebagai “alat bantu menulis.” Posisi yang lebih sehat adalah ini: **agentic AI paling berguna sebagai pendukung keputusan**, terutama untuk keputusan operasional dan manajerial yang padat data, padat dokumen, dan harus diambil cukup cepat.

Literatur enterprise yang dilampirkan secara eksplisit menyebut bahwa AI akan semakin berperan dalam **AI-augmented decision-making**, dari perencanaan strategis hingga operasi harian, karena kemampuannya menganalisis data dalam jumlah besar secara cepat.

Dalam praktik bisnis, dukungan keputusan ini bisa berbentuk:

- merangkum kondisi operasional harian,
- menyoroti anomali,
- membandingkan realisasi terhadap target,
- menghubungkan masalah dengan SOP atau kebijakan,
- menyiapkan opsi rekomendasi,
- dan menandai mana yang perlu perhatian pimpinan.

Yang menarik, agentic AI tidak harus selalu “lebih pintar” dari manajer. Cukup bila ia **lebih konsisten** dalam membaca informasi, **lebih cepat** dalam menyiapkan alternatif, dan **lebih disiplin** dalam menelusuri sumber. Nilai tambah ini sering kali sudah sangat besar.

Bagi manajer, agent yang baik bukan agent yang sok memutuskan semuanya. Agent yang baik adalah agent yang:

- menyiapkan konteks dengan cepat,
- menunjukkan sumber,
- menjelaskan alasan,
- memberi opsi,
- dan tahu kapan harus berhenti.

Di titik ini, agentic AI menjadi semacam “kepala staf digital” yang membantu pimpinan berpikir lebih jernih, bukan menggantikan penilaian pimpinan itu sendiri.

Tetapi ada satu batas penting. **Keputusan buruk tidak akan menjadi baik hanya karena dibantu AI.** Jika datanya jelek, targetnya kabur, atau proses organisasinya sudah penuh konflik, agent hanya akan mempercepat produksi rekomendasi yang tampak rapi tetapi rapuh. Karena itu dukungan keputusan harus selalu dipasangkan dengan kualitas data, kejelasan proses, dan pengawasan manusia.

2.6 KPI: waktu, biaya, akurasi, auditability, kepuasan pengguna

Bab tentang nilai bisnis akan terasa kosong bila tidak diakhiri dengan cara mengukurnya. Banyak proyek AI gagal bukan karena teknologinya buruk, tetapi karena **organisasi tidak sepakat tentang arti “berhasil.”** Ada yang menilai dari demo yang meyakinkan. Ada yang menilai dari rasa kagum pimpinan. Ada yang menilai dari banyaknya fitur. Semua itu terlalu lemah.

Nilai agentic AI harus diukur dengan KPI yang membumi. Literatur yang dilampirkan dan dokumentasi populer agentic AI sama-sama menunjukkan bahwa penilaian harus dilakukan pada dua level: **level sistem** dan **level hasil bisnis**. Pada level sistem, metrik seperti *latency*, *error rate*, frekuensi eskalasi, dan pemakaian sumber daya penting untuk melihat apakah agent bekerja dengan sehat. Pada level hasil, yang lebih penting adalah nilai nyata: waktu penyelesaian lebih cepat, tingkat penyelesaian lebih tinggi, penghematan biaya, dan umpan balik pengguna. Human feedback, *override rate*, serta laporan kesalahan juga penting karena memberi konteks yang tidak terlihat dari telemetri mentah.

Untuk buku ini, KPI yang paling relevan dapat dijelaskan sederhana sebagai berikut:

- **Waktu**
Seberapa cepat pekerjaan selesai dibanding cara lama?
Contohnya: waktu respons helpdesk, waktu penyusunan laporan, waktu pencarian SOP.
- **Biaya**
Apakah beban kerja manual turun? Apakah jam kerja terselamatkan?
Bukan berarti semua orang harus dikurangi, tetapi pekerjaan bernilai rendah bisa ditekan.
- **Akurasi**
Seberapa tepat hasil agent dibanding baseline?
Ini penting untuk klasifikasi tiket, ringkasan, rujukan SOP, dan rekomendasi awal.
- **Auditability**
Bisakah jawaban atau aksi ditelusuri kembali?
Adakah sumber dokumen, jejak log, alasan keputusan, dan jalur eskalasi yang jelas?
Dalam banyak konteks bisnis, ini justru lebih penting daripada sekadar “jawabannya terdengar pintar.”
- **Kepuasan pengguna**
Apakah pengguna merasa terbantu?
Apakah mereka percaya pada sistem?
Apakah mereka tetap harus memperbaiki terlalu banyak hasil agent?
Metrik ini penting karena sistem yang “akurat di atas kertas” tetapi dibenci pengguna biasanya tidak akan bertahan lama.

Ada satu prinsip evaluasi yang sangat penting: **selalu bandingkan dengan baseline manusia atau proses lama**. Jangan hanya mengukur agent terhadap harapan. Ukur terhadap kenyataan sebelumnya. Apakah lebih cepat? Apakah lebih konsisten? Apakah lebih murah? Apakah lebih mudah diaudit? Tanpa pembandingan, proyek AI mudah jatuh ke ilusi kemajuan.

Penutup bab

Bab ini harus meninggalkan satu kesan kuat pada pembaca: **nilai bisnis agentic AI muncul ketika ia masuk ke pekerjaan yang benar-benar terjadi setiap hari**.

Bukan ke panggung demo, bukan ke pertunjukan istilah, tetapi ke meja kerja nyata: email, tiket, SOP, rapat, laporan, persetujuan, dan keputusan operasional.

Di titik inilah agentic AI berubah dari sesuatu yang menarik menjadi sesuatu yang berguna. Ia membantu organisasi membaca konteks, memakai pengetahuan, mempercepat layanan internal, dan menyiapkan keputusan dengan lebih tertib. Literatur enterprise yang dilampirkan tepat ketika menempatkan AI bukan lagi sekadar alat bantu menulis, tetapi sebagai **partner operasional** yang ikut masuk ke *workflow*, layanan, dan keputusan.

Kalimat yang bisa ditanamkan ke pembaca sederhana saja: **bisnis tidak dibayar karena bisa membuat teks yang indah, tetapi karena bisa menjalankan pekerjaan dengan lebih cepat, lebih akurat, dan lebih bisa dipertanggungjawabkan**. Di situlah agentic AI mulai layak dibicarakan serius.

Bagian II. Memilih Use Case dan Mendesain Arsitektur yang Waras

Bab 3. Memilih use case bisnis yang masuk akal

3.1 Mulai dari rasa sakit proses, bukan dari demo

Banyak organisasi tergoda memulai dari demo. Mereka melihat agent yang bisa berbicara lancar, membuat ringkasan, atau menampilkan *workflow* yang tampak canggih. Lalu mereka buru-buru bertanya: “Bagaimana kalau ini dipasang di seluruh organisasi?” Pertanyaan seperti itu tampak visioner, tetapi sering justru menyesatkan. **Demo menunjukkan kemungkinan. Proses bisnis menunjukkan kenyataan.** Dan yang harus dibenahi selalu kenyataan, bukan panggung presentasi.

Cara berpikir yang lebih sehat adalah mulai dari **rasa sakit proses**. Di mana bottleneck terbesar? Pekerjaan apa yang paling menghabiskan waktu tetapi nilainya rendah? Titik mana yang paling sering menimbulkan salah kirim, salah baca, salah prioritas, atau keterlambatan? Use case yang baik hampir selalu lahir dari pertanyaan seperti itu. Bukan dari keinginan untuk terlihat modern, melainkan dari kebutuhan untuk membuat kerja harian menjadi lebih tertib. Literatur enterprise yang dilampirkan juga berkali-kali menekankan bahwa adopsi agentic AI yang matang harus dikaitkan dengan *business drivers*, metrik hasil, dan tantangan arsitektur yang nyata, bukan sekedar kemampuan model.

Agar mudah dipahami, bayangkan dua organisasi. Organisasi pertama berkata, “Kami ingin punya agent karena semua orang sedang bicara agent.” Organisasi kedua berkata, “Tiket internal kami lambat diprioritaskan, banyak email berulang, dan supervisor kelelahan membaca konteks yang sama setiap hari.” Organisasi kedua jauh lebih siap. Mengapa? Karena mereka tidak sedang mengejar label. Mereka sedang mengejar **perbaikan kerja**. Di situlah peluang agentic AI menjadi nyata.

Beberapa contoh **rasa sakit proses** yang sering layak dijadikan kandidat awal:

- **email masuk** terlalu banyak dan sulit diprioritaskan,
- **tiket internal** menumpuk karena klasifikasi awal lambat,
- **SOP dan kebijakan** ada, tetapi sulit ditemukan saat dibutuhkan,
- **notulen rapat** dibuat, tetapi tindak lanjutnya hilang,
- **approval dokumen** lambat karena konteks harus dibaca ulang berulang kali,
- **laporan rutin** memakan waktu besar untuk membaca, merangkum, dan menyusun narasi.

Dari sudut pandang pembaca mahasiswa maupun praktisi, ini kabar baik. Artinya, proyek agentic AI pertama **tidak harus heroik**. Ia tidak perlu langsung mengatur seluruh perusahaan. Sering kali cukup memilih satu proses yang menyebalkan, berulang, dan cukup jelas. Justru di situlah kemenangan awal paling mungkin terjadi.

3.2 Kriteria memilih use case awal

Use case awal yang baik tidak harus paling megah. Ia harus **cukup penting untuk bernilai**, tetapi **cukup sempit untuk dikendalikan**. Banyak kegagalan terjadi karena organisasi memilih kasus yang terlalu besar, terlalu politis, atau terlalu kabur sejak hari pertama. Buku ini perlu bersikap tegas: **proyek pertama harus dipilih untuk bisa belajar, bukan untuk pamer.**

Ada beberapa kriteria yang biasanya membuat use case awal menjadi sehat.

- **Masalahnya nyata dan terasa**
Orang benar-benar merasakan beban proses itu. Ada keluhan, keterlambatan, atau biaya yang jelas.
- **Input-nya cukup tersedia**
Ada dokumen, tiket, email, notulen, SOP, atau data lain yang bisa dipakai sebagai bahan kerja agent.
- **Output-nya cukup terdefinisi**
Agent tidak diminta “membantu semuanya,” tetapi diminta menghasilkan hal yang jelas: klasifikasi, ringkasan, rekomendasi awal, draf, atau routing.
- **Risikonya masih terkendali**
Bila agent salah, dampaknya masih bisa diperbaiki dan tidak langsung merusak operasi besar.
- **Ada pemilik proses**
Seseorang bertanggung jawab atas proses itu, mau memberi umpan balik, dan mau memperbaiki alurnya.
- **Keberhasilannya bisa diukur**
Ada baseline dan ada KPI: waktu, akurasi, biaya, kepuasan, atau tingkat eskalasi.

Prinsip ini juga konsisten dengan dokumentasi n8n yang menempatkan agent di dalam *workflow* yang jelas, terhubung ke tools, dan dievaluasi berdasarkan hasil kerja, bukan sekadar respons teks. n8n bahkan menekankan bahwa agent menggunakan tools dan API untuk bertindak di lingkungannya, sehingga nilai agent baru muncul bila lingkungannya memang terdefinisi dengan baik. ([n8n Docs](#))

Kalau ingin dibuat lebih tajam, pembaca bisa memakai pertanyaan saring sederhana berikut:

- **Apakah proses ini cukup sering terjadi?**
- **Apakah ada konteks yang harus dibaca, bukan hanya aturan kaku?**
- **Apakah ada pengetahuan atau dokumen yang perlu diambil?**
- **Apakah hasilnya bisa diperiksa manusia?**
- **Apakah kita punya data untuk mengukur sebelum dan sesudah?**

Bila jawaban untuk sebagian besar pertanyaan ini “ya,” biasanya use case itu layak diuji. Bila tidak, sering kali lebih baik menahan diri.

3.3 Kapan cukup RPA atau skrip biasa

Ini bagian yang harus ditulis dengan jujur. **Tidak semua masalah butuh agentic AI.** Dalam banyak kasus, *RPA*, otomasi biasa, atau skrip sederhana justru lebih masuk akal. Bila tugasnya stabil, aturannya eksplisit, langkahnya tidak banyak berubah, dan konteksnya rendah, maka menambahkan agentic AI bisa menjadi pemborosan yang dibungkus bahasa modern.

Misalnya:

- memindahkan file dari satu lokasi ke lokasi lain,
- menyalin data dari formulir terstruktur ke sistem lain,
- mengirim notifikasi periodik,
- melakukan pengecekan format,
- menjalankan urutan langkah yang tetap dan deterministik.

Untuk tugas-tugas seperti itu, skrip biasa atau otomasi *workflow* sering lebih murah, lebih cepat, dan lebih mudah dijelaskan. n8n sendiri pada dasarnya adalah platform otomasi alur kerja yang menggabungkan proses bisnis dengan kemampuan AI. Artinya, AI bukan selalu pusat solusi; kadang ia hanya salah satu komponen, dan kadang memang tidak diperlukan sama sekali. ([n8n Docs](#))

RPA atau skrip biasa biasanya cukup ketika:

- aturan keputusan hampir selalu tetap,
- input sangat terstruktur,
- variasi kasus rendah,
- tak ada kebutuhan penalaran bahasa yang berarti,
- dan hasil akhir harus sangat deterministik.

Kalimat penting yang perlu ditanamkan ke pembaca adalah ini: **jangan gunakan agentic AI untuk membuktikan bahwa Anda modern; gunakan hanya bila ia memberi nilai yang tidak bisa dicapai mudah oleh otomasi biasa.**

Di banyak organisasi, kedewasaan justru terlihat dari kemampuan berkata, “Untuk kasus ini, kita tidak butuh agent.” Sikap seperti itu bukan anti-inovasi. Itu tanda bahwa tim memahami perbedaan antara alat yang tepat dan alat yang sedang populer.

3.4 Kapan agentic AI lebih unggul

Setelah batasnya jelas, baru kita bisa bicara dengan adil: kapan agentic AI memang lebih unggul?

Agentic AI menjadi kuat saat pekerjaan berada di wilayah abu-abu: **tidak cukup sederhana untuk skrip kaku, tetapi juga tidak perlu diputuskan penuh oleh manusia dari nol setiap kali**. Di sinilah agent memberi nilai. Ia membaca konteks, memilih langkah, menggunakan tools, dan menyesuaikan arah berdasarkan hasil. n8n menjelaskan bahwa agent dapat memahami kemampuan berbagai tools dan menentukan tool mana yang dipakai tergantung tugas. Open WebUI juga memungkinkan model dibungkus dengan *knowledge*, tools, dan instruksi khusus agar bertindak sebagai agen yang lebih terarah. ([n8n Docs](#))

Agentic AI biasanya unggul ketika:

- **input-nya dominan teks atau dokumen**, bukan hanya tabel terstruktur,
- **ada banyak pengecualian** yang sulit ditangani aturan kaku,
- **perlu mengambil pengetahuan** dari SOP, kontrak, kebijakan, atau panduan,
- **perlu memilih tindakan berikutnya**, bukan sekadar menjalankan satu langkah,
- **perlu menggabungkan beberapa tools** atau beberapa sumber konteks,
- dan **hasil awal masih akan ditinjau atau disetujui manusia**.

Contohnya sederhana tetapi kuat:

- membaca email masuk lalu menentukan apakah ini keluhan, permintaan, atau eskalasi;
- membuka knowledge base untuk mencari SOP yang relevan;
- menyiapkan draf jawaban atau ringkasan kasus;
- lalu mengirimkannya ke orang yang tepat untuk ditinjau.

Itu bukan lagi chatbot biasa, karena ia tidak hanya menjawab. Ia **bergerak di dalam alur kerja**.

Open WebUI memberi petunjuk yang menarik di sini. Dokumentasinya menjelaskan bahwa dokumen bisa dicari “berdasarkan makna, bukan hanya kata kunci,” lalu dijadikan dasar jawaban dengan sitasi. Bahkan dengan *native function calling*, model dapat memanggil knowledge tools untuk mencari dan mengambil informasi yang relevan. Ini sangat penting untuk bisnis, karena agent yang baik harus **bisa mencari sebelum menjawab**, bukan sekadar terdengar percaya diri. ([Open WebUI](#))

OpenClaw menambah lapisan lain: agent melihat tools sebagai fungsi terstruktur yang bisa dipanggil, sementara *skills* mengajari agent kapan dan bagaimana menggunakannya. Ini membuat agent lebih cocok untuk use case yang membutuhkan kombinasi tindakan, konteks kerja, dan alat bantu yang eksplisit. ([OpenClaw](#))

Jadi, keunggulan agentic AI bukan terletak pada “kecerdasan magis.” Keunggulannya ada pada **kemampuan bekerja di proses yang kaya konteks, memakai alat, dan menjaga arah menuju tujuan.**

3.5 Human-in-the-loop: kapan manusia tetap memegang tombol akhir

Salah satu kesalahan paling berbahaya dalam implementasi AI adalah menganggap bahwa semakin otonom sistemnya, semakin hebat hasilnya. Itu tidak selalu benar. Dalam bisnis, **otonomi tanpa pagar** sering berubah menjadi risiko. Karena itu konsep **human-in-the-loop** sangat penting: manusia tetap menjadi pengawas, penilai, atau pemberi persetujuan pada titik-titik tertentu.

n8n bahkan mendokumentasikan mekanisme *human review* untuk tools tertentu. Ketika AI ingin memakai tool yang diganti, *workflow* bisa berhenti lebih dulu dan mengirim permintaan persetujuan melalui kanal yang dipilih. Hanya setelah disetujui, aksi dilanjutkan; bila ditolak, aksi dibatalkan. Ini contoh bagus bahwa agentic AI yang matang bukan yang “liar,” melainkan yang **tahu kapan harus berhenti dan meminta izin.** ([n8n Docs](#))

Manusia sebaiknya tetap memegang tombol akhir ketika:

- **keputusan menyentuh uang**, kontrak, atau komitmen resmi,
- **dampak reputasi tinggi**, misalnya komunikasi sensitif,
- **kasus ambigu** dan konteksnya belum cukup jelas,
- **risiko salah tinggi**, misalnya approval kebijakan atau akses,
- **aturan belum stabil** atau masih diperdebatkan,
- dan **agent belum memiliki rekam jejak yang cukup.**

Sebaliknya, manusia bisa mengurangi intervensi ketika:

- output agent hanya berupa **saran awal**,
- risikonya rendah dan mudah dikoreksi,
- data serta dokumen sumbernya cukup jelas,
- dan metrik kualitas agent sudah cukup baik.

Poin yang harus ditekankan ke pembaca: **human-in-the-loop bukan tanda bahwa AI gagal.** Justru itu tanda bahwa sistem dirancang dengan dewasa. Agentic AI terbaik di bisnis bukan yang paling berani bertindak, melainkan yang paling tepat membagi kerja antara mesin dan manusia.

3.6 Hands-on: memetakan satu proses bisnis menjadi goal, tools, memory, approval gate

Bagian ini perlu dibuat praktis dan membumi. Tanpa *hands-on*, pembaca akan berhenti di level konsep. Tujuan latihan ini bukan membangun sistem penuh, melainkan melatih cara berpikir. Kita ingin pembaca bisa melihat satu proses bisnis, lalu mengubahnya menjadi komponen agentic yang masuk akal.

Mari ambil contoh yang dekat: **triase tiket internal**.

Bayangkan sebuah organisasi menerima banyak tiket setiap hari. Sebagian adalah pertanyaan rutin. Sebagian butuh eskalasi cepat. Sebagian salah kirim. Sebagian tidak lengkap. Supervisor kelelahan membaca semuanya satu per satu. Di sinilah agent bisa masuk.

Langkah 1: tentukan goal

Pertama, definisikan tujuan secara sederhana.
Bukan “membuat helpdesk pintar,” tetapi misalnya:

- **mengklasifikasikan tiket masuk secara awal,**
- **menentukan prioritas awal,**
- **mengambil SOP yang relevan,**
- **dan mengarahkan tiket ke antrian yang benar.**

Tujuan yang baik harus jelas, sempit, dan dapat diperiksa hasilnya.

Langkah 2: tentukan tools

Lalu pikirkan alat apa saja yang benar-benar dibutuhkan agent. Dalam arsitektur agent modern, tools adalah fungsi atau layanan yang dapat dipanggil model untuk mengambil informasi atau melakukan aksi. OpenClaw mendeskripsikan tools sebagai fungsi terstruktur yang bisa dipanggil agent, sementara n8n mewajibkan agent terhubung ke paling sedikit satu tool sub-node untuk bekerja penuh sebagai *Tools Agent*. ([OpenClaw](#))

Untuk contoh tiket internal, tools yang relevan bisa berupa:

- pencari knowledge base,
- pembaca daftar kategori tiket,
- pengirim notifikasi,
- pencatat ke sistem tiket,
- atau pengarah ke antrian tertentu.

Buku ini tidak perlu masuk ke CLI. Yang penting pembaca paham bahwa **agent tanpa tools sering hanya berhenti di level bicara**. Sementara agent dengan tools bisa mulai bekerja.

Langkah 3: tentukan memory

Selanjutnya, pikirkan *memory* atau konteks kerja. Agent perlu tahu apa yang sedang terjadi, apa yang sudah dibaca, dan apa yang pernah diputuskan sebelumnya. Dalam sistem seperti OpenClaw, workspace agen menjadi direktori kerja untuk alat dan konteks; dalam Open WebUI, model juga bisa diperkaya dengan *knowledge* dan instruksi agar tetap berada pada domain yang benar. ([OpenClaw](#))

Dalam praktik bisnis, *memory* bisa berarti:

- riwayat tiket sebelumnya,
- kategori yang pernah dipakai,
- SOP yang relevan,
- daftar penanggung jawab,
- atau aturan prioritas tertentu.

Yang perlu dipahami pembaca: *memory* tidak harus mistis. Ia sering hanya berarti **konteks kerja yang disimpan dan dipakai ulang dengan disiplin**.

Langkah 4: tentukan approval gate

Terakhir, tentukan titik mana yang harus tetap diperiksa manusia. Misalnya:

- tiket prioritas tinggi harus diverifikasi supervisor,
- tiket yang menyangkut akses atau kebijakan harus dicek manual,
- rekomendasi agent boleh langsung lewat hanya untuk kasus rutin tertentu,
- semua tiket ambigu harus otomatis eskalasi.

Di sinilah **approval gate** menjadi pagar keselamatan. Ini bukan penghambat inovasi, melainkan mekanisme agar organisasi bisa percaya pada sistem sedikit demi sedikit.

Bentuk peta sederhananya

Pembaca bisa merangkum satu proses bisnis ke dalam pola berikut:

- **Proses:** triase tiket internal
- **Goal:** klasifikasi awal, prioritas awal, routing awal
- **Input:** isi tiket, lampiran, pengirim, waktu masuk
- **Tools:** knowledge search, kategori tiket, notifikasi, sistem tiket
- **Memory:** riwayat tiket, SOP, daftar PIC, aturan prioritas
- **Approval gate:** kasus sensitif, prioritas tinggi, kategori ambigu
- **Output:** label tiket, saran prioritas, referensi SOP, antrian tujuan
- **KPI:** waktu triase, akurasi klasifikasi, tingkat eskalasi, kepuasan pengguna

Pola ini bisa dipakai juga untuk proses lain, misalnya:

- ringkasan rapat dan tindak lanjut,
- approval dokumen,
- asisten SOP internal,
- penyusunan memo awal,
- atau pengelompokan email masuk.

Latihan seperti ini menantang karena memaksa pembaca berpikir bukan sebagai penggemar AI, tetapi sebagai **perancang kerja**. Dan memang itulah inti agentic AI untuk bisnis: bukan memamerkan model, tetapi mendesain cara kerja yang lebih masuk akal.

Penutup bab

Bab ini harus meninggalkan kesan yang tegas: **use case yang benar lebih penting daripada model yang heboh**. Organisasi tidak membutuhkan agent di semua tempat. Mereka membutuhkan agent di tempat yang tepat: proses yang menyakitkan, berulang, kaya konteks, cukup terukur, dan masih bisa dijaga dengan persetujuan manusia.

Karena itu, pelajaran terpenting dari bab ini sederhana:

- mulailah dari rasa sakit proses,
- pilih use case yang sempit tetapi nyata,
- jangan pakai agent bila skrip biasa sudah cukup,
- gunakan agent ketika konteks, tools, dan penalaran memang dibutuhkan,
- dan selalu tetapkan titik di mana manusia tetap memegang tombol akhir.

Bila pembaca memahami lima hal itu, mereka sudah melangkah jauh dari sekadar kagum pada AI. Mereka mulai belajar sesuatu yang lebih penting: **bagaimana memilih pertempuran yang layak dimenangkan**.

Bab 4. Arsitektur minimum agentic system untuk bisnis

4.1 Model, memori, tools, workflow, evaluasi, guardrails

Cara paling mudah memahami arsitektur minimum adalah dengan membayangkan agentic system sebagai **tim kerja mini**. Model adalah otaknya. Memori adalah catatan kerja dan konteksnya. *Tools* adalah tangan dan kakinya. *Workflow* adalah SOP dan alur tugasnya. Evaluasi adalah cara kita memeriksa apakah hasilnya benar atau tidak. *Guardrails* adalah pagar pengamannya. Tanpa kombinasi ini, yang kita punya biasanya hanya chatbot yang terdengar pintar, tetapi tidak benar-benar siap bekerja.

Model berfungsi sebagai mesin penalaran. Ia membaca input, memahami maksud, menimbang opsi, lalu menghasilkan langkah atau respons. Namun model tidak pernah cukup berdiri sendiri. Model bisa cakap berbicara, tetapi tanpa konteks dan alat, ia akan sering menjadi **pemberi jawaban yang meyakinkan tetapi dangkal**. Karena itu dalam arsitektur agentic, model harus diperlakukan sebagai satu komponen, bukan seluruh sistem.

Memori memberi kesinambungan. Banyak tugas bisnis tidak selesai dalam satu putaran tanya-jawab. Ada riwayat tiket, keputusan sebelumnya, dokumen yang sudah dibaca, preferensi kerja, daftar PIC, dan konteks proses. Literatur yang dilampirkan menegaskan bahwa arsitektur memori dan manajemen konteks menentukan kemampuan agent untuk mengambil keputusan yang lebih baik, karena agent dapat menggabungkan konteks saat ini dengan pengalaman atau data terdahulu.

Tools adalah jembatan ke dunia nyata. Di sinilah agent berubah dari “sistem yang bicara” menjadi “sistem yang bekerja”. Ia dapat mencari pengetahuan, membaca file, memanggil API, menulis ke sistem lain, atau mengirim hasil ke alur berikutnya. Dokumentasi Ollama menjelaskan bahwa *tool calling* memungkinkan model memanggil tools dan menggunakan hasilnya dalam respons. Dokumentasi OpenClaw juga menekankan bahwa *tools* adalah fungsi terstruktur yang bisa dipanggil agent, sedangkan *skills* mengajari agent kapan dan bagaimana menggunakannya. ([Ollama](#))

Workflow adalah urutan kerja yang memberi arah. Dalam bisnis, pekerjaan jarang berdiri sendiri. Ada tiket yang harus dibaca dulu, ada klasifikasi yang harus terjadi sebelum eskalasi, ada ringkasan yang harus dibuat sebelum persetujuan, ada pencarian SOP sebelum draf jawaban. Karena itu agentic system harus dipasang di dalam alur kerja yang jelas, bukan dilepas begitu saja. n8n secara eksplisit memposisikan *AI Agent* sebagai komponen yang hidup di dalam *workflow*, bersama trigger, tools, review, dan langkah-langkah proses lainnya. ([n8n Docs](#))

Evaluasi adalah rem akal sehat. Agentic system yang tidak diukur akan cepat berubah menjadi mainan mahal. Literatur referensi yang menekankan pentingnya *evaluation framework*, KPI, dan pemantauan berkelanjutan untuk melihat apakah agent benar-benar membantu proses, bukan sekadar terlihat sibuk.

Guardrails adalah pagar agar agent tidak bertindak liar. Guardrails bisa berupa batas aksi, persetujuan manusia, pembatasan akses, aturan *fallback*, *schema validation*, hingga step limit. Literatur engineering yang dilampirkan menegaskan bahwa panggilan tool harus dibatasi dengan kontrak, retry yang terkontrol, akses minimum, serta batas waktu dan token agar tidak terjadi *runaway execution*.

Kalau disederhanakan, komponen minimum agentic system untuk bisnis dapat dipahami seperti ini:

- **Model:** berpikir dan menyusun langkah.
- **Memori:** menjaga konteks dan kesinambungan.
- **Tools:** mengambil data dan melakukan aksi.
- **Workflow:** menentukan urutan kerja.
- **Evaluasi:** memastikan hasil dapat diukur.
- **Guardrails:** menjaga keamanan dan batas tindakan.

Tanpa salah satu dari unsur di atas, sistem biasanya menjadi timpang. Ia mungkin cepat, tetapi tidak aman. Atau aman, tetapi tidak berguna. Atau cerdas, tetapi tidak bisa diaudit.

4.2 Single-agent vs multi-agent

Salah satu jebakan paling sering dalam agentic AI adalah terlalu cepat tergoda ke *multi-agent*. Banyak orang melihat arsitektur dengan banyak agent spesialis lalu merasa itu pasti lebih canggih. Padahal untuk banyak kebutuhan bisnis, **satu agent yang rapi jauh lebih bernilai daripada lima agent yang saling mengganggu**. Literatur engineering yang dilampirkan juga menegaskan bahwa tidak ada satu arsitektur yang selalu benar; pilihan desain bergantung pada kompleksitas misi dan konteks bisnis. Untuk tugas ringan dan terikat latensi, *single-agent loop* sering justru paling cocok.

Single-agent cocok ketika:

- tugasnya cukup jelas,
- toolset-nya sedikit,
- alurnya pendek,
- kebutuhan koordinasinya rendah,
- dan organisasi masih di tahap awal belajar.

Model seperti ini mirip staf serba bisa yang cukup diberi konteks, pengetahuan, dan beberapa alat. Ia bisa membaca tiket, mencari SOP,

membuat ringkasan, dan menyiapkan draf. Untuk banyak organisasi, ini sudah sangat kuat.

Multi-agent cocok ketika pekerjaan mulai memiliki pembagian peran yang benar-benar berbeda. Misalnya ada agent pengumpul data, agent analis, agent penyusun laporan, dan agent pengawas. Literatur engineering menyebut beberapa topologi seperti **sequential**, **hierarchical**, **parallel**, dan **hybrid**. Dalam arsitektur hierarkis, ada supervisor yang membagi tugas ke agent spesialis; dalam arsitektur paralel, beberapa agent bekerja bersamaan; dalam arsitektur sekuensial, agent bekerja seperti rantai produksi.

Masalahnya, semakin banyak agent, semakin besar juga:

- biaya koordinasi,
- potensi konflik konteks,
- latensi,
- kebutuhan observability,
- dan beban pemeliharaan.

Karena itu aturan praktis yang sehat adalah ini: **mulailah dari single-agent**. Beralih ke *multi-agent* hanya bila benar-benar ada alasan bisnis yang kuat, misalnya:

- domain tugas berbeda jauh,
- ada kebutuhan paralelisme,
- ada beban kerja yang terlalu kompleks untuk satu loop,
- atau organisasi perlu pembagian peran yang ketat dan dapat diaudit.

Kalimat yang penting ditanamkan ke pembaca: **multi-agent bukan tanda kedewasaan; kadang justru tanda terlalu cepat berfantasi**. Kedewasaan terlihat dari kemampuan memilih desain sesederhana mungkin untuk menyelesaikan masalah nyata.

4.3 OpenClaw vs n8n: pilih alat sesuai gaya kerja

Bab ini juga perlu jujur dan praktis. **OpenClaw** dan **n8n** bukan musuh. Keduanya melayani kebutuhan yang berbeda, dan di banyak kasus justru bisa saling melengkapi.

Secara sederhana, **n8n** lebih kuat sebagai **workflow orchestrator**. Ia sangat cocok bila organisasi sudah berpikir dalam bahasa proses: trigger, langkah kerja, cabang keputusan, notifikasi, review, integrasi, dan alur persetujuan. Dokumentasi dan situs resminya menonjolkan bahwa n8n membangun AI agent yang “bekerja secara terprediksi di produksi”, dengan integrasi luas, guardrails, dan *human-in-the-loop*. ([n8n Docs](#))

Sementara itu, **OpenClaw** lebih dekat ke **agent runtime** yang aktif. Ia dirancang agar agent bisa “benar-benar melakukan sesuatu”, dengan *tools* dan *skills* yang memberi agen kemampuan operasional. Situs resminya bahkan secara eksplisit menyebut dirinya sebagai “AI that actually does things.” Dokumentasi skill OpenClaw juga menunjukkan bahwa perilaku agent dibentuk oleh *skill folders* berisi instruksi dan konfigurasi, sehingga agen dapat diajari cara memakai alat secara lebih spesifik. ([OpenClaw](#))

Dengan bahasa yang lebih membumi:

- **Pilih n8n** bila gaya kerja Anda adalah **alur proses**.
Anda ingin email masuk → klasifikasi → review → kirim ke PIC → catat ke sistem → notifikasi.
- **Pilih OpenClaw** bila gaya kerja Anda adalah **agent yang aktif memakai alat**.
Anda ingin agen yang lebih “berinisiatif” dalam mencari, membaca, mengeksekusi, dan menyusun langkah kerja.
- **Padukan keduanya** bila perlu.
Misalnya n8n menjadi jalur orkestrasi bisnis, sementara OpenClaw menjadi agen spesialis yang dipanggil pada titik tertentu.

Bagi mahasiswa dan praktisi, cara memilih yang paling sehat bukan dari “mana yang paling heboh”, tetapi dari pertanyaan ini:

- **Apakah saya sedang membangun proses, atau sedang membangun pelaku di dalam proses?**
- **Apakah kebutuhan saya lebih dominan workflow, atau lebih dominan penggunaan tool oleh agent?**
- **Apakah tim saya lebih nyaman dengan diagram alur, atau dengan pola agent dan skill?**

Pertanyaan-pertanyaan ini akan menyelamatkan banyak waktu.

4.4 Peran Open WebUI sebagai workspace dan knowledge front-end

Dalam stack buku ini, **Open WebUI** sebaiknya tidak dipahami hanya sebagai “tampilan chat yang cantik”. Itu terlalu sempit. Peran yang lebih tepat adalah: **workspace**, *knowledge front-end*, dan titik interaksi manusia dengan agentic system. Disini ditempatkan Open WebUI sebagai *workspace* dan *knowledge front-end*.

Dokumentasi resmi Open WebUI menegaskan dua hal penting. Pertama, fitur **Knowledge** memungkinkan dokumen menjadi *memory bank* yang dapat dicari secara semantik. Kedua, dengan *native function calling*, model tidak hanya mencari, tetapi bisa menelusuri knowledge base, membaca file per halaman, dan mensintesis jawaban lintas dokumen. Dokumentasi *Tools* juga mengingatkan bahwa ketika *native function calling* dipakai, knowledge tidak otomatis “masuk sendiri”; model perlu aktif memanggil knowledge tools untuk mencari dan mengambil informasi yang relevan. ([Open WebUI](#))

Ini sangat penting untuk bisnis. Open WebUI dapat menjadi tempat di mana:

- pengguna berinteraksi dengan agent,
- model diberi akses ke knowledge yang sesuai,
- hasil dapat dilihat bersama sumbernya,
- dan eksperimen dapat dilakukan tanpa harus langsung masuk ke sistem produksi.

Dengan kata lain, **Open WebUI adalah meja kerja**, bukan sekadar jendela obrolan. Ia cocok dipakai untuk:

- asisten SOP dan kebijakan,
- RAG untuk dokumen internal,
- ruang eksperimen prompt dan instruksi model,
- titik uji coba sebelum alur dibungkus ke workflow yang lebih keras.

Bagi pembaca, analoginya sederhana. Kalau Ollama adalah mesin model lokal, dan n8n atau OpenClaw adalah pelaku proses atau runtime agent, maka **Open WebUI adalah ruang kerja tempat manusia melihat, mencoba, memeriksa, dan memakai hasilnya**.

4.5 Logging, audit trail, session state, dan observability

Banyak implementasi AI terlihat meyakinkan sampai hari pertama ia salah. Setelah itu muncul pertanyaan yang sangat manusiawi: “Tadi ia membaca apa? Mengapa ia memilih langkah itu? Kapan ia memanggil tool ini? Siapa yang menyetujui?” Kalau sistem tidak bisa menjawab, maka organisasi akan kehilangan kepercayaan.

Karena itu **logging dan observability bukan aksesoris**. Ia adalah bagian inti dari arsitektur. Literatur enterprise yang dilampirkan menekankan **end-to-end tracing**, telemetry, debugging, monitoring, escalation threshold, dan governance sebagai komponen dasar arsitektur enterprise-grade. Literatur engineering juga tegas: observability berupa *structured logs* dan *replayable traces* adalah fitur arsitektural yang tidak bisa ditawar.

Ada empat hal yang perlu dibedakan:

- **Logging**
Catatan kejadian sistem: input, langkah, tool call, output, error.
- **Audit trail**
Jejak yang bisa dipakai untuk pertanggungjawaban: siapa melakukan apa, kapan, dengan dasar apa.
- **Session state**
Keadaan kerja saat ini: konteks yang sedang aktif, memori pendek, status proses.
- **Observability**
Kemampuan melihat perilaku sistem secara utuh: latensi, error, pola tool use, eskalasi, dan kualitas keputusan.

Dalam konteks bisnis, manfaatnya sangat nyata:

- memudahkan debugging ketika hasil agent aneh,
- membantu audit ketika ada keputusan yang dipertanyakan,
- memberi dasar untuk perbaikan terus-menerus,
- dan meningkatkan kepercayaan karena perilaku agent tidak gelap.

n8n juga mendukung gagasan ini dari sisi proses. Ketika *human-in-the-loop* dipakai, workflow berhenti, mengirim permintaan persetujuan, lalu melanjutkan atau membatalkan langkah berdasarkan keputusan reviewer. Mekanisme seperti ini pada dasarnya ikut memperkuat *audit trail* karena jalur keputusan menjadi jelas. ([n8n Docs](#))

Bab ini perlu menegaskan satu hal yang sangat penting: **agent tanpa jejak adalah risiko**. Mungkin ia terlihat cepat, tetapi ketika salah, tidak ada yang bisa menjelaskan mengapa.

4.6 Arsitektur minimum yang ringan, murah, dan bisa dirawat

Bab ini harus ditutup dengan sikap editorial yang tegas: **arsitektur minimum yang baik adalah yang cukup untuk bekerja, cukup untuk diawasi, dan cukup murah untuk dipelihara**. Bukan arsitektur paling kompleks. Bukan juga arsitektur paling trendi.

Literatur referensi sendiri sangat jelas bahwa desain agentic harus disesuaikan dengan kebutuhan bisnis, kompleksitas proses, dan biaya operasional. Sumber engineering menegaskan bahwa semakin kompleks misi, semakin sengaja desainnya harus dibuat. Tetapi untuk banyak kebutuhan bisnis awal, solusi terbaik justru sederhana: satu model lokal, satu lapisan knowledge, beberapa tools yang jelas, satu workflow, evaluasi dasar, dan approval gate di titik sensitif.

Arsitektur minimum yang sehat untuk banyak organisasi kecil–menengah biasanya cukup berisi:

- **satu model lokal** untuk reasoning dasar,
- **satu lapisan knowledge/RAG** untuk dokumen internal,
- **satu antarmuka kerja** untuk pengguna,
- **satu mekanisme workflow** untuk routing dan tindak lanjut,
- **sejumlah kecil tools** yang jelas manfaatnya,
- **logging dan approval gate** untuk kontrol,
- serta **KPI sederhana** untuk melihat apakah sistem layak dipertahankan.

Mengapa pendekatan minimum ini baik?

- **lebih murah**, karena komponen lebih sedikit,
- **lebih mudah dirawat**, karena alurnya jelas,
- **lebih mudah diaudit**, karena jejaknya pendek,
- **lebih mudah dipelajari tim**, karena tidak menuntut banyak peran baru,
- dan **lebih mudah dinaikkan bertahap**, karena dasar sistemnya rapi.

Kalimat yang sangat penting untuk pembaca adalah ini: **jangan membangun kota futuristik bila Anda belum bisa mengelola satu rumah yang rapi**. Dalam agentic AI bisnis, kemenangan awal paling sering datang dari arsitektur minimum yang disiplin, bukan dari sistem paling rumit.

Penutup bab

Bab ini harus meninggalkan satu pesan kunci: **agentic system yang berguna dibangun seperti sistem kerja, bukan seperti pertunjukan model**. Artinya, kita harus memikirkan model, memori, tools, workflow, evaluasi, guardrails, logging, dan pengawasan manusia sebagai satu kesatuan. Bukan aksesoris terpisah. Disini diletakkan semua unsur ini di jantung arsitektur minimum untuk bisnis.

Bagi pembaca mahasiswa dan praktisi, pelajaran terpenting dari bab ini sederhana:

- **mulailah dari arsitektur sekecil mungkin,**
- **pilih alat sesuai gaya kerja,**
- **buat jejak kerja agent terlihat,**
- **jaga agar manusia tetap bisa mengoreksi,**
- dan **bangun sistem yang bisa dirawat, bukan hanya dipamerkan.**

Kalau Bab 3 mengajari pembaca memilih pertempuran yang layak, maka Bab 4 mengajari mereka **membangun markas yang tidak mudah runtuh**.

Bagian III. Menyiapkan Ekosistem di Ubuntu 26.04

Bab 5. Menyiapkan host: Ubuntu 26.04, GPU NVIDIA, Docker

5.1 Kenapa Ubuntu 26.04 dijadikan target platform

Untuk buku ini, memilih **Ubuntu 26.04 LTS** sebagai platform sasaran adalah keputusan yang masuk akal. Alasannya bukan karena Ubuntu “paling canggih”, tetapi karena ia berada di titik manis antara **dukungan jangka panjang**, ekosistem paket yang luas, komunitas besar, dokumentasi server yang matang, dan kompatibilitas yang baik dengan stack modern seperti Docker dan driver NVIDIA. Canonical sendiri merekomendasikan jalur *Long Term Support* untuk server karena LTS mendapat **5 tahun standard support dan security updates**, sedangkan rilis non-LTS masa dukungannya jauh lebih pendek. ([Ubuntu](#))

Untuk konteks buku *agentic AI* bisnis, pilihan ini juga strategis. Kita tidak sedang mengejar distro yang eksotis. Kita sedang mencari fondasi yang:

- **mudah dipasang ulang** bila gagal,
- **mudah dirawat** oleh mahasiswa maupun praktisi,
- **mudah dicocokkan** dengan dokumentasi resmi,
- dan **mudah diintegrasikan** dengan driver, container, dan tools AI.

Docker sendiri sudah mencantumkan **Ubuntu Resolute 26.04 (LTS)** di daftar versi Ubuntu yang didukung untuk pemasangan Docker Engine resmi. Ini penting, karena artinya jalur instalasi Docker untuk platform ini sudah mulai dianggap kelas utama, bukan eksperimen liar. ([Docker Documentation](#))

Secara praktis, Ubuntu 26.04 cocok dijadikan sasaran untuk tiga tipe pembaca:

- **mahasiswa** yang ingin belajar sistem AI lokal dengan lingkungan modern,
- **praktisi teknis** yang ingin membangun host untuk eksperimen dan pilot,
- **organisasi kecil-menengah** yang ingin memiliki fondasi lokal yang murah, cukup aman, dan mudah diaudit.

Jadi, alasan memilih Ubuntu 26.04 bukan sekadar karena versi terbaru. **Alasannya adalah kombinasi stabilitas LTS, dukungan ekosistem, dan keselarasan dengan stack open source yang dipakai buku ini.**

5.2 Catatan realitas: status versi dan kehati-hatian produksi

Di sinilah buku harus jujur. **Ubuntu 26.04 LTS boleh dijadikan target platform, tetapi pembaca harus sadar bahwa dokumentasi dan perilaku paket di awal siklus rilis bisa masih bergerak.** Release notes resmi Ubuntu 26.04 secara eksplisit menyatakan bahwa rilis ini masih memiliki catatan isu dan perubahan yang perlu diperhatikan, termasuk beberapa keterbatasan fitur seperti TPM-backed full disk encryption pada titik rilis 26.04.0. ([Dokumentasi Ubuntu](#))

Artinya, untuk pembaca buku ini, narasi yang paling aman adalah sebagai berikut:

- **Ubuntu 26.04 adalah platform sasaran**, bukan jaminan bahwa semua kombinasi hardware akan mulus tanpa penyesuaian.
- **Host pilot dan host produksi ringan** boleh memakai 26.04, tetapi harus diuji lebih dulu pada kombinasi motherboard, GPU, driver, dan kernel yang nyata.
- Untuk lingkungan yang sangat sensitif terhadap perubahan, pembaca tetap perlu disiplin pada prinsip klasik: **uji di mesin kecil dulu, dokumentasikan, baru replikasi.**

Ada beberapa bentuk kehati-hatian yang sebaiknya disebut sejak awal:

- jangan langsung menganggap semua *driver branch* NVIDIA akan nyaman di semua mesin;
- jangan mencampur terlalu banyak repositori tanpa alasan;
- jangan buru-buru memasang CUDA dan toolkit lain bila kebutuhan awal baru sebatas inference lokal via container;
- dan jangan menganggap “versi terbaru” otomatis berarti “kombinasi paling aman.”

Kalimat yang perlu ditanamkan ke pembaca: **LTS memberi jalur yang tenang, tetapi ketenangan tetap harus dibangun dengan disiplin uji coba.**

5.3 Hardware minimum dan rekomendasi GPU NVIDIA

Untuk *agentic AI* bisnis, host tidak harus menjadi monster mahal. Yang lebih penting adalah **kesesuaian antara beban kerja, VRAM, RAM sistem, storage, dan gaya pemakaian**. Banyak pembaca pemula salah fokus ke jumlah core CPU, padahal dalam praktik AI lokal untuk inference, bottleneck paling sering terasa di **VRAM GPU, RAM, dan I/O storage**.

Sebagai aturan praktis untuk buku ini, pembaca bisa memakai kerangka berikut:

Minimum yang masih masuk akal untuk belajar

- CPU modern 6–8 core
- RAM **16 GB**
- SSD **512 GB**
- GPU NVIDIA dengan **8 GB VRAM**
- jaringan stabil untuk menarik image dan model

Konfigurasi ini cukup untuk:

- belajar host Linux,
- mencoba Docker,
- menjalankan container dasar,
- mencoba model lokal ukuran kecil–menengah,
- dan menguji alur Open WebUI atau workflow sederhana.

Rekomendasi yang lebih sehat untuk praktik serius

- CPU 8–12 core
- RAM **32 GB**
- SSD/NVMe **1 TB**
- GPU NVIDIA dengan **12–16 GB VRAM**
- pendinginan dan PSU yang baik

Konfigurasi ini jauh lebih nyaman untuk:

- model lokal yang lebih besar,
- sesi kerja lebih panjang,
- RAG dan knowledge base,
- eksperimen beberapa container,
- dan beban workflow yang lebih realistis.

Untuk laboratorium kecil atau pilot tim

- RAM **64 GB**
- NVMe **1–2 TB**
- satu GPU NVIDIA kelas workstation atau gaming kelas atas dengan VRAM lebih longgar
- atau beberapa host kecil bila ingin memisahkan fungsi

Mengapa NVIDIA ditekankan? Karena ekosistem driver, container, dan AI tooling untuk NVIDIA masih paling matang untuk banyak alur inference dan containerized GPU workloads. NVIDIA sendiri mendokumentasikan bahwa **NVIDIA Container Toolkit** dipakai agar container dapat mengakses GPU host, dan verifikasi dasar yang paling lazim tetap berputar pada kesehatan stack driver dan utilitas seperti `nvidia-smi`. ([NVIDIA Docs](#))

Untuk pembaca buku ini, pesan sederhananya:

- **VRAM lebih penting daripada gengsi model GPU.**
- **RAM sistem jangan terlalu pelit.**
- **SSD itu wajib, HDD jangan dijadikan fondasi utama.**
- **PSU dan pendinginan bukan detail sepele.**

Host AI lokal yang stabil bukan hanya soal kencang. **Ia harus tahan dipakai kerja.**

5.4 Instalasi sistem, update, user, SSH, dan firewall

Bagian ini adalah *how-to* inti. Targetnya bukan sekadar “Ubuntu berhasil terpasang”, tetapi host sudah cukup rapi untuk dijadikan fondasi kerja.

Tahap 1. Pasang Ubuntu Server atau Desktop sesuai kebutuhan

Untuk host yang fokus ke layanan lokal dan container, **Ubuntu Server** biasanya lebih ringan dan lebih bersih. Untuk pembaca pemula yang masih ingin desktop visual, Ubuntu Desktop juga bisa dipakai. Yang penting, pilih instalasi yang sederhana dan jangan membebani host dengan paket yang tidak relevan.

Hal yang sebaiknya diputuskan saat instalasi:

- hostname yang jelas,
- timezone yang benar,
- user admin pertama yang akan dipakai,
- disk layout yang masuk akal,
- dan apakah OpenSSH ingin langsung dipasang.

Ubuntu Server punya dokumentasi resmi yang luas untuk instalasi, manajemen user, manajemen paket, dan praktik keamanan dasar setelah sistem menyala. ([Ubuntu](#))

Tahap 2. Update sistem segera setelah login pertama

Setelah mesin hidup, langkah pertama adalah memastikan indeks paket dan patch keamanan sudah segar.

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

Contoh keluaran yang sehat biasanya memperlihatkan paket berhasil diambil, dependensi diselesaikan, lalu sistem reboot bersih.

```
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security
InRelease
Reading package lists... Done
Building dependency tree... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not
upgraded.
```

APT memang merupakan jalur paket yang direkomendasikan di Ubuntu Server untuk instalasi, upgrade, dan pengelolaan software. ([Ubuntu](#))

Tahap 3. Cek identitas host

Sebelum melangkah jauh, biasakan memeriksa identitas sistem: versi OS, kernel, dan user aktif.

```
lsb_release -a
uname -r
whoami
hostnamectl
```

Contoh keluaran:

```
Distributor ID: Ubuntu
Description:    Ubuntu 26.04 LTS
Release:        26.04
Codename:       resolute
```

Perintah ini tampak sepele, tetapi sangat berguna saat nanti pembaca melakukan troubleshooting. Banyak kebingungan teknis ternyata hanya karena orang lupa ia sedang berada di host yang berbeda atau versi kernel yang tidak sama.

Tahap 4. Buat user kerja terpisah bila perlu

Secara praktik yang sehat, jangan jadikan semua pekerjaan sehari-hari bergantung pada satu akun yang dipakai untuk semuanya. Dokumentasi Ubuntu Server menjelaskan manajemen user dan praktik penggunaan sudo untuk administrasi sistem. ([Ubuntu](#))

Contoh membuat user baru:

```
sudo adduser aiops
sudo usermod -aG sudo aiops
id aiops
```

Contoh keluaran:

```
uid=1001(aiops) gid=1001(aiops)
groups=1001(aiops),27(sudo)
```

Langkah ini berguna bila host akan dipakai bersama, atau bila kita ingin memisahkan user admin awal dari user kerja harian.

Tahap 5. Pasang dan aktifkan OpenSSH

Kalau host akan diakses jarak jauh, pasang OpenSSH lebih awal.

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo systemctl status ssh --no-pager
```

Contoh keluaran sehat:

```
• ssh.service - OpenBSD Secure Shell server
  Loaded: loaded
  Active: active (running)
```

Dengan ini host sudah siap diakses melalui SSH, yang sangat penting untuk server rumahan, mini lab, atau mesin yang tidak selalu ditemplei monitor.

Tahap 6. Aktifkan firewall dasar

Ubuntu menggunakan **ufw** sebagai alat firewall default. Dokumentasi Ubuntu Server menjelaskan bahwa **ufw** adalah front-end yang mempermudah pengelolaan aturan firewall host-based, dan secara default biasanya **terpasang tetapi belum aktif**. ([Ubuntu](#))

Untuk host dasar, langkah aman yang paling umum adalah mengizinkan OpenSSH lebih dulu, lalu baru mengaktifkan firewall.

```
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
```

Contoh keluaran:

```
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
```

Ini terlihat sederhana, tetapi sangat penting. Kesalahan klasik pemula adalah mengaktifkan firewall dulu tanpa membuka akses SSH, lalu mengunci dirinya sendiri dari host.

Tahap 7. Rapikan kebiasaan dasar

Sebelum masuk ke GPU dan Docker, biasakan kebiasaan kecil berikut:

- pakai hostname yang bermakna,
- simpan catatan perubahan penting,
- jangan memasang paket yang tidak diperlukan,
- dan biasakan reboot terkontrol setelah perubahan besar.

Host yang rapi dari awal akan menghemat banyak jam debugging di belakang.

5.5 Driver NVIDIA, verifikasi nvidia-smi, dan dasar keamanan host

Di sinilah banyak orang tersandung. Mereka terlalu cepat masuk ke container dan model, padahal **GPU di host belum sehat**. NVIDIA dan Ubuntu sama-sama menekankan bahwa driver di host harus benar dulu sebelum lapisan container dipasang di atasnya. Ubuntu Server punya panduan resmi pemasangan driver NVIDIA, dan panduan itu juga membedakan antara driver umum desktop dan **Enterprise Ready Drivers** yang diberi akhiran `-server`, yang direkomendasikan untuk server dan beban komputasi. ([Ubuntu](#))

Tahap 1. Pastikan GPU NVIDIA terdeteksi

Sebelum memasang driver, lihat dulu apakah PCIe device terbaca.

```
lspci | grep -i nvidia
```

Contoh keluaran:

```
01:00.0 VGA compatible controller: NVIDIA Corporation  
Device xxxx  
01:00.1 Audio device: NVIDIA Corporation Device yyyy
```

Kalau GPU tidak muncul di sini, masalahnya masih di level hardware, BIOS, atau koneksi fisik.

Tahap 2. Pasang driver NVIDIA melalui paket distro

Untuk host Ubuntu, jalur yang paling aman biasanya **paket resmi Ubuntu**, bukan installer `.run` manual yang sering membuat pemeliharaan lebih sulit. Ubuntu Server secara eksplisit memberi panduan pemasangan driver menggunakan APT dan menjelaskan isu seperti Secure Boot dan penandatanganan kernel module. ([Ubuntu](#))

Langkah eksplorasi paket kandidat:

```
ubuntu-drivers list
ubuntu-drivers devices
```

Contoh keluaran:

```
nvidia-driver-570
nvidia-driver-570-server
```

Untuk server atau komputasi, jalur **-server** sering lebih cocok bila tersedia. Setelah memilih seri driver yang sesuai:

```
sudo apt install -y nvidia-driver-570-server
sudo reboot
```

Setelah reboot, periksa lagi.

Tahap 3. Verifikasi dengan nvidia-smi

nvidia-smi adalah utilitas resmi NVIDIA untuk melihat status GPU, driver, suhu, pemakaian memori, dan proses. NVIDIA mendeskripsikannya sebagai tool untuk monitoring dan manajemen GPU pada Linux dan Windows. ([NVIDIA Docs](#))

Perintah:

```
nvidia-smi
```

Contoh keluaran sehat:

```
+-----+
| NVIDIA-SMI 570.xx                Driver Version: 570.xx          CUDA Version: 12.x        |
+-----+-----+-----+
| GPU  Name                               | Bus-Id                  | Volatile Uncorr. ECC      |
| 0    NVIDIA GeForce RTX ...           | 00000000:01:00.0        | N/A                       |
| Fan  Temp  Perf  Pwr:Usage               | Memory-Usage            | GPU-Util                  |
| 30%   42C   P8     18W / 200W           | 512MiB / 12288MiB       | 3%                        |
+-----+-----+-----+
```

Kalau keluaran ini sudah ada, itu tanda besar bahwa host sudah melewati gerbang penting.

Tahap 4. Pahami gejala umum bila belum sehat

Beberapa gejala umum:

- `nvidia-smi: command not found`
Biasanya driver belum terpasang benar.
- `No devices were found`
Bisa berarti GPU tidak terbaca, driver tidak cocok, atau ada masalah Secure Boot.
- `Failed to initialize NVML`
Sering menandakan ada masalah akses driver atau runtime. ([NVIDIA Docs](#))

Buku ini sebaiknya menekankan: **jangan lanjut ke container GPU sebelum `nvidia-smi` sehat di host.**

Tahap 5. Dasar keamanan host

Sebelum Docker dipasang, host sudah sebaiknya punya kebersihan minimum:

- pakai user biasa untuk kerja harian,
- jangan buka port sembarangan,
- aktifkan SSH hanya bila perlu,
- batasi akses sudo,
- dan biasakan update keamanan rutin.

Untuk GPU host, prinsipnya sederhana: **GPU itu akselerator, bukan alasan untuk melupakan disiplin admin sistem.**

5.6 Instalasi Docker Engine dan Docker Compose

Setelah OS rapi dan GPU sehat, baru kita masuk ke Docker. Dokumentasi resmi Docker untuk Ubuntu menekankan beberapa hal penting: Ubuntu 26.04 didukung, paket konflik lama sebaiknya dibersihkan dulu, dan Compose modern hadir sebagai **plugin** bernama `docker-compose-plugin`, bukan lagi kebiasaan lama `docker-compose` v1. ([Docker Documentation](#))

Tahap 1. Bersihkan paket konflik lama bila ada

```
for pkg in docker.io docker-doc docker-compose
docker-compose-v2 podman-docker containerd runc; do
    sudo apt remove -y $pkg
done
```

Contoh keluaran bisa berbeda-beda. Bila paket tidak ada, itu normal.

```
Package 'docker.io' is not installed, so not removed
```

Langkah ini penting karena paket bawaan distro atau sisa instalasi lama bisa bentrok dengan paket resmi Docker. ([Docker Documentation](#))

Tahap 2. Pasang prasyarat repo

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
```

Ini bagian persiapan standar sebelum menambahkan repository Docker resmi. ([Docker Documentation](#))

Tahap 3. Tambahkan GPG key dan repository Docker resmi

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
  | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Lalu perbarui indeks paket:

```
sudo apt update
```

Contoh keluaran sehat:

```
Get:1 https://download.docker.com/linux/ubuntu resolute
InRelease
Reading package lists... Done
```

Tahap 4. Pasang Docker Engine dan Compose plugin

```
sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Setelah itu, verifikasi:

```
docker --version
docker compose version
sudo systemctl status docker --no-pager
```

Contoh keluaran:

```
Docker version 28.x.x, build xxxxx
Docker Compose version v2.x.x
```

Docker juga menyediakan panduan resmi *post-installation* untuk Linux, termasuk menjalankan Docker sebagai non-root user dan opsi rootless mode. ([Docker Documentation](#))

Tahap 5. Uji container dasar

```
sudo docker run hello-world
```

Contoh keluaran sehat:

```
Hello from Docker!
This message shows that your installation appears to be
working correctly.
```

Ini verifikasi minimum bahwa daemon Docker berjalan normal.

Tahap 6. Opsi menjalankan Docker tanpa sudo

Secara default, Docker berjalan sebagai root-managed daemon. Dokumentasi resmi Docker menjelaskan bahwa menambahkan user ke grup **docker** memungkinkan user menjalankan Docker tanpa **sudo**, tetapi **grup docker memberi hak istimewa setara root-level privileges**. Karena itu langkah ini nyaman, tetapi perlu dipahami risikonya. ([Docker Documentation](#))

Bila tetap ingin memakainya:

```
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

Contoh keluaran yang berhasil akan sama-sama menampilkan pesan **Hello from Docker!**.

Untuk pembaca yang ingin lebih ketat, Docker juga menyediakan **rootless mode**, tetapi untuk buku ini jalur paling mudah biasanya tetap Docker standar dulu, selama pembaca memahami implikasi keamanan grup **docker**. ([Docker Documentation](#))

Tahap 7. Catatan menuju GPU container

Pada tahap ini host sudah siap untuk Docker biasa. Tetapi untuk container yang memakai GPU, masih ada satu lapisan lagi: **NVIDIA Container Toolkit**. NVIDIA menegaskan bahwa toolkit ini memungkinkan container mengakses GPU host dan harus dikonfigurasi setelah driver host dan container engine sudah benar. Verifikasi tipikal stack ini sering memakai uji container `ubuntu nvidia-smi` setelah runtime NVIDIA aktif. ([NVIDIA Docs](#))

Karena bab ini memang berhenti di fondasi host, kita cukup pastikan:

- Ubuntu sehat,
- GPU sehat,
- Docker sehat,
- Compose sehat.

Itu sudah kemenangan besar.

Ringkasan langkah cepat bab ini

Agar mudah diingat, urutan kerja yang paling aman adalah:

- **Pasang Ubuntu 26.04**
- **Update sistem dan reboot**
- **Rapikan user, hostname, SSH, dan firewall**
- **Pasang driver NVIDIA**
- **Pastikan `nvidia-smi` sehat**
- **Pasang Docker Engine resmi**
- **Pasang Compose plugin**
- **Uji `hello-world`**
- **Baru nanti lanjut ke runtime GPU dan stack AI**

Kalau urutannya dibalik, pembaca sering tersesat. Kalau urutannya disiplin, host akan jauh lebih mudah dipelihara.

Penutup bab

Bab ini harus meninggalkan satu pelajaran penting: **host yang baik bukan yang paling heboh, tetapi yang paling tertib**. Ubuntu 26.04 memberi fondasi modern dan dukungan LTS; NVIDIA memberi akselerasi yang dibutuhkan; Docker memberi wadah kerja yang rapi. Tetapi ketiganya hanya akan menjadi fondasi yang kuat bila dipasang dengan urutan yang masuk akal: **OS dulu, keamanan dasar dulu, driver dulu, verifikasi dulu, baru container**. ([Dokumentasi Ubuntu](#))

Bagi mahasiswa dan praktisi, ini tantangan yang sehat. **Jangan buru-buru mengejar model besar. Menaklukkan host dengan rapi adalah kemenangan awal yang jauh lebih penting**. Setelah fondasi ini stabil, barulah stack *agentic AI* di atasnya punya peluang besar untuk benar-benar bekerja.

Bab 6. Menyiapkan fondasi AI lokal: Ollama + Open WebUI

6.1 Instalasi Ollama di Linux

Ollama adalah fondasi model lokal di buku ini. Perannya sederhana: **menyediakan runtime untuk menjalankan model AI secara lokal**, memudahkan penarikan model, memberi API yang konsisten, dan menjadi “mesin belakang” yang nanti akan dipakai oleh Open WebUI, workflow, maupun agent lain. Dokumentasi resmi Linux dari Ollama menyediakan jalur instalasi sangat singkat melalui *install script*, dan juga menyediakan jalur *manual install* untuk mereka yang ingin lebih sadar terhadap isi paket yang dipasang. ([Ollama](#))

Untuk pembaca pemula, jalur termudah adalah *install script* resmi berikut:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Perintah ini adalah jalur instalasi Linux yang direkomendasikan di dokumentasi resmi Ollama. ([Ollama](#))

Setelah instalasi selesai, kita bisa memeriksa apakah biner Ollama sudah tersedia:

```
ollama -v
```

Contoh keluaran yang sehat biasanya mirip seperti ini:

```
ollama version is 0.x.x
```

Dokumentasi *manual install* Ollama sendiri mencontohkan bahwa setelah **ollama serve** dijalankan, verifikasi sederhana dapat dilakukan dengan **ollama -v**. ([Ollama](#))

Bagi pembaca yang lebih hati-hati, jalur *manual install* juga tersedia. Ini berguna bila kita ingin memahami bahwa Ollama pada dasarnya hanya perlu diekstrak ke sistem, lalu dijalankan sebagai layanan.

```
curl -fsSL
https://ollama.com/download/ollama-linux-amd64.tar.zst | sudo
tar x -C /usr
ollama serve
```

Dokumentasi resmi Linux menampilkan pola ini untuk *manual install*. ([Ollama](#))

Catatan penting: pada Linux, Ollama menyimpan model di lokasi default sistem. Dokumentasi FAQ menyebut lokasi model pada Linux berada di **/usr/share/ollama/.ollama/models**, dan lokasi ini bisa dipindahkan dengan variabel lingkungan **OLLAMA_MODELS** bila diperlukan. Itu berguna bila kita ingin menaruh model di disk yang lebih besar. ([Ollama](#))

Secara praktis, ada tiga kebiasaan baik setelah Ollama terpasang:

- **cek versi** agar tahu instalasi berhasil,
- **cek ruang disk** karena model bisa besar,
- dan **pastikan host Anda tidak kehabisan RAM/VRAM** saat mulai menarik model.

Bab ini perlu menanamkan satu pelajaran awal: **memasang Ollama itu mudah; merawatnya tetap butuh disiplin kapasitas.**

6.2 Menjalankan model lokal untuk tugas bisnis

Setelah Ollama terpasang, langkah berikutnya adalah menarik model pertama dan menjalankannya. Di sinilah pembaca mulai melihat bahwa AI lokal bukan konsep abstrak. Ia benar-benar bisa dipanggil, diajak bicara, dan diuji untuk pekerjaan nyata. Dokumentasi resmi Ollama mencontohkan penggunaan cepat melalui perintah API dan menyebut model contoh seperti [gemma3](#); dokumentasi lain juga menunjukkan bahwa berbagai alat integrasi dapat mendeteksi model yang sudah di-*pull* ke Ollama. ([Ollama](#))

Pola kerja paling sederhana adalah:

1. **tarik model,**
2. **jalankan model,**
3. **uji pertanyaan bisnis kecil,**
4. **lihat apakah host cukup kuat.**

Contoh menarik model:

```
ollama pull gemma3
```

Contoh keluaran yang lazim akan menunjukkan proses unduh *manifest*, *layers*, lalu verifikasi.

```
pulling manifest
pulling ...
verifying sha256 digest
writing manifest
success
```

Nama model dapat disesuaikan, tetapi prinsipnya sama: **tarik model secukupnya dulu**, jangan langsung memburu model besar hanya karena penasaran.

Dokumentasi Ollama menampilkan contoh request dengan [gemma3](#), sehingga model ini aman dijadikan contoh awal yang netral. ([Ollama](#))

Setelah model tersedia, jalankan sesi interaktif:

```
ollama run gemma3
```

Lalu coba pertanyaan yang dekat dengan konteks bisnis:

Buat ringkasan singkat memo internal tentang keterlambatan approval proyek.

Contoh keluaran yang sehat tidak harus “sempurna”. Yang penting, model merespons, host tidak macet, dan kita mulai punya rasa tentang kemampuan model lokal.

Untuk pembaca buku ini, cara memilih model awal sebaiknya sangat praktis:

- **model kecil–menengah** untuk uji coba awal,
- **model yang cukup baik di instruksi dan ringkasan** untuk pekerjaan bisnis,
- **hindari langsung model besar** bila VRAM/RAM Anda terbatas,
- dan **uji pada tugas nyata**, bukan hanya pertanyaan lucu.

Dokumentasi Ollama juga menegaskan bahwa panjang konteks default sangat dipengaruhi VRAM: di bawah 24 GiB VRAM default-nya lebih pendek, sementara tugas yang butuh konteks besar—seperti *agents*, *web search*, dan coding tools—disarankan menuju setidaknya 64k token. Ini penting karena pembaca buku akan segera masuk ke RAG dan agentic workflow. ([Ollama](#))

Kalimat penting yang perlu ditanamkan: **jangan ukur model dari “wow”-nya, ukur dari apakah ia cukup berguna untuk pekerjaan Anda.**

6.3 Menjalankan Ollama sebagai service dan mengakses API lokal

Di lingkungan kerja nyata, Ollama tidak cukup hanya dijalankan sesekali lewat terminal. Ia perlu dipahami sebagai **layanan lokal** yang siap melayani aplikasi lain. Dokumentasi API resmi menyebutkan bahwa setelah Ollama berjalan, API otomatis tersedia pada <http://localhost:11434/api>. FAQ resmi juga menjelaskan bahwa secara default Ollama *bind* ke 127.0.0.1:11434, sehingga aman sebagai layanan lokal dan tidak otomatis terbuka ke jaringan. ([Ollama](#))

Cara paling sederhana untuk menjalankan server Ollama secara manual adalah:

```
ollama serve
```

Perintah ini menyalakan layanan HTTP lokal untuk model-model Ollama.

Dokumentasi Linux *manual install* memang menggunakan `ollama serve` sebagai jalur start dasar. ([Ollama](#))

Setelah server aktif, uji API lokal dengan `curl`:

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3",
  "prompt": "Jelaskan singkat manfaat agentic AI untuk helpdesk
internal."
}'
```

Contoh keluaran yang lazim berbentuk JSON berisi teks respons model:

```
{
```

```

    "model": "gemma3",
    "response": "Agentic AI dapat membantu helpdesk internal
dengan membaca tiket, mencari SOP terkait, dan menyiapkan
respons awal.",
    "done": true
}

```

Ini mengikuti pola request yang ditampilkan di dokumentasi API Ollama. ([Ollama](#))

Untuk gaya *chat*, dokumentasi Quickstart juga memberi contoh endpoint `/api/chat`:

```

curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [
    { "role": "user", "content": "Buat 3 ide use case agentic AI
untuk approval workflow." }
  ]
}'

```

Contoh keluaran:

```

{
  "model": "gemma3",
  "message": {
    "role": "assistant",
    "content": "Tiga use case awal: klasifikasi dokumen,
pengecekan kelengkapan, dan routing ke approver yang tepat."
  },
  "done": true
}

```

Pola ini ditunjukkan di dokumentasi Quickstart Ollama. ([Ollama](#))

Ada dua hal penting yang perlu dipahami pembaca:

- Secara lokal, Ollama tidak memerlukan autentikasi saat diakses via <http://localhost:11434>.
- Secara default, ia hanya mendengar di loopback lokal.

Dua sifat ini membuatnya nyaman untuk host tunggal. Tetapi bila kita ingin membukanya ke jaringan, dokumentasi FAQ menegaskan bahwa alamat bind bisa diubah lewat `OLLAMA_HOST`, dan akses lintas origin tambahan dapat diatur lewat `OLLAMA_ORIGINS`. ([Ollama](#))

Contoh membuka bind ke semua interface:

```
OLLAMA_HOST=0.0.0.0:11434 ollama serve
```

Ini mengikuti prinsip konfigurasi `OLLAMA_HOST` yang didokumentasikan di FAQ. ([Ollama](#))

Namun untuk buku ini, saran paling aman adalah: **biarkan Ollama lokal dulu**, lalu hubungkan Open WebUI ke host yang sama. Itu jauh lebih sederhana untuk belajar dan jauh lebih kecil risikonya.

6.4 Instalasi Open WebUI dengan Docker

Setelah mesin model siap, kita masuk ke Open WebUI. Di sinilah pengguna manusia mulai punya “meja kerja” yang enak dipakai. Dokumentasi resmi Open WebUI sangat jelas: untuk banyak pengguna, **Docker adalah jalur tercepat dan yang direkomendasikan**, dan *quick start* Docker menjadi jalur utama dari nol ke UI yang hidup. Dokumentasi juga menekankan bahwa image resmi tersedia di ghcr.io/open-webui/open-webui maupun Docker Hub [openwebui/open-webui](https://openwebui.com/open-webui). ([Open WebUI](#))

Langkah pertama adalah menarik image resmi:

```
docker pull ghcr.io/open-webui/open-webui:main
```

Perintah ini sesuai dengan Quick Start resmi Open WebUI. ([Open WebUI](#))

Setelah image tersedia, jalankan container dasar:

```
docker run -d \
  -p 3000:8080 \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

Contoh keluaran:

```
a1b2c3d4e5f6g7h8i9j0...
```

Quick Start resmi menjelaskan arti dua flag terpenting:

- `-p 3000:8080` membuka UI di port 3000 host,
- `-v open-webui:/app/backend/data` membuat data tetap tersimpan saat container di-*restart* atau dibuat ulang. ([Open WebUI](#))

Setelah container hidup, periksa statusnya:

```
docker ps
```

Contoh keluaran yang sehat:

CONTAINER ID	IMAGE	PORTS	NAMES
a1b2c3d4e5f6	open-webui:main	0.0.0.0:3000->8080/tcp	open-webui

Lalu buka browser ke:

`http://localhost:3000`

Quick Start resmi memang mengarahkan pengguna ke URL ini setelah container jalan. ([Open WebUI](#))

Satu catatan keamanan yang penting: dokumentasi *Security Policy* Open WebUI menegaskan bahwa kita sebaiknya **hanya memakai image resmi**, yaitu dari [ghcr.io/open-webui/open-webui](#) atau [openwebui/open-webui](#), karena image tidak resmi berisiko disusupi. Dokumentasi yang sama juga menekankan pentingnya melindungi direktori data `/app/backend/data`. ([Open WebUI](#))

Jadi pesan sederhananya:

- pakai image resmi,
- selalu pasang volume persisten,
- jangan asal ganti image dari sumber tak jelas.

6.5 Menghubungkan Open WebUI ke Ollama

Begitu Open WebUI hidup, langkah berikutnya adalah menghubungkannya ke Ollama. Ini inti fondasi AI lokal buku ini: **Ollama menjalankan model, Open WebUI menjadi antarmuka kerja**. Dokumentasi Open WebUI menyebut bahwa ia mendukung berbagai provider, termasuk Ollama. Dokumentasi Quick Start juga menyediakan pola untuk menghubungkan Open WebUI ke Ollama pada server lain lewat `OLLAMA_BASE_URL`. ([Open WebUI](#))

Ada dua pola umum.

Pola 1. Ollama dan Open WebUI di host yang sama

Bila Ollama berjalan langsung di host Linux, sementara Open WebUI berjalan di Docker, Kita perlu memastikan container dapat mencapai alamat Ollama yang benar. Dokumentasi resmi memang mencontohkan `OLLAMA_BASE_URL` untuk kasus “connecting to Ollama on a different server,” dan konsep yang sama dapat dipakai untuk menunjuk ke alamat host yang menjalankan Ollama. ([Open WebUI](#))

Contoh menjalankan Open WebUI sambil menunjuk ke Ollama:

```
docker rm -f open-webui

docker run -d \
  -p 3000:8080 \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:main
```

Contoh keluaran:

```
f0e1d2c3b4a59687...
```

Lalu cek log singkat:

```
docker logs --tail=50 open-webui
```

Contoh keluaran yang sehat biasanya akan menunjukkan aplikasi berhasil start dan siap menerima koneksi.

```
INFO: Started server process
INFO: Application startup complete
```

Secara praktik, alamat yang dipakai bisa berbeda tergantung jaringan Docker dan host. Yang penting secara konsep adalah: **Open WebUI harus tahu di mana Ollama berada.**

Pola 2. Satu container Open WebUI yang sudah dibundel dengan Ollama

Quick Start Open WebUI juga menyediakan image `:ollama`, yakni satu container yang memadukan Open WebUI dan Ollama. Ada versi GPU dan CPU-only. Ini jalur yang menarik untuk pembaca yang ingin serba cepat. ([Open WebUI](#))

Contoh CPU-only:

```
docker run -d \
  -p 3000:8080 \
  -v ollama:/root/.ollama \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:ollama
```

Contoh GPU:

```
docker run -d \
  -p 3000:8080 \
  --gpus=all \
  -v ollama:/root/.ollama \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:ollama
```

Dokumentasi Quick Start resmi menampilkan dua pola ini. ([Open WebUI](#))

Bagi buku ini, saya sarankan pembaca memahami dulu pola terpisah:

- **Ollama sebagai mesin model,**
- **Open WebUI sebagai front-end.**

Mengapa? Karena pola ini lebih mudah dipelihara, lebih mudah di-debug, dan lebih jelas secara arsitektur.

6.6 Workspace, model preset, knowledge attachment, dan user access

Setelah koneksi berhasil, nilai Open WebUI baru benar-benar terasa pada lapisan kerja sehari-hari. Dokumentasi resmi menjelaskan bahwa Open WebUI bukan hanya UI chat, tetapi *workspace* yang dapat:

- mengelola model,
- membungkus model dengan instruksi,
- menempelkan knowledge,
- mengaktifkan tools,
- dan membangun agent spesialis tanpa mengubah model dasar. ([Open WebUI](#))

a. Workspace sebagai ruang kerja tim

Open WebUI menempatkan dirinya sebagai antarmuka terpusat untuk percakapan, knowledge, tools, dan model. Dalam dokumentasinya, ia bahkan menyebut dirinya sebagai semacam “AI operating system for teams.” Ini cocok sekali untuk konteks bisnis kecil–menengah yang ingin satu pintu akses ke AI lokal. ([Open WebUI](#))

Artinya, begitu kita masuk ke UI, jangan berpikir sempit bahwa ini hanya “tempat chat”. Ini adalah:

- ruang eksperimen,
- ruang kerja knowledge,
- ruang pengelolaan model,
- dan nantinya ruang agent.

b. Model preset

Fitur **Models workspace** di Open WebUI memungkinkan kita membuat *preset* di atas model dasar. Dokumentasi resmi menjelaskan bahwa satu model dasar dapat dibungkus dengan *system prompt*, knowledge bases, tools, skills, dan parameter override agar berubah menjadi agent yang punya perilaku khusus. ([Open WebUI](#))

Dalam praktik buku ini, contoh *preset* yang masuk akal adalah:

- Ringkasan Rapat
- Asisten SOP Internal
- Pemeriksa Draft Memo
- Helpdesk Triage Awal

Artinya, kita tidak perlu memasang model baru untuk setiap tugas. Cukup satu model dasar yang dibungkus dengan konfigurasi perilaku yang tepat.

c. Knowledge attachment

Di sinilah Open WebUI menjadi sangat relevan untuk bisnis. Fitur **Knowledge** memungkinkan dokumen dan koleksi file dicari, dibaca, dan dipakai model untuk menjawab dengan dasar yang lebih kuat. Dokumentasi resmi menyebutkan bahwa *Knowledge* adalah tempat menyimpan file dan koleksi yang bisa dicari dan dipikirkan oleh AI, memakai retrieval (RAG) sehingga model menarik potongan relevan saat diperlukan, bukan menebak. Ia juga mendukung *Focused Retrieval* dan *Full Context*. ([Open WebUI](#))

Dari sudut pandang bisnis, manfaat langsungnya besar:

- SOP bisa ditempelkan ke model,
- kebijakan internal bisa dijadikan dasar jawaban,
- kontrak atau panduan bisa diubah menjadi basis tanya-jawab,
- dan jawaban bisa lebih transparan karena ada sitasi sumber.

Dokumentasi bahkan menegaskan bahwa AI dapat mencari dokumen berdasarkan makna, bukan hanya kata kunci, dan menampilkan sitasi dari tempat jawaban ditemukan. ([Open WebUI](#))

d. User access

Begitu Open WebUI mulai dipakai tim, pengelolaan akses menjadi penting. Dokumentasi *Getting Started* menyinggung adanya pengaturan admin dan user, mode multi-account, serta opsi *single-user mode* dengan `WEBUI_AUTH=False`. Quick Start juga memberi peringatan bahwa setelah mengubah ke *single-user mode*, Kita tidak bisa bolak-balik dengan bebas ke mode multi-account. ([Open WebUI](#))

Untuk host belajar pribadi, *single-user mode* mungkin nyaman:

```
docker rm -f open-webui

docker run -d \
  -p 3000:8080 \
  -e WEBUI_AUTH=False \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

Contoh keluaran:

```
c8d9e0f1a2b3...
```

Tetapi untuk tim atau lab kecil, mode multi-user lebih sehat karena:

- akses lebih jelas,
- perubahan lebih mudah dilacak,
- dan *workspace* bisa dibagi dengan lebih tertib.

Quick Start resmi memang memberi contoh *single-user mode* dan memperingatkan implikasinya. ([Open WebUI](#))

Langkah ringkas dari nol sampai siap dipakai

Untuk memudahkan pembaca, seluruh bab ini bisa diringkas menjadi urutan berikut:

- **Pasang Ollama di Linux**
- **Tarik satu model awal**
- **Uji model via `ollama run`**
- **Jalankan `ollama serve`**
- **Uji API lokal di `localhost:11434/api`**
- **Tarik image resmi Open WebUI**
- **Jalankan container dengan volume persisten**
- **Hubungkan Open WebUI ke Ollama**
- **Masuk ke UI**
- **Buat preset model pertama**
- **Lampirkan knowledge sederhana**
- **Uji satu tugas bisnis nyata**

Urutan ini sederhana, tetapi sangat kuat. Kalau pembaca berhasil sampai titik ini, mereka sebenarnya sudah memiliki **fondasi AI lokal yang serius**, bukan sekadar demo.

Contoh skenario uji sederhana

Agar bab ini terasa hidup, pembaca bisa mencoba satu skenario uji kecil setelah semua aktif.

1. Tarik model:

```
ollama pull gemma3
```

2. Jalankan server Ollama:

```
ollama serve
```

3. Uji API:

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [
    { "role": "user", "content": "Ringkas email internal
tentang keterlambatan approval." }
  ]
}'
```

4. Jalankan Open WebUI:

```
docker run -d \
-p 3000:8080 \
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
-v open-webui:/app/backend/data \
--name open-webui \
ghcr.io/open-webui/open-webui:main
```

5. Buka UI dan buat preset:

- nama preset: **Ringkasan Memo**
- model dasar: **gemma3**
- instruksi sistem: fokus pada ringkasan singkat, risiko, dan tindak lanjut
- lampirkan knowledge bila ada template memo

Semua pola ini konsisten dengan dokumentasi resmi Ollama dan Open WebUI.
([Ollama](#))

Penutup bab

Bab ini harus meninggalkan satu gagasan utama: **AI lokal yang berguna lahir dari pasangan yang tepat antara mesin model dan ruang kerja manusia**. Dalam buku ini, pasangan itu adalah **Ollama + Open WebUI**. Ollama memberi runtime, model, dan API lokal. Open WebUI memberi antarmuka, *workspace*, knowledge, dan jalan menuju agent yang lebih terarah. Dokumentasi resmi keduanya sangat jelas ke arah ini: Ollama tersedia sebagai layanan lokal di `localhost:11434`, sedangkan Open WebUI merekomendasikan Docker sebagai jalur tercepat dan paling praktis untuk banyak pengguna. ([Ollama](#))

Bagi mahasiswa dan praktisi, pelajaran terbesarnya sederhana:

- **pasang dulu yang paling penting,**
- **uji dengan tugas bisnis yang kecil tetapi nyata,**
- **jangan buru-buru ke arsitektur rumit,**
- **dan biasakan memisahkan mesin model dari meja kerja pengguna.**

Kalau Bab 5 membangun rumahnya, maka Bab 6 mulai **menyalakan isi rumah itu**.

Bab 7. Menyiapkan runtime agent dan orkestrasi workflow: OpenClaw / n8n

7.1 Peran OpenClaw sebagai *agent runtime*

Cara paling mudah memahami OpenClaw adalah ini: **OpenClaw adalah pelaku aktifnya**. Kalau Ollama adalah mesin model, dan Open WebUI adalah meja kerja untuk manusia, maka OpenClaw adalah **runtime yang membuat agent bisa bertindak, memakai tool, memegang skills, dan berjalan sebagai agen yang lebih hidup**. Dokumentasi OpenClaw menekankan bahwa instalasi resminya akan memasang OpenClaw, menyiapkan Node bila perlu, lalu menjalankan *onboarding*. Dokumentasi lain menjelaskan bahwa OpenClaw memakai *skill folders* untuk mengajari agent cara menggunakan tools. ([OpenClaw](#))

Dalam bahasa kerja, OpenClaw cocok ketika kita ingin agent:

- membaca konteks,
- memilih tool,
- menjalankan aksi,
- mempertahankan *workspace* sendiri,
- dan beroperasi seperti “staf digital” yang punya perilaku spesifik.

Literatur engineering yang dilampirkan menegaskan bahwa **tool use mengubah reasoning menjadi action**. Tanpa tool, agent hanya bisa berbicara tentang dunia. Dengan tool, agent bisa mengambil data, memanggil API, menulis hasil, atau memicu aksi lain. Itu sebabnya runtime seperti OpenClaw penting: ia menjadi jembatan antara niat model dan tindakan nyata.

Dokumentasi OpenClaw juga memperlihatkan bahwa agent punya *gateway* sendiri, dashboard, konsep *skills*, dan pemisahan *workspace*. Ini berarti OpenClaw cocok dipakai saat pembaca ingin membangun agent yang lebih mandiri, lebih personal, atau lebih kaya perilaku dibanding sekadar node AI di dalam satu workflow. (docs.openwebui.com)

7.2 Peran n8n sebagai orkestrator *workflow* bisnis

Kalau OpenClaw adalah “pelaku”, maka **n8n adalah sutradara alur kerjanya**. Ia sangat kuat ketika proses bisnis perlu dipetakan sebagai rangkaian langkah yang jelas: ada trigger, ada percabangan, ada persetujuan, ada notifikasi, ada jadwal, dan ada integrasi ke banyak sistem. Dokumentasi resmi n8n untuk Docker sangat jelas: n8n diposisikan sebagai aplikasi yang dijalankan lewat container, dengan volume persisten, timezone, dan task runners yang memang ditujukan untuk operasi workflow yang stabil. (docs.n8n.io)

Dalam konteks bisnis, n8n sangat cocok untuk pola seperti:

- email masuk → klasifikasi → kirim ke PIC,
- tiket masuk → triase → approval → update sistem,
- dokumen masuk → cek kelengkapan → antrian review,
- jadwal periodik → ambil data → ringkas → kirim laporan.

Di sini AI bukan “raja tunggal”. AI adalah **salah satu komponen di dalam workflow**. Ini pendekatan yang sehat, karena banyak organisasi sebenarnya lebih butuh **proses yang rapi** daripada agent yang tampak canggih tetapi liar.

Dokumentasi *human-in-the-loop* n8n juga penting sekali untuk buku ini. n8n mendokumentasikan bahwa sebelum AI Agent menjalankan tool tertentu, workflow bisa berhenti dan menunggu manusia untuk **Approve** atau **Deny**. Ini sangat cocok untuk aksi berisiko: mengirim pesan, mengubah record, menghapus data, atau langkah yang punya dampak bisnis besar. (docs.n8n.io)

Jadi, dalam bahasa yang lebih tajam: **n8n unggul ketika yang ingin Anda kendalikan adalah proses. OpenClaw unggul ketika yang ingin Anda bentuk adalah perilaku agent.**

7.3 Instalasi OpenClaw via installer resmi

Bagian ini perlu sangat praktis. Dokumentasi resmi OpenClaw menyebut bahwa cara tercepat instalasi adalah *installer script* yang mendeteksi OS, memasang Node bila perlu, memasang OpenClaw, lalu menjalankan *onboarding*. Untuk Linux/macOS/WSL2, perintah resminya adalah: ([OpenClaw](https://openclaw.ai))

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Contoh keluaran yang sehat biasanya memperlihatkan alur seperti ini:

```
Detecting platform...
Installing Node.js...
Installing OpenClaw...
Launching onboarding...
```

Dokumentasi juga menyediakan opsi tanpa *onboarding* otomatis, yang berguna untuk otomatisasi atau bila pembaca ingin menunda konfigurasi awal: ([OpenClaw](https://openclaw.ai))

```
curl -fsSL https://openclaw.ai/install.sh | bash -s --
--no-onboard
```

Setelah instalasi, verifikasi minimum yang disarankan dokumentasi adalah:

```
openclaw --version
openclaw doctor
openclaw gateway status
```

Dokumentasi OpenClaw memang menyebut tiga langkah ini sebagai verifikasi dasar: CLI tersedia, konfigurasi sehat, dan *gateway* berjalan. ([OpenClaw](https://openclaw.ai))

Contoh keluaran:

```
openclaw 0.x.x
Doctor: OK
Gateway: not running
```

Kalau *gateway* belum jalan, pembaca dapat memulainya:

```
openclaw gateway
```

Dan memeriksanya lagi:

```
openclaw gateway status
```

Dokumentasi integrasi OpenClaw dengan Open WebUI menegaskan bahwa *gateway* ini berjalan di **port 18789** secara default, dan dapat diverifikasi dengan `openclaw gateway status`. (docs.openwebui.com)

Contoh keluaran sehat:

```
Gateway running on http://localhost:18789
```

Kalau pembaca ingin menghubungkan OpenClaw ke Open WebUI, dokumentasi Open WebUI menjelaskan bahwa endpoint *OpenAI-compatible* harus diaktifkan di konfigurasi gateway. Contoh konfigurasi yang ditunjukkan dokumentasi adalah mengaktifkan `chatCompletions.enabled`. (docs.openwebui.com)

Contoh potongan konfigurasi:

```
{
  "gateway": {
    "http": {
      "endpoints": {
        "chatCompletions": {
          "enabled": true
        }
      }
    }
  }
}
```

Setelah konfigurasi diubah, *gateway* perlu di-*restart* agar perubahan berlaku. Dokumentasi koneksi Open WebUI juga menegaskan bahwa URL koneksinya harus berbentuk `http://localhost:18789/v1`, dan jika Open WebUI berjalan di Docker, `localhost` harus diganti menjadi `host.docker.internal`. (docs.openwebui.com)

7.4 Instalasi n8n via Docker Compose

Untuk n8n, dokumentasi resmi mendorong dua jalur utama: **Docker cepat** dan **Docker Compose**. Untuk buku ini, Docker Compose lebih cocok karena:

- lebih rapi,
- mudah diulang,
- mudah dibaca mahasiswa dan praktisi,
- dan lebih dekat ke pola *self-hosting* yang sehat. (docs.n8n.io)

Dokumentasi n8n untuk Docker Compose menjelaskan langkah-langkah umum: buat direktori proyek, siapkan `.env`, buat direktori `local-files`, tulis `compose.yaml`, lalu jalankan `docker compose up -d`. Dokumentasi *single container* juga menunjukkan pola cepat untuk mulai dengan volume `n8n_data` dan port `5678`. (docs.n8n.io)

Tahap 1. Buat direktori kerja

```
mkdir n8n-compose
cd n8n-compose
mkdir local-files
```

Langkah ini selaras dengan dokumentasi Compose resmi, yang memang meminta pembaca membuat direktori proyek dan `local-files` lebih dulu. (docs.n8n.io)

Tahap 2. Buat file `.env`

Contoh isi minimum yang praktis untuk pembaca Indonesia:

```
GENERIC_TIMEZONE=Asia/Jakarta
TZ=Asia/Jakarta
```

Dokumentasi n8n menekankan bahwa `TZ` mengatur timezone sistem di container, sedangkan `GENERIC_TIMEZONE` penting untuk node yang bergantung jadwal, seperti *Schedule Trigger*. (docs.n8n.io)

Tahap 3. Buat `compose.yaml`

Untuk buku ini, versi sederhana yang cukup aman adalah:

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    restart: always
    ports:
      - "5678:5678"
    env_file:
      - .env
    environment:
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
      - N8N_RUNNERS_ENABLED=true
    volumes:
      - n8n_data:/home/node/.n8n
      - ./local-files:/files

volumes:
  n8n_data:
```

Konfigurasi ini mengikuti prinsip dari dokumentasi resmi Docker n8n: port [5678](#), volume persisten, timezone, *secure file permissions*, dan *task runners* aktif. ([docs.n8n.io](#))

Tahap 4. Jalankan Docker Compose

```
sudo docker compose up -d
```

Dokumentasi Docker Compose n8n memang menampilkan perintah ini sebagai langkah start utama. ([docs.n8n.io](#))

Contoh keluaran:

```
[+] Running 2/2
✓ Volume "n8n-compose_n8n_data" created
✓ Container n8n started
```

Tahap 5. Verifikasi container

```
docker ps
docker logs --tail=50 n8n
```

Contoh keluaran `docker ps`:

CONTAINER ID	IMAGE	PORTS	NAMES
abcd1234efgh	n8nio/n8n	0.0.0.0:5678->5678/tcp	n8n

Setelah itu, buka browser ke:

`http://localhost:5678`

Kalau semua sehat, pembaca akan masuk ke layar awal n8n untuk membuat akun admin dan mulai menyusun workflow.

7.5 *Webhook, tools, skill, approval flow, dan scheduling*

Di sinilah runtime agent dan orkestrasi workflow mulai terasa sebagai sistem kerja nyata.

a. *Webhook*

Webhook adalah pintu masuk otomatis dari dunia luar ke workflow. Ia sangat berguna untuk:

- menerima form atau data dari aplikasi lain,
- menerima event dari layanan eksternal,
- menerima trigger dari sistem internal.

Dalam konteks n8n, *webhook* sering menjadi node pertama yang membangunkan workflow. Dalam konteks bisnis, ia cocok untuk:

- formulir permintaan internal,
- tiket yang dikirim sistem lain,
- atau callback dari aplikasi layanan.

b. *Tools*

Dari sudut agent, *tools* adalah kemampuan bertindak. Literatur yang dilampirkan menjelaskan bahwa *tool calling* berarti LLM menghasilkan panggilan terstruktur ke fungsi atau layanan, tetapi eksekusinya tetap harus dilakukan oleh runtime atau controller. Tool dapat berupa API, pencarian pengetahuan, pembacaan database, notifikasi, atau tindakan lain.

Dokumentasi Ollama sendiri menegaskan bahwa Ollama mendukung *tool calling* atau *function calling*, sehingga model dapat meminta tool dipanggil lalu memakai hasilnya dalam respons. Ini penting untuk pembaca buku karena berarti fondasi model lokal yang dipasang di Bab 6 memang siap dipakai untuk *agent workflows* yang lebih serius. (docs.ollama.com)

c. Skill

Untuk OpenClaw, *skills* adalah lapisan yang sangat khas. Dokumentasi resminya menjelaskan bahwa setiap *skill* adalah sebuah direktori yang berisi **SKILL.md** dengan *frontmatter* YAML dan instruksi. OpenClaw memuat *skills* bawaan maupun lokal, lalu menyaringnya sesuai lingkungan dan konfigurasi. ([OpenClaw](#))

Artinya, *skill* bisa dipahami sebagai:

- cara mengajari agent gaya kerja tertentu,
- paket instruksi penggunaan tools,
- atau kemampuan spesialis yang dapat dipasang per *workspace* atau per agent.

Ini sangat menarik untuk bisnis, karena kita bisa punya:

- *skill* untuk helpdesk,
- *skill* untuk pemeriksaan dokumen,
- *skill* untuk routing approval,
- *skill* untuk ringkasan rapat.

Jadi, **tool adalah tangan**, sedangkan **skill adalah kebiasaan kerja tangan itu**.

d. Approval flow

Untuk alur yang punya risiko, manusia tetap perlu memegang tombol akhir. Dokumentasi n8n tentang *human-in-the-loop* sangat eksplisit: bila tool tertentu ditandai butuh review, workflow akan berhenti dan menunggu manusia untuk **Approve** atau **Deny**. n8n juga menjelaskan bahwa pola ini sangat berguna untuk aksi yang tidak bisa dibalik, komunikasi keluar, keputusan bernilai tinggi, dan situasi yang masih membangun kepercayaan pada workflow AI. (docs.n8n.io)

Ini tepat sekali untuk buku Anda, karena memberi pesan yang sehat:

- **AI boleh cepat,**
- **tetapi bisnis tetap butuh pagar.**

e. Scheduling

Tidak semua workflow harus bangun karena *webhook*. Banyak juga yang bekerja berbasis waktu:

- setiap pagi membuat ringkasan,
- setiap jam memeriksa antrian,
- setiap minggu membuat laporan,
- atau setiap akhir hari menyiapkan draft tindak lanjut.

Di sini n8n unggul karena *schedule-oriented nodes* memang menjadi bagian alami dari ekosistemnya, dan dokumentasi resminya sampai menekankan pentingnya **GENERIC_TIMEZONE** untuk node semacam ini. (docs.n8n.io)

7.6 Kapan memakai OpenClaw, kapan n8n, kapan dipadukan

Ini bagian yang harus sangat jernih, karena banyak pembaca akan bertanya: **jadi saya pakai yang mana?**

Jawaban sederhananya bukan “pilih satu yang paling hebat”, tetapi **pilih sesuai gaya kerja dan bentuk masalah**.

Pakai OpenClaw ketika:

- Anda ingin **agent yang aktif** dan terasa seperti pelaku kerja.
- Anda butuh **tools + skills + workspace** agent.
- Anda ingin agent yang punya perilaku spesifik dan bisa dihubungkan ke Open WebUI.
- Anda lebih fokus ke **runtime agent**, bukan diagram alur proses.

Dokumentasi OpenClaw dan integrasinya dengan Open WebUI memang menegaskan pola ini: gateway, agent routing lewat model ID seperti **openclaw/default**, dan kemampuan agent untuk muncul di dropdown model Open WebUI. (docs.openwebui.com)

Pakai n8n ketika:

- Anda ingin **workflow bisnis** yang jelas, bertahap, dan mudah diaudit.
- Anda perlu trigger, branching, scheduling, notifikasi, approval, dan integrasi sistem.
- Anda ingin memadukan AI dengan node proses biasa.
- Anda perlu *human-in-the-loop* yang eksplisit untuk aksi berisiko.

Dokumentasi n8n sangat kuat di area ini, terutama pada jalur Docker/Compose dan approval tool calls. (docs.n8n.io)

Padukan keduanya ketika:

- Anda ingin **workflow** yang rapi, tetapi pada titik tertentu perlu **agent runtime** yang lebih kaya.
- Anda butuh n8n sebagai sutradara proses, sementara OpenClaw menjadi aktor spesialis.
- Anda ingin Open WebUI menjadi antarmuka manusia, OpenClaw menjadi agent, dan n8n menjadi penggerak proses bisnis di belakang.

Ini pola yang paling menarik untuk organisasi yang mulai serius, tetapi belum mau meloncat ke arsitektur yang terlalu rumit.

Kalimat paling penting untuk pembaca adalah ini:

- **OpenClaw lebih dekat ke “siapa yang bekerja”.**
- **n8n lebih dekat ke “bagaimana pekerjaan mengalir”.**
- **Gabungan keduanya cocok saat organisasi butuh pelaku yang kuat di dalam proses yang tertib.**

Contoh *step by step* ringkas: dari nol sampai dua komponen hidup

Agar bab ini terasa sangat praktis, pembaca bisa mengikuti urutan berikut:

Jalur OpenClaw

1. Pasang OpenClaw:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

2. Verifikasi:

```
openclaw --version  
openclaw doctor
```

3. Jalankan gateway:

```
openclaw gateway
```

4. Cek status:

```
openclaw gateway status
```

5. Bila ingin ke Open WebUI, tambahkan koneksi ke:

```
http://localhost:18789/v1
```

Semua langkah ini mengikuti dokumentasi resmi OpenClaw dan panduan integrasi Open WebUI. ([OpenClaw](#))

Jalur n8n

1. Buat direktori:

```
mkdir n8n-compose && cd n8n-compose && mkdir  
local-files
```

2. Siapkan `.env`

3. Siapkan `compose.yaml`

4. Jalankan:

```
sudo docker compose up -d
```

5. Verifikasi:

```
docker ps  
docker logs --tail=50 n8n
```

6. Buka:

```
http://localhost:5678
```

Langkah-langkah ini konsisten dengan dokumentasi Docker Compose n8n.
(docs.n8n.io)

Penutup bab

Bab ini harus menanamkan satu gagasan kunci: **agentic AI tidak cukup hanya punya model; ia butuh pelaku dan alur kerja**. Di dalam buku ini, pelaku itu direpresentasikan oleh **OpenClaw**, dan alur kerja itu direpresentasikan oleh **n8n**. Keduanya tidak saling meniadakan. Justru keduanya mengisi ruang yang berbeda: runtime agent dan orkestrasi workflow. Disini diposisikan OpenClaw dan n8n sebagai fondasi berikutnya setelah Ollama dan Open WebUI.

Bagi mahasiswa dan praktisi, pesan akhirnya sederhana:

- kalau ingin agent yang aktif, lihat OpenClaw;
- kalau ingin alur proses yang rapi, lihat n8n;
- kalau ingin sistem kerja yang lengkap, pikirkan cara menggabungkan keduanya secara hemat dan masuk akal.

Bab 6 menyalakan mesin dan meja kerjanya. **Bab 7** mulai mengajari mesin itu bekerja di dalam proses nyata.

Bagian IV. RAG, Knowledge, Dataset, dan Data Kerja Agent

Bab 8. RAG untuk bisnis: membangun otak eksternal yang bisa diaudit

8.1 Mengapa agent bisnis butuh RAG

Banyak orang berharap model AI bisa “tahu semuanya.” Itu harapan yang terlalu romantis. Dalam dunia bisnis, pengetahuan yang kita perlukan sering justru berada di luar model: SOP internal, kebijakan HR, notulen rapat, katalog layanan, template kontrak, panduan approval, FAQ internal, email penting, sampai dokumen proyek yang terus berubah. **Kalau agent hanya mengandalkan pengetahuan bawaan model, ia akan cepat terlihat pintar, tetapi mudah meleset.** RAG dibangun untuk mengatasi masalah ini. Ia membiarkan model mengambil potongan dokumen yang relevan saat dibutuhkan, lalu menyusun jawaban berdasarkan konteks tersebut. Open WebUI menjelaskan bahwa *Knowledge* menyimpan file dan koleksi yang bisa dicari, dibaca, dan dipikirkan AI, sementara mode RAG hanya menarik potongan yang relevan saat dibutuhkan agar tidak memenuhi jendela konteks secara sia-sia. ([Open WebUI](#))

Dalam bahasa bisnis, manfaat RAG ada tiga dan sangat konkret.

- **Pengetahuan bisa diperbarui tanpa melatih ulang model.**
Saat SOP, harga, kebijakan, atau daftar PIC berubah, Kita cukup memperbarui dokumen atau basis pengetahuan, bukan membangun ulang model dari nol. Open WebUI secara eksplisit menempatkan *Knowledge* sebagai tempat dokumen yang dapat dicari dan dibaca model saat menjawab. ([Open WebUI](#))
- **Jawaban lebih tertambat pada fakta.**
Karena model diberi dokumen relevan sebelum menyusun jawaban, peluang halusinasi turun. Open WebUI juga menekankan bahwa AI akan “mencari berdasarkan makna, bukan sekadar kata kunci,” lalu “mengutip di mana jawaban ditemukan.” ([Open WebUI](#))
- **Jejak sumber lebih mudah diaudit.**
Di lingkungan bisnis, terutama yang menyentuh kebijakan dan keputusan, jawaban tanpa sumber adalah masalah. Dokumentasi Open WebUI menekankan bahwa batas dokumen dijaga untuk membantu sitasi, atribusi sumber, dan konteks retrieval. ([Open WebUI](#))

Itulah sebabnya RAG jauh lebih cocok untuk banyak kebutuhan bisnis dibanding sekadar bertanya ke model umum. **Model umum memberi kelancaran bahasa. RAG memberi pijakan.**

8.2 Knowledge base di Open WebUI

Untuk buku ini, Open WebUI adalah pintu masuk paling nyaman untuk membangun *knowledge base*. Dokumentasi resminya menjelaskan bahwa bagian **Knowledge** adalah tempat menyimpan file dan koleksi yang dapat dicari, dibaca, dan dinalar oleh AI. Dokumen bisa berupa PDF, spreadsheet, kode, atau dokumen teks lain. Ia juga mendukung *scoped access*, artinya basis pengetahuan bisa ditempelkan hanya ke model atau agent tertentu agar tetap rapi dan relevan. ([Open WebUI](#))

Cara berpikir yang sehat

Jangan membayangkan *knowledge base* sebagai “tempat melempar semua file.” Itu resep kekacauan. Bayangkan ia sebagai **perpustakaan kerja**. Perpustakaan yang baik bukan yang paling banyak bukunya, tetapi yang:

- raknya jelas,
- koleksinya bersih,
- versi dokumennya terkontrol,
- dan pengguna tahu buku mana yang relevan untuk masalah tertentu.

Langkah demi langkah menyiapkan knowledge base

Tahap 1. Siapkan folder sumber dokumen

Misalnya kita ingin membangun *knowledge base* untuk satu fungsi bisnis sederhana: SOP dan kebijakan internal.

```
mkdir -p ~/kb-bisnis/{sop,kebijakan,faq}
```

Isi folder contoh:

```
cat > ~/kb-bisnis/sop/approval-pengadaan.txt <<'EOF'
SOP Approval Pengadaan
1. Permintaan diajukan oleh unit kerja.
2. Verifikasi anggaran dilakukan oleh keuangan.
3. Approval akhir oleh kepala unit.
4. Semua dokumen harus tersimpan di arsip proyek.
EOF

cat > ~/kb-bisnis/kebijakan/akses-data.txt <<'EOF'
Kebijakan Akses Data
- Data pelanggan hanya boleh diakses sesuai peran.
- File sensitif tidak boleh dibagikan ke pihak luar.
- Semua akses harus tercatat.
EOF
```

```
cat > ~/kb-bisnis/faq/helpdesk.txt <<'EOF'  
FAQ Helpdesk  
Tiket prioritas tinggi harus ditindaklanjuti dalam 4  
jam kerja.  
EOF
```

Contoh keluaran tidak akan menampilkan apa-apa bila file berhasil dibuat.

Tahap 2. Masuk ke Open WebUI

Buka:

`http://localhost:3000`

Masuk ke **Workspace** → **Knowledge**.

Tahap 3. Buat *collection* baru

Buat satu *knowledge base* bernama misalnya:

- **SOP Internal**
- **Kebijakan Internal**
- **FAQ Helpdesk**

Open WebUI menjelaskan bahwa kita bisa membangun koleksi berdasarkan proyek, tim, atau topik. Itu sangat membantu agar basis pengetahuan tidak menjadi gudang serba campur. ([Open WebUI](#))

Tahap 4. Unggah file

Unggah dokumen yang relevan ke koleksi yang tepat. Di tahap awal, lebih baik:

- mulai dari dokumen pendek dan jelas,
- hindari file yang isinya duplikat,
- dan hindari file usang yang belum dibersihkan.

Tahap 5. Tempelkan *knowledge base* ke model

Masuk ke *model preset* atau agent yang ingin dipakai, lalu *attach knowledge base* tersebut. Open WebUI menjelaskan bahwa basis pengetahuan bisa ditempelkan spesifik ke model agar model hanya mencari di ruang yang relevan. ([Open WebUI](#))

Tahap 6. Uji pertanyaan nyata

Misalnya:

- “Siapa yang melakukan approval akhir pengadaan?”
- “Berapa SLA tiket prioritas tinggi?”
- “Apa aturan dasar akses data pelanggan?”

Kalau *knowledge base* dibangun dengan baik, model tidak hanya menjawab, tetapi juga menunjuk sumber.

Prinsip praktis yang perlu ditanamkan

- Pisahkan koleksi berdasarkan fungsi atau domain.
- Jangan campur SOP, kebijakan, dan arsip acak tanpa struktur.
- Lampirkan hanya basis pengetahuan yang relevan ke model tertentu.
- Mulai kecil, lalu perluas.

8.3 Struktur dokumen yang baik untuk retrieval

Salah satu kesalahan terbesar dalam RAG adalah mengira semua dokumen otomatis “menjadi cerdas” setelah diunggah. Tidak begitu. **RAG sangat bergantung pada kualitas dokumen sumber.** Kalau dokumen berantakan, retrieval juga berantakan.

Dokumen yang baik untuk retrieval biasanya punya ciri seperti ini:

- Judul jelas
- Subbagian konsisten
- Bahasa langsung
- Versi dokumen diketahui
- Tidak terlalu banyak isi campuran dalam satu file
- Tidak penuh tabel atau gambar tanpa penjelasan teks

Mengapa ini penting? Karena mesin retrieval bekerja dengan potongan teks, struktur, dan metadata. Open WebUI menegaskan bahwa dalam pipeline RAG-nya, batas dokumen dijaga, tidak ada *cross-document merging*, dan metadata diwariskan dari chunk awal untuk menjaga konteks sumber dan sitasi. ([Open WebUI](#))

Struktur yang disarankan

Untuk dokumen SOP atau kebijakan, format sederhana seperti ini sangat membantu:

```
Judul: SOP Approval Pengadaan
Versi: 1.2
Tanggal berlaku: 2026-01-10
Pemilik proses: Divisi Keuangan
```


Tujuan
Menjelaskan langkah approval pengadaan.

Ruang lingkup
Berlaku untuk seluruh unit kerja.

Langkah
1. Unit mengajukan permintaan.
2. Keuangan memverifikasi anggaran.
3. Kepala unit memberi approval akhir.

Dokumen terkait
Form permintaan pengadaan
Checklist verifikasi anggaran

Struktur seperti ini membantu karena:

- agent mudah memahami bagian,
- retrieval lebih mudah menemukan bagian yang tepat,
- dan pembaca manusia juga lebih mudah memeriksa sumber.

Dokumen yang kurang sehat

Dokumen retrieval yang buruk biasanya:

- hasil *copy-paste* dari banyak sumber tak terkait,
- terlalu panjang tanpa heading,
- berisi versi lama dan baru bercampur,
- penuh screenshot tanpa teks,
- atau memakai istilah yang tidak konsisten.

Kalimat yang perlu ditanamkan ke pembaca: **RAG bukan sulap untuk dokumen buruk. Ia bekerja baik bila bahan bakarnya bersih.**

8.4 Chunking, metadata, embedding, reranking

Ini inti teknis RAG. Tetapi ia bisa dijelaskan dengan bahasa yang tetap ringan.

a. Chunking

Chunking adalah proses memotong dokumen menjadi potongan-potongan yang cukup kecil untuk dicari, tetapi cukup utuh untuk tetap bermakna. Dokumentasi Open WebUI menunjukkan bahwa sistem menjaga batas ukuran chunk, tidak menggabungkan chunk dari dokumen berbeda, dan memakai *metadata inheritance* agar chunk tetap membawa asal-usulnya. Ini sangat penting untuk auditability dan sitasi. ([Open WebUI](#))

Kalau chunk terlalu besar:

- retrieval jadi kabur,
- konteks terlalu banyak,
- latensi naik.

Kalau chunk terlalu kecil:

- makna terpotong,
- potongan kehilangan konteks,
- dan model sulit menyusun jawaban.

Di dunia nyata, pertanyaan yang paling sehat bukan “berapa ukuran chunk terbaik secara universal?” tetapi **“berapa ukuran chunk yang paling pas untuk jenis dokumen saya?”**

b. Metadata

Metadata adalah label tambahan pada dokumen atau chunk. Misalnya:

- judul,
- versi,
- pemilik proses,
- kategori dokumen,
- tanggal berlaku,
- unit kerja,
- tingkat sensitivitas.

Metadata sangat membantu untuk:

- membatasi pencarian,
- memisahkan dokumen lama dan baru,
- menambah konteks bagi agent,
- dan mempermudah audit.

Contoh metadata sederhana dalam JSON:

```
{
  "title": "SOP Approval Pengadaan",
  "version": "1.2",
  "owner": "Keuangan",
  "category": "SOP",
  "effective_date": "2026-01-10",
  "sensitivity": "internal"
}
```

c. Embedding

Embedding mengubah teks menjadi vektor angka agar bisa dicari berdasarkan kemiripan makna. Dokumentasi Ollama menjelaskan bahwa *embeddings* adalah vektor numerik yang dapat disimpan di *vector database*, dicari dengan *cosine similarity*, dan dipakai dalam pipeline RAG. Dokumentasi itu juga menyebut model *embedding* yang direkomendasikan seperti **embeddinggemma**, **qwen3-embedding**, dan **all-minilm**. ([Ollama](#))

Contoh membuat *embedding* via CLI:

```
ollama run embeddinggemma "SOP approval pengadaan dan verifikasi anggaran"
```

Contoh keluaran:

```
[0.0123, -0.0831, 0.0411, ...]
```

Dokumentasi Ollama memang menyebut bahwa perintah ini akan menghasilkan *JSON array*. ([Ollama](#))

Contoh lewat API:

```
curl -X POST http://localhost:11434/api/embed \
-H "Content-Type: application/json" \
-d '{
  "model": "embeddinggemma",
  "input": "SOP approval pengadaan dan verifikasi anggaran"
}'
```

Contoh keluaran:

```
{
  "embeddings": [[0.0123, -0.0831, 0.0411]]
}
```

Dokumentasi resmi menyebut endpoint `/api/embed` mengembalikan vektor yang sudah dinormalisasi (*L2-normalized*). ([Ollama](#))

d. Reranking

Retrieval awal sering memakai kombinasi *BM25* dan *vector search*. Tetapi hasil terbaik belum tentu ada di urutan atas. Karena itu digunakan *reranking*, yaitu penyusunan ulang kandidat hasil agar yang paling relevan naik ke depan. Open WebUI menjelaskan bahwa saat *hybrid search* aktif, retrieval menggabungkan *BM25*, *vector search*, dan *reranking* untuk akurasi yang lebih baik. ([Open WebUI](#))

Dalam bahasa sederhana:

- *retrieval* mencari kandidat,
- *reranking* memilih kandidat terbaik.

Ini penting karena di dokumen bisnis, istilah yang mirip bisa muncul di banyak file, tetapi hanya satu atau dua yang benar-benar tepat untuk pertanyaan tertentu.

8.5 Focused Retrieval vs Full Context

Ini salah satu keputusan desain paling praktis dalam Open WebUI, dan sangat penting untuk pembaca buku Anda. Dokumentasi resminya menjelaskan dua mode utama:

- **Focused Retrieval (default)**
Sistem memakai RAG untuk mencari dan menyuntikkan potongan paling relevan berdasarkan pertanyaan pengguna. Saat *hybrid search* aktif, ia menggabungkan BM25, vector search, dan reranking. Mode ini paling cocok untuk koleksi dokumen besar. ([Open WebUI](#))
- **Full Context**
Sistem menyuntikkan seluruh isi file ke setiap pesan. Tidak ada chunking, tidak ada semantic search. Mode ini paling cocok untuk dokumen pendek yang selalu relevan. ([Open WebUI](#))

Kapan pakai Focused Retrieval

Gunakan mode ini bila:

- dokumen banyak,
- koleksi besar,
- pertanyaan bervariasi,
- hanya sebagian dokumen yang biasanya relevan,
- dan efisiensi konteks penting.

Contoh:

- basis SOP perusahaan,
- koleksi kontrak,
- kumpulan notulen proyek,
- kebijakan lintas unit.

Kapan pakai Full Context

Gunakan mode ini bila:

- dokumen pendek,
- seluruh dokumen hampir selalu relevan,
- Anda butuh presisi kata demi kata,
- atau dokumen memang ingin selalu “hadir” di setiap percakapan.

Contoh:

- pedoman gaya penulisan,
- instruksi peran agent,
- satu template memo,
- satu prosedur singkat yang selalu dipakai.

Cara berpikir yang sehat

Jangan memaksa semua dokumen masuk Full Context. Itu boros konteks dan bisa membuat model lambat.

Jangan juga terlalu mengandalkan Focused Retrieval untuk dokumen yang pendek tetapi sangat penting secara utuh.

Aturan praktis yang mudah:

- koleksi besar → **Focused Retrieval**
- dokumen pendek dan selalu relevan → **Full Context**

8.6 Evaluasi RAG: *relevance, faithfulness, latency, source traceability*

Ini bagian yang paling sering diabaikan, padahal justru paling penting untuk bisnis.

RAG yang tidak dievaluasi hanya tampak meyakinkan di demo. Untuk lingkungan nyata, kita perlu tahu:

- apakah retrieval-nya benar,
- apakah jawabannya setia pada sumber,
- apakah sistemnya cukup cepat,
- dan apakah sumbernya benar-benar bisa ditelusuri.

a. Relevance

Relevance berarti apakah chunk atau dokumen yang diambil benar-benar relevan dengan pertanyaan. Kalau retrieval salah, model bisa menjawab dengan sangat lancar tetapi di atas dokumen yang keliru.

Cara uji sederhana:

- siapkan 10–20 pertanyaan bisnis nyata,
- tentukan dokumen mana yang seharusnya muncul,
- lalu lihat apakah hasil retrieval memang membawa dokumen itu.

Contoh checklist:

- Apakah SOP yang muncul memang SOP yang benar?
- Apakah versi terbaru yang muncul, bukan versi usang?
- Apakah chunk yang diambil cukup dekat dengan bagian yang dibutuhkan?

b. Faithfulness

Faithfulness berarti apakah jawaban model benar-benar setia pada sumber yang diambil, bukan menambahkan klaim liar. Dalam praktik, ini sangat penting untuk auditability.

Cara uji sederhana:

- baca jawaban model,
- bandingkan dengan potongan sumber,
- tandai bagian yang benar-benar didukung sumber,
- tandai bagian yang merupakan improvisasi berlebihan.

Prinsipnya sederhana: **jawaban bagus bukan yang paling indah, tetapi yang paling setia pada sumber.**

c. Latency

Latency adalah waktu yang diperlukan dari pertanyaan sampai jawaban keluar. RAG hampir selalu menambah biaya waktu karena ada langkah retrieval, kadang juga reranking. Itulah sebabnya literatur enterprise di referensi tepat ketika mengingatkan bahwa RAG memberi transparansi dan pengetahuan dinamis, tetapi membawa trade-off berupa latensi dan beban infrastruktur tambahan. Untuk Open WebUI sendiri, mode retrieval dan hybrid search memang menambah lapisan kerja dibanding model yang hanya diberi prompt langsung. ([Open WebUI](#))

Cara uji sederhana:

```
time curl -X POST http://localhost:11434/api/chat \
  -d '{
    "model": "gemma3",
    "messages": [
      {"role": "user", "content": "Apa approval
akhir dalam SOP pengadaan?"}
    ]
  }'
```

Contoh keluaran:

```
real    0m2.143s
user    0m0.010s
sys     0m0.004s
```

Ini bukan metrik RAG Open WebUI penuh, tetapi bisa menjadi baseline sederhana di sisi model lokal.

d. Source traceability

Ini metrik yang sangat penting untuk bisnis. Jawaban harus bisa ditelusuri:

- dari dokumen mana,
- bagian mana,
- versi mana,
- dan apakah sumber itu memang valid.

Open WebUI sangat menekankan sitasi sumber dan menjaga batas dokumen agar atribusi tetap jelas. Itu menjadikannya cocok untuk lingkungan yang ingin auditability, bukan sekadar kecanggihan gaya bahasa. ([Open WebUI](#))

Langkah evaluasi praktis

Untuk pembaca buku ini, evaluasi RAG bisa dibuat sangat membumi:

- **Siapkan 20 pertanyaan nyata** dari pekerjaan sehari-hari.
- **Tentukan jawaban referensi** dan dokumen sumbernya.
- **Uji mode Focused Retrieval** dan catat hasil.
- **Uji Full Context** untuk beberapa dokumen pendek.
- **Bandingkan relevance, faithfulness, latency, dan sitasi sumber.**
- **Perbaiki dokumen, chunking, atau struktur knowledge base** bila hasil jelek.

Tanda sistem RAG Anda sehat

- jawaban sering menunjuk sumber yang tepat,
- versi dokumen yang muncul konsisten,
- halusinasi turun,
- retrieval tidak terlalu lambat,
- dan pengguna mulai percaya pada sistem.

Tanda sistem RAG Anda bermasalah

- jawaban terdengar bagus tetapi sumbernya salah,
- dokumen lama sering muncul,
- banyak chunk tidak relevan,
- model sering “mengarang penghubung” di luar sumber,
- atau waktu respons mulai mengganggu alur kerja.

Hands-on lab: membuat knowledge base SOP, kebijakan, dan FAQ internal

Agar bab ini benar-benar terasa hidup, pembaca bisa mencoba latihan sederhana berikut.

Tahap 1. Siapkan tiga dokumen

```
mkdir -p ~/rag-lab/{sop,kebijakan,faq}

cat > ~/rag-lab/sop/sop-helpdesk.txt <<'EOF'
SOP Helpdesk
- Tiket prioritas tinggi harus direspons maksimal 4 jam kerja.
- Tiket prioritas sedang maksimal 1 hari kerja.
- Semua eskalasi harus dicatat pada sistem tiket.
EOF

cat > ~/rag-lab/kebijakan/kebijakan-data.txt <<'EOF'
Kebijakan Data
- Data pelanggan hanya boleh diakses sesuai peran.
- Semua akses ke data sensitif harus tercatat.
EOF

cat > ~/rag-lab/faq/faq-internal.txt <<'EOF'
FAQ Internal
- Approval akhir pengadaan dilakukan oleh kepala unit.
- Verifikasi anggaran dilakukan oleh tim keuangan.
EOF
```

Tahap 2. Buat *embedding model* tersedia

```
ollama pull embeddinggemma
```

Contoh keluaran:

```
pulling manifest
verifying sha256 digest
success
```

Tahap 3. Buka Open WebUI dan unggah dokumen

Masuk ke **Workspace** → **Knowledge**, buat tiga koleksi:

- SOP
- Kebijakan
- FAQ

Unggah dokumen ke koleksi yang benar.

Tahap 4. Tempelkan *knowledge base* ke model preset

Misalnya buat preset:

- nama: **Asisten SOP Internal**
- model: model lokal utama Anda
- knowledge: SOP + Kebijakan + FAQ
- retrieval mode: mulai dari **Focused Retrieval**

Tahap 5. Uji tiga pertanyaan

- “Berapa SLA tiket prioritas tinggi?”
- “Siapa approval akhir pengadaan?”
- “Apa aturan dasar akses data pelanggan?”

Tahap 6. Bandingkan dengan Full Context

Untuk dokumen FAQ yang sangat pendek, ubah ke **Full Context**, lalu bandingkan:

- apakah jawaban lebih konsisten,
- apakah latensinya lebih ringan atau justru lebih berat,
- apakah model jadi lebih patuh pada isi dokumen.

Penutup bab

Bab ini harus meninggalkan satu pesan besar: **RAG bukan aksesoris tambahan, tetapi otak eksternal yang membuat agent bisnis lebih berguna, lebih jujur, dan lebih bisa diaudit.** Open WebUI secara resmi mendukung *Knowledge*, *Focused Retrieval*, *Full Context*, sitasi sumber, *hybrid search*, dan *reranking*; sementara Ollama menyediakan fondasi embeddings dan API lokal yang membuat retrieval semantik bisa dibangun sepenuhnya di lingkungan sendiri. ([Open WebUI](#))

Bagi mahasiswa dan praktisi, pelajaran paling penting dari bab ini sederhana:

- **jangan hanya mengandalkan model,**
- **rapikan dokumen sumber,**
- **bangun knowledge base yang masuk akal,**
- **pilih mode retrieval dengan sadar,**
- **dan ukur kualitas jawaban dari sumbernya, bukan dari kefasihannya.**

Kalau Bab 6 menyalakan mesin model, dan Bab 7 mulai membuat agent bekerja, maka **Bab 8 memberi agent itu ingatan kerja yang bisa dipertanggungjawabkan.**

Bab 9. Dataset kerja, instruction data, dan data governance

9.1 SOP, kebijakan, tiket, email, laporan, notulen, dan katalog

Kalau Bab 8 membahas bagaimana agent mengambil pengetahuan, maka Bab 9 menjelaskan **bahan mentah** yang menjadi dasar seluruh sistem itu. Dalam konteks bisnis, dataset kerja tidak harus selalu berupa kumpulan jutaan contoh seperti di riset AI. Justru yang paling bernilai sering adalah dokumen sehari-hari organisasi:

- **SOP**
- **kebijakan**
- **tiket helpdesk**
- **email operasional**
- **laporan**
- **notulen rapat**
- **katalog layanan atau produk**

Ini penting dipahami pembaca sejak awal: **dataset kerja bisnis sering lahir dari arsip kerja biasa, bukan dari laboratorium khusus**. Disini ditempatkan jenis-jenis dokumen ini sebagai inti bab.

Agar bahan kerja ini bisa dipakai dengan rapi, langkah pertama adalah **inventarisasi dokumen**. Jangan langsung berpikir tentang training. Mulailah dengan bertanya:

- dokumen apa yang paling sering dipakai untuk menjawab pertanyaan kerja?
- dokumen apa yang paling sering berubah?
- dokumen apa yang paling sensitif?
- dokumen apa yang paling sering menimbulkan salah tafsir?

Langkah demi langkah: inventarisasi bahan kerja

Tahap 1. Buat folder sumber berdasarkan jenis dokumen

```
mkdir -p  
~/data-bisnis/{sop,kebijakan,tiket,email,laporan,notulen,k  
atalog}
```

Tahap 2. Isi contoh dokumen

```
cat > ~/data-bisnis/sop/sop-approval.txt <<'EOF'  
Judul: SOP Approval Pengadaan  
Versi: 1.2  
Pemilik proses: Keuangan  
Isi:  
1. Unit mengajukan permintaan.  
2. Keuangan memverifikasi anggaran.  
3. Kepala unit memberi approval akhir.  
EOF  
  
cat > ~/data-bisnis/tiket/tiket-001.txt <<'EOF'
```

```
Kategori: Helpdesk
Prioritas: Tinggi
Isi: Pengguna tidak dapat mengakses sistem pengadaan sejak
pagi.
EOF
```

```
cat > ~/data-bisnis/notulen/rapat-mingguan.txt <<'EOF'
Notulen rapat:
- Approval pengadaan lambat karena verifikasi anggaran
manual.
- Tindak lanjut: tim keuangan menyusun checklist
verifikasi baru.
EOF
```

Tahap 3. Cek isi folder

```
find ~/data-bisnis -type f | sort
```

Contoh keluaran:

```
/home/user/data-bisnis/notulen/rapat-mingguan.txt
/home/user/data-bisnis/sop/sop-approval.txt
/home/user/data-bisnis/tiket/tiket-001.txt
```

Langkah sederhana ini penting karena memaksa pembaca melihat bahwa sebelum bicara AI, mereka harus lebih dulu **melihat arsip kerjanya sebagai aset**. Literatur Hugging Face tentang *dataset cards* juga menekankan bahwa dataset yang baik perlu konteks: isinya apa, dipakai untuk apa, dari mana asalnya, dan apa batasannya. ([Hugging Face](#))

9.2 Membangun instruction dataset dari proses internal

Instruction dataset adalah kumpulan contoh yang mengajari model bagaimana merespons tugas tertentu. Dalam konteks bisnis, kita tidak selalu perlu *fine-tuning* besar. Tetapi kita sering perlu **dataset instruksi kecil dan rapi** untuk:

- menyusun format jawaban,
- menanamkan gaya respons,
- mencontohkan keputusan awal,
- dan menertibkan perilaku agent.

Di titik ini, bedakan dua hal:

- **dokumen pengetahuan** untuk RAG, seperti SOP dan kebijakan,
- **dataset instruksi** untuk melatih atau menilai cara model menjawab tugas tertentu.

Contoh *instruction dataset* yang sehat untuk bisnis bisa berbentuk:

- *input*: isi tiket
output: kategori, prioritas, tindakan awal
- *input*: memo rapat
output: ringkasan, keputusan, PIC, tenggat
- *input*: pertanyaan kebijakan
output: jawaban singkat + rujukan SOP

Literatur PEFT dari Hugging Face menjelaskan bahwa *parameter-efficient fine-tuning* seperti **LoRA** dipakai untuk mengadaptasi model ke aplikasi hilir tertentu tanpa harus melatih ulang seluruh parameter model, sehingga biaya komputasi dan penyimpanan jauh lebih rendah dibanding *full fine-tuning*. Ini membuat ide *instruction dataset* kecil menjadi sangat relevan untuk organisasi dengan sumber daya terbatas. ([Hugging Face](#))

Langkah demi langkah: membuat instruction dataset kecil

Misalnya kita ingin membuat dataset untuk tugas **triase tiket internal**.

Tahap 1. Buat direktori dataset

```
mkdir -p ~/dataset-instruksi/helpdesk
```

Tahap 2. Tulis file JSONL

```
cat > ~/dataset-instruksi/helpdesk/train.jsonl <<'EOF'
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas awal.","input":"Pengguna tidak dapat login ke
sistem pengadaan sejak pagi.","output":"Kategori: akses
sistem\nPrioritas: tinggi\nTindakan awal: cek autentikasi
dan status layanan."}
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas awal.","input":"Permintaan reset password untuk
staf baru.","output":"Kategori: akun pengguna\nPrioritas:
sedang\nTindakan awal: verifikasi identitas dan lakukan
reset password."}
EOF
```

Tahap 3. Cek isi

```
cat ~/dataset-instruksi/helpdesk/train.jsonl
```

Contoh keluaran:

```
{"instruction": "Klasifikasikan tiket dan tentukan prioritas awal.", "input": "Pengguna tidak dapat login ke sistem pengadaan sejak pagi.", "output": "Kategori: akses sistem\nPrioritas: tinggi\nTindakan awal: cek autentikasi dan status layanan."}
{"instruction": "Klasifikasikan tiket dan tentukan prioritas awal.", "input": "Permintaan reset password untuk staf baru.", "output": "Kategori: akun pengguna\nPrioritas: sedang\nTindakan awal: verifikasi identitas dan lakukan reset password."}
```

Format seperti ini sangat mudah dipahami mahasiswa maupun praktisi. Ia juga mudah ditumbuhkan sedikit demi sedikit dari pekerjaan nyata. **Kuncinya bukan banyaknya data lebih dulu, tetapi konsistensi pola tugas dan kualitas label.**

9.3 Dataset evaluasi vs dataset training

Ini salah satu disiplin paling penting, dan paling sering dilanggar. **Dataset training dan dataset evaluasi tidak boleh dicampur.** Kalau dicampur, model akan tampak hebat hanya karena “mengingat” contoh yang sudah pernah dilihat. Itu bukan kemajuan. Itu ilusi.

Dokumentasi Hugging Face menjelaskan bahwa dataset lazim dibagi ke dalam *splits* seperti **train**, **validation**, dan **test/eval**, dan bahkan penamaan file seperti **train**, **valid**, **test**, atau **eval** dikenali sebagai pemisahan split yang berbeda. ([Hugging Face](#))

Dalam konteks buku ini:

- **training set** dipakai untuk membentuk perilaku model atau adapter,
- **validation set** dipakai untuk penyetelan selama eksperimen,
- **evaluation/test set** dipakai untuk mengukur performa akhir secara jujur.

Langkah demi langkah: memisahkan split

Tahap 1. Buat struktur file

```
mkdir -p ~/dataset-instruksi/helpdesk/{train,eval}
```

Tahap 2. Simpan contoh terpisah

```
cat > ~/dataset-instruksi/helpdesk/train/data.jsonl <<'EOF'
{"instruction":"Ringkas tiket.","input":"Aplikasi lambat
saat jam sibuk.","output":"Masalah performa sistem pada
jam sibuk."}
EOF

cat > ~/dataset-instruksi/helpdesk/eval/data.jsonl <<'EOF'
{"instruction":"Ringkas tiket.","input":"Approval
pengadaan tidak bisa disimpan di akhir
proses.","output":"Kegagalan penyimpanan pada tahap akhir
approval pengadaan."}
EOF
```

Tahap 3. Verifikasi file

```
find ~/dataset-instruksi/helpdesk -type f | sort
```

Contoh keluaran:

```
/home/user/dataset-instruksi/helpdesk/eval/data.jsonl
/home/user/dataset-instruksi/helpdesk/train/data.jsonl
```

Prinsip sederhananya:

- **jangan uji model pada contoh yang pernah dipakai melatihnya**
- **buat evaluasi dari kasus nyata yang cukup berbeda**
- **jangan hanya menilai dari “jawaban terasa bagus”**

Disini disinggung pentingnya *ongoing performance monitoring* dan *drift detection* setelah deployment. Artinya, evaluasi bukan kegiatan sekali selesai, tetapi kebiasaan yang harus dijaga.

9.4 Data sensitif, masking, minimisasi, dan hak akses

Ini bagian yang tidak boleh diperlakukan sebagai lampiran kecil. **Data governance bukan dekorasi; ia adalah syarat agar agentic AI bisa dipercaya.**

Sumber-sumber yang Disini ditekankan *privacy-preserving data handling*, *access control*, compliance, dan keamanan sebagai fondasi sistem agent.

Open WebUI sendiri sudah punya lapisan **RBAC**, **roles**, **permissions**, dan **group-based access**, serta kemampuan membatasi akses ke *knowledge base* atau model tertentu lewat *groups* atau *individual grants*. Ini sangat berguna ketika kita

memiliki basis pengetahuan yang sensitif, misalnya dokumen HR, kontrak, atau kebijakan internal tertentu. ([Open WebUI](#))

Empat prinsip praktis yang harus ditanamkan

- **Minimisasi data**
Jangan masukkan semua data hanya karena “mungkin berguna”. Masukkan yang benar-benar diperlukan untuk tugas.
- **Masking**
Data pribadi, nomor identitas, alamat, atau data pelanggan perlu disamarkan bila tidak perlu tampil utuh.
- **Hak akses**
Tidak semua user boleh melihat semua *knowledge base* atau dataset.
- **Jejak perubahan**
Harus jelas siapa menambah, mengubah, atau menghapus data penting.

Langkah demi langkah: masking sederhana

Misalnya kita punya file email yang mengandung nomor telepon dan email pribadi.

Tahap 1. Buat contoh file

```
mkdir -p ~/data-sensitif
cat > ~/data-sensitif/email-001.txt <<'EOF'
Nama: Budi Santoso
Email: budi@example.com
Telepon: 081234567890
Isi: Mohon reset akses sistem pengadaan.
EOF
```

Tahap 2. Buat versi *masked*

```
sed -E
's/[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+/[EMAIL-DISAMARKAN]/g;
s/08[0-9]{8,12}/[NO-HP-DISAMARKAN]/g' \
~/data-sensitif/email-001.txt >
~/data-sensitif/email-001-masked.txt
```

Tahap 3. Cek hasil

```
cat ~/data-sensitif/email-001-masked.txt
```

Contoh keluaran:

```
Nama: Budi Santoso  
Email: [EMAIL-DISAMARKAN]  
Telepon: [NO-HP-DISAMARKAN]  
Isi: Mohon reset akses sistem pengadaan.
```

Tentu di lingkungan nyata kita akan membutuhkan pola masking yang lebih rapi, tetapi contoh ini cukup untuk menanamkan kebiasaan. **Jangan jadikan RAG atau dataset sebagai tempat bocornya data sensitif.**

Untuk Open WebUI, pembaca juga perlu memahami bahwa file yang diunggah dapat dikelola melalui **File Management** dan **Data Controls**, termasuk penghapusan file beserta *knowledge base entries* dan *vector embeddings* terkait. Ini penting untuk *data hygiene* dan hak untuk menghapus data yang tidak lagi boleh dipakai. ([Open WebUI](#))

9.5 Kapan RAG cukup, kapan fine-tuning perlu, kapan hybrid masuk akal

Ini mungkin bagian paling strategis dalam bab ini. Banyak orang bertanya:

“Kalau sudah ada RAG, untuk apa *fine-tuning*?”

Jawabannya: karena keduanya menyelesaikan masalah yang berbeda.

Literatur enterprise di referensi menjelaskannya dengan sangat baik:

- **RAG** berfungsi seperti mekanisme persepsi. Ia mengambil informasi eksternal secara dinamis, membuat model sadar situasi, mengikuti perubahan, dan memberi jejak sumber. Tetapi RAG menambah latensi dan bergantung pada kualitas basis pengetahuan serta pipeline retrieval.
- **Fine-tuning** menanamkan spesialisasi yang lebih dalam ke bobot model. Ia cocok untuk bahasa, prosedur, dan pola keputusan yang relatif stabil. Kelebihannya adalah konsistensi lebih tinggi, latensi lebih rendah, dan bisa cocok untuk lingkungan sensitif yang tidak ingin bergantung pada retrieval dinamis. Kekurangannya: pengetahuan menjadi statis, update butuh retraining, dan risiko bias lebih tinggi bila data jelek.
- **Hybrid** menjadi masuk akal ketika organisasi butuh keduanya: model yang sudah “fasih” pada domain dan tetap bisa mengambil pengetahuan terbaru secara dinamis. Literatur itu bahkan menyebut hybrid sebagai pilihan strategis yang kuat ketika tuntutan akurasi, auditability, dan adaptabilitas harus dipenuhi sekaligus.

Dari sisi alat, Ollama juga sudah mendukung **import adapter fine-tuned** dengan instruksi **ADAPTER** di **Modelfile**. Artinya, jalur *fine-tuning* atau adapter lokal memang realistis di ekosistem buku ini, meski tetap bukan langkah pertama untuk kebanyakan organisasi. ([Ollama](#))

Aturan praktis yang sangat sederhana

RAG cukup bila:

- pengetahuan sering berubah,
- dokumen harus bisa disitasi,
- organisasi butuh auditability,
- domain cukup luas,
- dan kita ingin cepat mulai tanpa training berat.

Fine-tuning perlu bila:

- pola tugas sangat stabil,
- gaya respons harus sangat konsisten,
- domain bahasanya sangat khusus,
- latensi harus rendah,
- atau lingkungan ingin *self-contained intelligence* tanpa retrieval dinamis.

Hybrid masuk akal bila:

- Kita butuh bahasa atau perilaku domain yang kuat,
- tetapi juga butuh dokumen terbaru saat inferensi,
- atau kita ingin kombinasi spesialisasi + grounding.

Cara berpikir yang sehat

- **Jangan langsung fine-tune kalau RAG belum rapi.**
- **Jangan pakai hybrid hanya karena terdengar lebih canggih.**
- **Mulai dari pertanyaan bisnis, bukan dari ego arsitektur.**

9.6 Risiko data buruk: bias, drift, ilusi kualitas

Bab ini harus ditutup dengan peringatan yang jujur. **Data buruk akan menghasilkan agent yang tampak berguna tetapi berbahaya.** Literatur enterprise di referensi bahkan menyebut secara eksplisit bahwa tanpa kurasi dataset dan version control yang ketat, fine-tuning dapat mewariskan kebijakan usang, narasi bias, atau pengetahuan yang tidak lagi akurat. Ia juga memperingatkan bahwa RAG bukan jalan pintas untuk data hygiene yang buruk: kalau repositori pengetahuan kacau, retrieval juga akan kacau.

Ada tiga risiko utama yang perlu dipahami pembaca.

a. Bias

Bias muncul ketika data kerja terlalu mewakili satu pola, satu kelompok, atau satu kebiasaan pengambilan keputusan yang tidak sehat. Misalnya:

- semua tiket prioritas tinggi historis ternyata berasal dari tim tertentu,
- semua contoh email keluhan hanya ditulis oleh satu jenis pelanggan,
- atau label “mendesak” diberikan secara subyektif tanpa aturan.

Agent yang dibangun dari data seperti ini akan tampak “konsisten”, padahal sebenarnya mewarisi ketidakadilan lama.

b. Drift

Drift terjadi ketika data, proses, atau kebijakan berubah, tetapi model atau basis pengetahuan tidak ikut menyesuaikan. Misalnya:

- SOP diperbarui, tetapi versi lama masih dominan,
- kategori tiket berubah, tetapi dataset belum ikut,
- atau struktur approval sudah diganti, tetapi contoh lama masih dipakai.

Literatur enterprise menekankan *ongoing performance monitoring* dan *drift detection* justru karena organisasi nyata tidak pernah diam.

c. Ilusi kualitas

Ini yang paling berbahaya. Jawaban model terlihat bagus, rapi, dan percaya diri, sehingga tim merasa sistem sudah hebat. Padahal:

- sumbernya salah,
- split evaluasi bocor,
- data latih terlalu mirip dengan data uji,
- atau retrieval sebenarnya sering meleset.

Ilusi kualitas biasanya tumbuh ketika tim terlalu kagum pada kefasihan model dan lupa memeriksa landasan datanya.

Langkah sederhana untuk mengurangi risiko

- **versikan dataset**
- **pisahkan train dan eval**
- **bersihkan dokumen usang**
- **cek label yang inkonsisten**
- **uji pada kasus nyata yang belum pernah dilihat**
- **catat perubahan proses dan kebijakan**
- **lakukan review manusia berkala**

Hands-on lab: membuat dataset kerja kecil yang rapi

Agar bab ini terasa sangat praktis, pembaca bisa mencoba latihan sederhana berikut.

Tahap 1. Buat direktori proyek data

```
mkdir -p ~/lab-data-agent/{raw,masked,train,eval}
```

Tahap 2. Siapkan contoh dokumen mentah

```
cat > ~/lab-data-agent/raw/tiket-001.txt <<'EOF'
Nama: Siti
Email: siti@example.com
Isi: Approval pengadaan tidak bisa disimpan pada tahap akhir.
EOF
```

Tahap 3. Masking data sensitif

```
sed -E 's/[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+/[EMAIL-DISAMARKAN]/g' \
~/lab-data-agent/raw/tiket-001.txt >
~/lab-data-agent/masked/tiket-001.txt
```

Tahap 4. Buat train split

```
cat > ~/lab-data-agent/train/data.jsonl <<'EOF'
{"instruction":"Klasifikasikan tiket dan beri prioritas awal.", "input":"Approval pengadaan tidak bisa disimpan pada tahap akhir.", "output":"Kategori: approval workflow\nPrioritas: tinggi\nTindakan awal: cek log aplikasi dan status validasi akhir."}
EOF
```

Tahap 5. Buat eval split

```
cat > ~/lab-data-agent/eval/data.jsonl <<'EOF'
{"instruction":"Klasifikasikan tiket dan beri prioritas awal.", "input":"Permintaan revisi memo keuangan belum diproses selama dua hari.", "output":"Kategori: dokumen internal\nPrioritas: sedang\nTindakan awal: cek antrian approval dan PIC terkait."}
EOF
```

Tahap 6. Verifikasi struktur

```
find ~/lab-data-agent -type f | sort
```

Contoh keluaran:

```
/home/user/lab-data-agent/eval/data.jsonl  
/home/user/lab-data-agent/masked/tiket-001.txt  
/home/user/lab-data-agent/raw/tiket-001.txt  
/home/user/lab-data-agent/train/data.jsonl
```

Latihan kecil ini sudah menanamkan tiga kebiasaan yang sangat penting:

- **data mentah tidak langsung dipakai**
- **data sensitif disamarkan**
- **train dan eval dipisah sejak awal**

Penutup bab

Bab ini harus meninggalkan satu kesan kuat: **dataset bukan sekadar file yang dikumpulkan, tetapi kontrak kualitas antara organisasi dan agent yang akan dipakainya**. SOP, kebijakan, tiket, email, laporan, notulen, dan katalog adalah bahan baku yang sangat kaya, tetapi hanya akan menjadi aset bila dipilih, dibersihkan, dipisah, dan diatur hak aksesnya dengan disiplin. Disini ditempatkan dataset kerja, *instruction data*, dan *data governance* sebagai bagian tersendiri, bukan sekadar catatan kaki teknis.

Untuk pembaca mahasiswa dan praktisi, pelajaran terpenting dari bab ini sederhana:

- **mulailah dari dokumen kerja nyata**
- **ubah sebagian menjadi dataset instruksi kecil**
- **pisahkan data evaluasi dari data training**
- **jaga data sensitif dengan serius**
- **pilih RAG, fine-tuning, atau hybrid berdasarkan kebutuhan, bukan gengsi**
- **dan jangan pernah percaya pada jawaban yang rapi kalau fondasi datanya berantakan**

Kalau Bab 8 memberi agent “otak eksternal”, maka **Bab 9** mengajari kita memilih dan merawat bahan bakar otak itu agar tidak meracuni sistem yang kita bangun.

Bab 10. Synthetic data bila data nyata belum cukup

10.1 Mengapa bisnis kadang perlu data sintetis

Di dunia ideal, organisasi punya arsip kerja yang bersih, lengkap, konsisten, dan siap dipakai. Di dunia nyata, yang sering terjadi justru sebaliknya. Tiket helpdesk masih acak. Email masih campur antara hal penting dan hal remeh. Notulen rapat tidak seragam. SOP ada, tetapi belum cukup untuk membentuk banyak contoh tanya-jawab atau keputusan. **Di sinilah *synthetic data* berguna: ia membantu mengisi celah, memperluas variasi, dan mempercepat pembentukan dataset awal.**

Ada beberapa alasan bisnis yang membuat *synthetic data* layak dipertimbangkan.

- **Data nyata terlalu sedikit** untuk membentuk pola yang stabil.
- **Data nyata terlalu sensitif** untuk langsung dipakai mentah.
- **Distribusi kasus tidak seimbang**, misalnya hanya sedikit contoh kasus prioritas tinggi.
- **Organisasi ingin menguji workflow lebih dulu** sebelum dataset nyata cukup matang.
- **Tim perlu variasi skenario** untuk mengajari agent menangani pengecualian.

Self-Instruct memberi dasar pemikiran yang kuat untuk ini. Pendekatan tersebut menunjukkan bahwa model dapat dipakai untuk membangkitkan instruksi dan contoh baru, lalu menyaring hasilnya agar menjadi dataset pelatihan yang lebih kaya. Artinya, ketika organisasi belum punya cukup contoh kerja, model bisa dipakai sebagai “mesin pembuat contoh” — tentu dengan syarat hasilnya tetap diperiksa dan dibersihkan. ([aqlanthology.org](https://arxiv.org/abs/2310.11478))

Kalimat penting yang perlu ditanamkan ke pembaca adalah ini: **data sintetis bukan alat untuk berbohong pada model, tetapi alat untuk melatih variasi dan menutup kekurangan awal data nyata.**

10.2 Teknik: self-instruct, pair generation, scenario expansion

Tiga teknik ini sangat cocok karena praktis, mudah dipahami, dan bisa dijalankan di stack lokal.

a. Self-instruct

Self-Instruct adalah teknik ketika model diminta membuat sendiri instruksi baru, lalu membangkitkan pasangan input-output yang sesuai. Makalah *Self-Instruct* menjelaskan bahwa pipeline-nya membangkitkan instruksi, input, dan output dari model, kemudian menyaring contoh yang tidak valid atau terlalu mirip sebelum dipakai untuk *instruction tuning*. ([aqlanthology.org](https://arxiv.org/abs/2310.11478))

Dalam konteks bisnis, *self-instruct* cocok untuk:

- membuat variasi tugas triase tiket,
- membuat variasi pertanyaan SOP,
- membuat contoh ringkasan memo,
- membuat instruksi pengelompokan notulen.

b. Pair generation

Pair generation lebih sederhana. Kita mulai dari satu contoh input nyata, lalu minta model membuat pasangan output yang konsisten dengan aturan kerja. Atau sebaliknya: mulai dari format output yang diinginkan, lalu model diminta membuat beberapa input yang masuk akal.

Contoh:

- input: tiket helpdesk
- output: kategori, prioritas, tindakan awal

Lalu model diminta membuat 20–50 pasangan tambahan dengan gaya dan variasi kasus yang berbeda.

c. Scenario expansion

Ini teknik yang sangat berguna untuk bisnis. Kita mulai dari satu kasus nyata, lalu memperluasnya menjadi banyak skenario turunan:

- versi ringan,
- versi mendesak,
- versi ambigu,
- versi dengan lampiran tidak lengkap,
- versi dengan approval yang macet,
- versi dengan konflik antar unit.

Framework seperti **Distilabel** memang dibangun untuk skenario seperti ini: sintesis data, evaluasi AI, dan pipeline yang bisa diskalakan berdasarkan pola riset yang sudah diverifikasi. Itu sangat cocok untuk pembaca yang ingin mulai dari eksperimen sederhana lalu naik ke proses yang lebih sistematis.

([Hugging Face](#))

Kalau disederhanakan:

- **self-instruct** membuat tugas baru,
- **pair generation** membuat pasangan contoh,
- **scenario expansion** memperkaya variasi situasi.

10.3 Membuat variasi kasus untuk helpdesk, triase, approval, dan laporan

Di titik ini pembaca perlu melihat bahwa *synthetic data* bukan teori. Ia bisa langsung dipakai untuk menghasilkan contoh kerja yang dekat dengan operasi sehari-hari.

a. Helpdesk

Kasus dasar:

- “Pengguna tidak bisa login.”

Variasi sintetis:

- pengguna tidak bisa login sejak pagi,
- login gagal hanya di browser tertentu,
- login gagal setelah reset password,
- login gagal untuk pengguna baru,
- login gagal saat jam sibuk.

Dari satu pola dasar, model bisa membantu membuat banyak variasi yang tetap relevan untuk klasifikasi dan prioritasasi.

b. Triase tiket

Kasus dasar:

- “Approval pengadaan gagal disimpan.”

Variasi sintetis:

- gagal di tahap akhir,
- gagal hanya untuk nilai di atas batas tertentu,
- gagal bila lampiran kurang,
- gagal di unit tertentu,
- gagal setelah perubahan SOP.

Ini membantu agent belajar bahwa “masalah yang mirip” bisa datang dalam bentuk kalimat yang berbeda.

c. Approval

Kasus dasar:

- “Dokumen menunggu persetujuan kepala unit.”

Variasi sintetis:

- approval macet karena verifikasi anggaran belum ada,
- approval tertahan karena PIC sedang cuti,
- approval ditolak karena dokumen tidak lengkap,
- approval tertunda karena versi file berbeda.

d. Laporan

Kasus dasar:

- “Ringkas laporan mingguan.”

Variasi sintetis:

- ringkas laporan yang berisi 3 isu utama,
- buat *executive summary*,
- sorot risiko dan tindak lanjut,
- ubah laporan naratif menjadi poin tindakan.

Tujuan variasi bukan membuat data tampak banyak, tetapi membuat agent menghadapi bentuk kasus yang lebih beragam. Itu penting karena model yang hanya dilatih pada kalimat seragam akan tampak hebat di demo, tetapi mudah goyah ketika bertemu dunia nyata.

10.4 Memisahkan dataset training dari dataset evaluasi

Ini harus diulang lagi, karena dalam *synthetic data* godaannya lebih besar. Orang sering menghasilkan satu kumpulan contoh sintesis, lalu memakai semuanya sekaligus untuk training dan evaluasi. Hasilnya? Model tampak sangat bagus — padahal ia hanya diuji pada gaya data yang dibangkitkan dari pola yang sama. **Itu bukan evaluasi jujur.**

Dokumentasi Hugging Face tentang *dataset splits* menjelaskan secara eksplisit pentingnya pemisahan *train*, *validation*, dan *test/eval* untuk menghindari kebocoran penilaian. ([Hugging Face](#))

Prinsipnya sederhana:

- **training set** dipakai untuk membentuk perilaku model,
- **eval/test set** dipakai untuk menilai performa secara jujur,
- dan **data sintetis evaluasi sebaiknya dibangkitkan dengan pola yang berbeda** dari data sintetis training.

Langkah praktis

Tahap 1. Buat direktori split

```
mkdir -p ~/synthetic-data/helpdesk/{train,eval}
```

Tahap 2. Isi data training

```
cat > ~/synthetic-data/helpdesk/train/data.jsonl <<'EOF'
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas.","input":"Pengguna tidak bisa login ke sistem
pengadaan sejak pagi.","output":"Kategori: akses
sistem\nPrioritas: tinggi\nTindakan awal: cek autentikasi
dan status layanan."}
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas.","input":"Permintaan reset password untuk staf
baru.","output":"Kategori: akun pengguna\nPrioritas:
```



```
sedang\nTindakan awal: verifikasi identitas dan reset  
password."}  
EOF
```

Tahap 3. Isi data evaluasi dengan pola lain

```
cat > ~/synthetic-data/helpdesk/eval/data.jsonl <<'EOF'  
{ "instruction": "Klasifikasikan tiket dan tentukan  
prioritas.", "input": "Approval pengadaan gagal disimpan  
pada tahap akhir.", "output": "Kategori: approval  
workflow\nPrioritas: tinggi\nTindakan awal: cek log  
aplikasi dan validasi akhir." }  
EOF
```

Tahap 4. Verifikasi

```
find ~/synthetic-data/helpdesk -type f | sort
```

Contoh keluaran:

```
/home/user/synthetic-data/helpdesk/eval/data.jsonl  
/home/user/synthetic-data/helpdesk/train/data.jsonl
```

Kalimat yang perlu masuk ke kepala pembaca: **data sintetis pun harus dievaluasi dengan disiplin seperti data nyata.**

10.5 Risiko synthetic data: bias, drift, overconfidence, rasa palsu

Di sinilah bab ini harus bersikap tegas. *Synthetic data* memang membantu, tetapi ia juga membawa jebakan yang sangat khas.

a. Bias

Kalau model pembangkit punya kecenderungan tertentu, bias itu bisa diwariskan dan bahkan diperbesar. Misalnya semua tiket sintetis “prioritas tinggi” dibuat dengan bahasa tertentu, atau semua contoh approval bermasalah selalu mengarah ke unit yang sama. **Data sintetis bisa meniru pola lama, termasuk pola buruk.**

b. Drift

Bila proses bisnis berubah tetapi generator sintetis tetap memakai pola lama, dataset sintetis akan cepat usang. Ini berbahaya karena tampilannya tetap rapi, tetapi isinya tidak lagi sesuai realitas kerja.

c. Overconfidence

Ini sangat umum. Karena data sintetis sering bersih, konsisten, dan “teratur”, model terlihat sangat percaya diri dan performanya tampak bagus. Padahal dunia nyata jauh lebih berantakan. Model lalu menjadi **terlalu optimistis pada input yang rapi** dan tidak siap menghadapi kasus ambigu.

d. Rasa palsu

Ini istilah yang penting untuk buku ini. Dataset sintetis sering memberi **rasa palsu bahwa kita sudah punya data banyak dan berkualitas**, padahal yang kita punya hanyalah variasi buatan dari pola yang sama. Hasilnya bisa sangat licin di atas kertas, tetapi miskin kejutan nyata.

Karena itu, setiap bab tentang *synthetic data* harus menanamkan tiga pagar:

- **selalu campurkan dengan data nyata bila tersedia**
- **selalu lakukan evaluasi terpisah**
- **selalu baca hasil sintetis seperti editor yang curiga, bukan seperti penggemar AI yang kagum**

10.6 Hands-on: membuat dataset kecil untuk satu use case bisnis nyata

Sekarang kita buat latihan yang benar-benar bisa diikuti. Ambil *use case* sederhana: **triase tiket helpdesk internal**.

Tahap 1. Siapkan satu benih kasus nyata

```
mkdir -p ~/synthetic-lab/helpdesk
cat > ~/synthetic-lab/helpdesk/seed.txt <<'EOF'
Kasus dasar:
Pengguna tidak bisa login ke sistem pengadaan sejak pagi.
Tugas:
Klasifikasikan tiket, tentukan prioritas, dan beri
tindakan awal.
EOF
```

Tahap 2. Gunakan Ollama untuk membuat variasi skenario

Dokumentasi Ollama menjelaskan bahwa endpoint `/api/generate` dapat dipanggil dengan `curl`, dan responsnya bisa diminta dalam format JSON atau *streaming*. ([Ollama](#))

Contoh prompt sintetis:

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3",
  "prompt": "Buat 5 variasi kasus helpdesk internal dari
kasus berikut. Setiap variasi harus realistis, singkat,
dan berbeda tingkat urgensinya. Kasus dasar: Pengguna
tidak bisa login ke sistem pengadaan sejak pagi.",
  "stream": false
}'
```

Contoh keluaran:

```
{
  "response": "1. Pengguna baru gagal login setelah reset
password...\n2. Login gagal hanya pada jam sibuk...\n3.
Kepala unit tidak bisa login menjelang approval...\n4.
Login gagal setelah migrasi browser...\n5. Staf keuangan
tidak bisa login untuk verifikasi anggaran..."
}
```

Tahap 3. Ubah variasi menjadi pasangan instruksi

Sekarang minta model mengubah variasi tadi menjadi format JSONL.

```
curl http://localhost:11434/api/generate -d '{
  "model": "gemma3",
  "prompt": "Ubah 3 variasi kasus helpdesk berikut menjadi
data JSONL dengan field instruction, input, output. Output
berisi kategori, prioritas, dan tindakan awal. Variasi: 1)
Pengguna baru gagal login setelah reset password. 2)
Kepala unit tidak bisa login menjelang approval. 3) Login
gagal hanya pada jam sibuk.",
  "stream": false
}'
```

Contoh keluaran:

```
{
  "response": "{\"instruction\":\"Klasifikasikan
tiket...\", \"input\":\"Pengguna baru gagal login setelah
reset password.\", \"output\":\"Kategori: akun
pengguna\\nPrioritas: sedang\\nTindakan awal: verifikasi
akun dan reset ulang.\"}\\n..."
}
```

Tahap 4. Simpan ke file training

```
cat > ~/synthetic-lab/helpdesk/train.jsonl <<'EOF'
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas.","input":"Pengguna baru gagal login setelah
reset password.","output":"Kategori: akun
pengguna\nPrioritas: sedang\nTindakan awal: verifikasi
akun dan reset ulang."}
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas.","input":"Kepala unit tidak bisa login
menjelang approval.","output":"Kategori: akses
sistem\nPrioritas: tinggi\nTindakan awal: cek autentikasi
dan eskalasi cepat."}
EOF
```

Tahap 5. Buat data evaluasi terpisah

```
cat > ~/synthetic-lab/helpdesk/eval.jsonl <<'EOF'
{"instruction":"Klasifikasikan tiket dan tentukan
prioritas.","input":"Staf keuangan tidak bisa login saat
verifikasi anggaran bulanan.","output":"Kategori: akses
sistem\nPrioritas: tinggi\nTindakan awal: cek autentikasi,
status layanan, dan dampak proses anggaran."}
EOF
```

Tahap 6. Periksa hasil

```
wc -l ~/synthetic-lab/helpdesk/train.jsonl
~/synthetic-lab/helpdesk/eval.jsonl
```

Contoh keluaran:

```
2 /home/user/synthetic-lab/helpdesk/train.jsonl
1 /home/user/synthetic-lab/helpdesk/eval.jsonl
3 total
```

Tahap 7. Lakukan pemeriksaan manual

Sebelum dataset dipakai, baca lagi:

- apakah kategorinya masuk akal?
- apakah prioritasnya konsisten?
- apakah tindakan awal sesuai SOP?
- apakah variasinya cukup beragam?
- apakah ada contoh yang terlalu mirip satu sama lain?

Di sinilah kualitas manusia tetap menentukan. **Model bisa membangkitkan contoh. Manusia tetap harus menjadi editor.**

Pola kerja yang disarankan untuk pembaca

Untuk mahasiswa dan praktisi, pola kerja paling sehat adalah:

- mulai dari **1–3 kasus nyata**
- ubah jadi **5–20 variasi sintetis**
- saring contoh yang terlalu mirip atau ngawur
- pisahkan train dan eval
- uji di tugas nyata kecil
- lalu campurkan sedikit demi sedikit dengan data nyata yang mulai terkumpul

Pola ini jauh lebih sehat daripada langsung membuat 5.000 contoh sintetis yang belum tentu bagus.

Penutup bab

Bab ini harus meninggalkan satu pelajaran inti: **data sintetis adalah alat bantu akselerasi, bukan lisensi untuk malas mengurus data nyata**. Ia sangat berguna saat data riil belum cukup, terlalu sensitif, atau belum cukup beragam. Teknik seperti *self-instruct*, *pair generation*, dan *scenario expansion* memang masuk akal dan didukung literatur, tetapi nilainya baru muncul bila hasilnya disaring, dievaluasi terpisah, dan dibandingkan dengan realitas kerja. (aclanthology.org)

Bagi pembaca buku ini, rumus sederhananya bisa dibuat begini:

- **pakai data nyata sebagai jangkar**
- **pakai data sintetis sebagai pengganda**
- **pakai evaluasi sebagai rem**
- **pakai akal sehat sebagai editor akhir**

Kalau Bab 9 mengajari kita mengelola bahan baku yang sudah ada, maka **Bab 10 mengajari kita membuat bahan tambahan secara cerdas—tanpa tertipu oleh kilap sintetisnya sendiri**.

Bagian V. Agentic AI untuk Process Management dan Process Mining

Bab 11. Dari process management ke agentic workflow

11.1 Mengapa proses bisnis sering lambat bukan karena orang bodoh, tapi karena alur kerja buruk

Banyak organisasi terlalu cepat menyalahkan orang: katanya staf lambat, admin kurang sigap, supervisor terlalu lama berpikir, atau approval terlalu birokratis. Sering kali masalahnya bukan pada kecerdasan orang, melainkan pada **alur kerja yang buruk**. Dokumen harus dibaca berulang oleh orang berbeda. Tiket tidak punya jalur prioritas yang jelas. Email penting tenggelam di antrian. Notulen rapat tidak berubah menjadi tindak lanjut. Persetujuan berputar tanpa titik keputusan yang tegas. Dalam kondisi seperti itu, menambah AI tanpa membenahi proses hanya akan mempercepat kebingungan.

Literatur referensi juga memberi pesan serupa: agentic system yang baik tidak lahir dari model semata, tetapi dari desain proses, peran, koordinasi, dan pengawasan yang rapi. Bahkan ketika agent dibuat semakin otonom, nilai bisnisnya tetap bergantung pada **bagaimana ia masuk ke workflow** dan bagaimana manusia tetap bisa memeriksa, mengarahkan, atau menghentikan keputusan saat perlu.

Dalam praktik, gejala alur kerja buruk biasanya mudah dikenali:

- tiket yang sama dibaca banyak orang, tetapi tidak ada yang merasa memilikinya,
- approval bolak-balik tanpa kriteria yang eksplisit,
- laporan selesai, tetapi tidak jelas tindak lanjutnya,
- email penting baru disadari setelah telat,
- pekerjaan kecil menunggu lama karena harus melewati jalur yang tidak proporsional.

Kalimat kuncinya sederhana: **proses lambat sering bukan karena orang lemah, tetapi karena sistem kerja menuntut terlalu banyak langkah yang tidak memberi nilai**. Agentic AI menjadi menarik justru karena ia bisa membantu membaca, memilah, merangkum, merutekan, dan menyiapkan langkah awal di titik-titik yang selama ini membuat proses tersendat. ([n8n Docs](#))

11.2 Event log, bottleneck, rework, exception, dan approval loop

Sebelum memasang agent, pembaca harus belajar satu kebiasaan baru: **melihat proses sebagai jejak kejadian**, bukan sebagai cerita abstrak. Dalam *process management*, yang kita cari bukan hanya “siapa salah”, tetapi **apa yang benar-benar terjadi, kapan, dalam urutan apa, dan di titik mana proses macet**.

Karena itu istilah seperti *event log*, *bottleneck*, *rework*, *exception*, dan *approval loop* menjadi sangat penting.

Event log adalah catatan kejadian proses. Ia bisa berupa tiket masuk, email diterima, file dibuat, dokumen direvisi, approval dikirim, approval disetujui, atau kasus dieskalasi. Event log tidak harus selalu berasal dari sistem process mining yang rumit. Untuk organisasi kecil, file CSV sederhana pun sudah bisa menjadi awal yang sangat kuat. Yang penting, ia menyimpan **identitas kasus, aktivitas, waktu, dan pelaku**. Ini sejalan dengan cara berpikir workflow di n8n, di mana setiap trigger, langkah tool, dan respons bisa dilihat sebagai rangkaian event dalam satu alur kerja. ([n8n Docs](#))

Bottleneck adalah titik macet. Ia muncul ketika kasus menumpuk di satu tahap lebih lama daripada seharusnya. **Rework** adalah kerja ulang: dokumen direvisi berkali-kali, tiket dipindah-pindah kategori, ringkasan diperbaiki berulang. **Exception** adalah penyimpangan dari jalur umum: file kurang lengkap, approver sedang cuti, lampiran hilang, SLA terlewat. **Approval loop** adalah lingkaran persetujuan yang berputar tanpa akhir yang jelas. Semua istilah ini penting karena di sinilah agentic AI biasanya paling cepat memberi nilai: **bukan menggantikan seluruh proses, tetapi mengurangi hambatan di titik-titik yang berulang dan penuh konteks.**

Agar pembaca langsung merasakan bentuknya, berikut contoh *event log* sederhana untuk proses approval internal.

```
mkdir -p ~/process-lab
cat > ~/process-lab/event-log.csv <<'EOF'
case_id,activity,timestamp,actor
REQ-001,Submit Request,2026-04-10T08:00:00,Unit A
REQ-001,Verify Budget,2026-04-10T10:30:00,Finance
REQ-001,Request Revision,2026-04-10T11:00:00,Finance
REQ-001,Resubmit Request,2026-04-11T09:15:00,Unit A
REQ-001,Verify Budget,2026-04-11T10:00:00,Finance
REQ-001,Final Approval,2026-04-11T15:20:00,Head Unit
REQ-002,Submit Request,2026-04-10T09:20:00,Unit B
REQ-002,Verify Budget,2026-04-12T14:00:00,Finance
REQ-002,Final Approval,2026-04-14T16:10:00,Head Unit
EOF
```

Cek isi file:

```
column -s, -t < ~/process-lab/event-log.csv
```

Contoh keluaran:

case_id	activity	timestamp	actor
REQ-001	Submit Request	2026-04-10T08:00:00	Unit A
REQ-001	Verify Budget	2026-04-10T10:30:00	Finance
REQ-001	Request Revision	2026-04-10T11:00:00	Finance
REQ-001	Resubmit Request	2026-04-11T09:15:00	Unit A
REQ-001	Verify Budget	2026-04-11T10:00:00	Finance
REQ-001	Final Approval	2026-04-11T15:20:00	Head Unit
REQ-002	Submit Request	2026-04-10T09:20:00	Unit B
REQ-002	Verify Budget	2026-04-12T14:00:00	Finance
REQ-002	Final Approval	2026-04-14T16:10:00	Head Unit

Dari contoh kecil ini saja, pembaca sudah bisa mulai melihat:

- REQ-001 mengalami *rework* karena ada *Request Revision*,
- REQ-002 punya jeda panjang antara *Submit Request* dan *Verify Budget*,
- dan titik *Verify Budget* tampak sebagai kandidat *bottleneck*.

Itulah cara berpikir proses yang harus ditanamkan sebelum berbicara tentang agent.

11.3 Memetakan proses bisnis sebelum memasang agent

Salah satu kesalahan paling mahal dalam proyek AI adalah memasang agent ke proses yang belum dipetakan. Hasilnya sering terlihat modern, tetapi sebenarnya hanya menutupi kekacauan yang sudah ada. Karena itu, **sebelum memasang agent, proses harus dipetakan**. Bukan harus sempurna, tetapi cukup jelas untuk menjawab empat pertanyaan:

- langkah apa saja yang benar-benar terjadi,
- siapa aktornya,
- input dan output tiap langkah apa,
- dan di mana titik yang paling menyita waktu atau paling sering salah.

Pemetaan proses bisa dimulai dari versi sangat sederhana. Ambil satu alur, misalnya *approval pengadaan internal*. Lalu tulis:

1. pengajuan masuk,
2. verifikasi anggaran,
3. perbaikan bila ada kekurangan,
4. approval akhir,
5. arsip dan notifikasi.

Setelah itu baru kita tanyakan:

- langkah mana yang paling sering menunggu,
- langkah mana yang paling banyak membaca dokumen,
- langkah mana yang paling sering memicu revisi,
- langkah mana yang sebenarnya bisa dibantu agent.

Literatur agentic dilampiran menekankan bahwa agent yang sehat perlu diberi **peran, batas, dan tujuan** yang jelas. Ada agent yang berperan sebagai manager, analyst, reflektor, atau searcher. Ini berguna sekali untuk proses bisnis, karena kita bisa mulai membedakan antara agent yang membaca konteks, agent yang memberi saran, dan agent yang menilai apakah proses perlu diblokkan atau eskalasi.

Langkah demi langkah: memetakan proses sederhana

Tahap 1. Tulis alur dalam teks

```
cat > ~/process-lab/process-map.txt <<'EOF'
```

Proses: Approval Pengadaan Internal

1. Unit kerja mengajukan permintaan.
2. Keuangan memverifikasi anggaran.
3. Jika tidak lengkap, dokumen dikembalikan untuk revisi.
4. Jika lengkap, kepala unit memberi approval akhir.
5. Hasil disimpan dan notifikasi dikirim.

EOF

Tahap 2. Lihat kembali peta proses

```
cat ~/process-lab/process-map.txt
```

Contoh keluaran:

Proses: Approval Pengadaan Internal

1. Unit kerja mengajukan permintaan.
2. Keuangan memverifikasi anggaran.
3. Jika tidak lengkap, dokumen dikembalikan untuk revisi.
4. Jika lengkap, kepala unit memberi approval akhir.
5. Hasil disimpan dan notifikasi dikirim.

Tahap 3. Tandai titik kandidat agent

Misalnya:

- membaca kelengkapan dokumen,
- merangkum konteks permintaan,
- memberi prioritas,
- menyiapkan notifikasi,
- atau memberi rekomendasi apakah kasus perlu revisi atau lanjut.

Di sinilah pembaca mulai berpikir bukan “AI mau diapakan?”, tetapi **bagian proses mana yang paling pantas dibantu mesin lebih dulu.**

11.4 Agent untuk membaca tiket, email, dokumen, dan notulen sebagai jejak proses

Di banyak organisasi, jejak proses tidak selalu hadir sebagai log API yang rapi. Sering kali ia tersebar dalam:

- tiket helpdesk,
- email internal,
- dokumen pengajuan,
- notulen rapat,
- komentar revisi,
- dan lampiran-lampiran kecil yang tampak sepele.

Justru di sinilah agentic AI punya nilai yang khas. Agent bisa dipakai untuk **membaca jejak proses yang berbahasa alami**, lalu mengubahnya menjadi sinyal kerja yang lebih terstruktur: kategori masalah, urgensi, langkah berikutnya, PIC, atau kebutuhan eskalasi. Ini sangat selaras dengan Open WebUI yang menempatkan *Knowledge* sebagai tempat dokumen dan koleksi yang dapat dicari, dibaca, dan dinalar oleh AI. ([Open WebUI](#))

Literatur agentic di referensi juga menekankan bahwa agent dapat menjadi *analyst*, *searcher*, atau *reflector* yang membaca informasi dan memberi rekomendasi untuk proses berikutnya. Bahkan untuk multi-agent systems, kemampuan berbagi pengetahuan dan mengkoordinasikan pembagian tugas adalah bagian penting dari arsitektur. Artinya, membaca tiket, email, dokumen, dan notulen bukan sekadar pekerjaan administrasi; itu bisa menjadi **input strategis** bagi workflow berikutnya.

Contoh sederhana: kita punya email yang secara isi sebenarnya adalah sinyal bahwa approval macet.

```
cat > ~/process-lab/email-approval.txt <<'EOF'
Subjek: Approval pengadaan belum selesai
Isi:
Dokumen pengadaan untuk proyek A sudah dikirim dua hari lalu.
Tim keuangan menyatakan lampiran anggaran kurang lengkap.
Mohon tindak lanjut karena approval dibutuhkan hari ini.
EOF
```

Sekarang kita minta model lokal membaca email ini dan mengubahnya menjadi ringkasan proses.

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [
    {
      "role": "user",
      "content": "Baca email berikut dan ubah menjadi ringkasan
proses dengan format: masalah, tahap proses, urgensi, dan
tindakan awal. Email: Dokumen pengadaan untuk proyek A sudah
dikirim dua hari lalu. Tim keuangan menyatakan lampiran anggaran
```

```

    kurang lengkap. Mohon tindak lanjut karena approval dibutuhkan
    hari ini."
  }
]
}'

```

Contoh keluaran:

```

{
  "model": "gemma3",
  "message": {
    "role": "assistant",
    "content": "Masalah: approval tertunda karena lampiran
anggaran tidak lengkap. Tahap proses: verifikasi anggaran.
Urgensi: tinggi. Tindakan awal: minta lampiran dilengkapi dan
tandai kasus untuk tindak lanjut cepat."
  },
  "done": true
}

```

Ini mengikuti pola API *chat* Ollama yang didokumentasikan resmi. ([Ollama](#))

Pelajaran pentingnya: **agent tidak selalu harus langsung bertindak. Kadang nilai terbesarnya justru pada kemampuannya membaca jejak proses yang selama ini tersebar dan mengubahnya menjadi konteks yang bisa dipakai workflow.**

11.5 Agent untuk rekomendasi langkah awal dan eskalasi

Setelah agent bisa membaca jejak proses, langkah berikutnya adalah membuatnya memberi **rekomendasi langkah awal**. Ini titik yang sangat kuat untuk bisnis, karena banyak proses macet bukan karena keputusan final terlalu sulit, tetapi karena **langkah awalnya lambat**. Orang bingung harus mulai dari mana. Harus kirim ke siapa. Apakah kasus ini biasa atau darurat. Apakah perlu revisi atau eskalasi. Di sinilah agent sangat membantu sebagai **advisor** atau **assistant**, bukan sebagai penguasa mutlak.

n8n mendokumentasikan dua pola yang sangat relevan di sini. Pertama, *AI Agent* dapat memakai tools untuk menjalankan langkah proses. Kedua, jika tool tertentu diberi *human review*, workflow akan berhenti dan meminta persetujuan manusia sebelum aksi dijalankan. Ini sangat cocok untuk *escalation flow*: agent boleh memberi saran dan menyiapkan aksi, tetapi manusia tetap memegang tombol akhir saat dampaknya besar. ([n8n Docs](#))

Contoh logika rekomendasi sederhana:

- bila masalah adalah “lampiran tidak lengkap” → kirim balik untuk revisi,
- bila masalah adalah “approval akhir dibutuhkan hari ini” → tandai urgensi tinggi,
- bila masalah menyentuh anggaran besar → minta review manusia,
- bila masalah ambigu → eskalasi ke supervisor.

Untuk mengujinya secara lokal, kita bisa memakai satu prompt sederhana:

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [
    {
      "role": "user",
      "content": "Berikut kasus proses internal: approval
tertunda karena lampiran anggaran tidak lengkap dan dibutuhkan
hari ini. Beri rekomendasi langkah awal dan apakah perlu
eskalasi."
    }
  ]
}'
```

Contoh keluaran:

```
{
  "model": "gemma3",
  "message": {
    "role": "assistant",
    "content": "Langkah awal: hubungi pengaju untuk melengkapi
lampiran, tandai status revisi, dan informasikan PIC keuangan.
Eskalasi: ya, karena kebutuhan approval hari ini menunjukkan
urgensi tinggi."
  },
  "done": true
}
```

Yang terpenting, pembaca perlu memahami bahwa rekomendasi awal bukan akhir dari proses. **Agent membantu mengurangi kebingungan, mempercepat langkah pertama, dan menaikkan kasus penting lebih cepat.** Itu sudah sangat bernilai.

11.6 Hands-on: membangun workflow agent untuk satu proses manajemen internal

Sekarang kita gabungkan semuanya dalam latihan kecil yang realistis. Ambil satu proses sederhana: **triase approval internal**. Tujuan kita bukan membuat sistem sempurna, tetapi membangun pola pikir:

1. baca sinyal masuk,
2. ringkas konteks,
3. tentukan kategori dan urgensi,
4. rekomendasikan langkah awal,
5. dan siapkan eskalasi bila perlu.

Untuk ini, n8n cocok dipakai sebagai penggerak workflow, sementara Ollama menjadi model lokal yang memberi analisis awal. Dokumentasi n8n menempatkan *Webhook* sebagai pintu masuk workflow dan *Respond to Webhook* sebagai titik balasan; dokumentasi *AI Agent* dan *human-in-the-loop* membuat pola approval juga masuk akal. ([n8n Docs](#))

Tahap 1. Siapkan payload contoh

Buat file JSON yang akan kita kirim ke workflow.

```
cat > ~/process-lab/request.json <<'EOF'
{
  "subject": "Approval pengadaan proyek A tertunda",
  "body": "Dokumen sudah dikirim dua hari lalu. Tim
keuangan menyatakan lampiran anggaran belum lengkap.
Approval dibutuhkan hari ini."
}
EOF
```

Cek isi:

```
cat ~/process-lab/request.json
```

Contoh keluaran:

```
{
  "subject": "Approval pengadaan proyek A tertunda",
  "body": "Dokumen sudah dikirim dua hari lalu. Tim
keuangan menyatakan lampiran anggaran belum lengkap.
Approval dibutuhkan hari ini."
}
```

Tahap 2. Buat workflow dasar di n8n

Di n8n, susun alur sederhana seperti ini:

- **Webhook** menerima kasus,
- **AI Agent** atau node LLM membaca isi,
- node berikutnya menyusun hasil ke format terstruktur,
- **IF / Switch** memisahkan kasus biasa dan kasus eskalasi,
- **Respond to Webhook** mengembalikan hasil,
- dan bila perlu tambahkan *human review* untuk aksi tertentu.

Pola ini sesuai dengan dokumentasi n8n tentang Webhook, AI Agent, dan *human-in-the-loop*. ([n8n Docs](#))

Tahap 3. Kirim request ke webhook

Misalnya URL webhook di n8n adalah:

```
http://localhost:5678/webhook/approval-triage
```

Kirim payload dengan **curl**:

```
curl -X POST http://localhost:5678/webhook/approval-triage \
-H "Content-Type: application/json" \
-d @~/process-lab/request.json
```

Contoh keluaran yang diharapkan dari workflow:

```
{
  "category": "approval workflow",
  "stage": "budget verification",
  "urgency": "high",
  "recommended_action": "minta lampiran anggaran
dilengkapi dan tandai kasus untuk tindak lanjut cepat",
  "escalation": true
}
```

Di sinilah pembaca mulai melihat keindahan agentic workflow: bukan lagi sekadar model yang menjawab pertanyaan, tetapi **alur kerja yang membaca input, menyiapkan konteks, memberi langkah awal, dan memutuskan apakah manusia perlu masuk**.

Tahap 4. Tambahkan human review untuk aksi sensitif

Bila workflow ingin benar-benar mengirim notifikasi, mengubah status sistem, atau memicu persetujuan, gunakan *human review* pada tool terkait.

Dokumentasi n8n menjelaskan bahwa saat AI ingin memakai tool yang digate, workflow akan berhenti dan mengirim permintaan approval ke kanal pilihan; bila disetujui, tool dieksekusi, bila ditolak, aksi dibatalkan. ([n8n Docs](#))

Tahap 5. Simpan hasil sebagai jejak proses

Agar nanti bisa dianalisis lagi, simpan hasil akhir ke file log sederhana.

```
cat >> ~/process-lab/triage-log.jsonl <<'EOF'
{"case_id":"REQ-003","category":"approval
workflow","urgency":"high","escalation":true}
EOF
```

Cek log:

```
cat ~/process-lab/triage-log.jsonl
```

Contoh keluaran:

```
{"case_id":"REQ-003","category":"approval
workflow","urgency":"high","escalation":true}
```

Kebiasaan kecil seperti ini akan sangat membantu saat pembaca masuk ke bab berikutnya tentang process mining.

Ringkasan langkah kerja bab ini

Agar mudah diingat, seluruh bab ini bisa dibaca sebagai enam langkah sederhana:

- **lihat proses sebagai rangkaian kejadian**
- **kenali bottleneck, rework, exception, dan approval loop**
- **petakan alur kerja sebelum memasang agent**
- **gunakan agent untuk membaca jejak proses yang berbahasa alami**
- **minta agent memberi rekomendasi langkah awal, bukan langsung jadi hakim final**
- **bungkus semuanya ke workflow yang punya trigger, jalur eskalasi, dan jejak log**

Penutup bab

Bab ini harus meninggalkan satu pelajaran kuat: **agentic AI paling berguna saat ia masuk ke proses yang nyata, berulang, dan penuh konteks — bukan saat ia dipamerkan sebagai chatbot yang fasih.** Proses bisnis sering lambat karena alur kerja buruk, tanggung jawab kabur, dan jejak proses tidak pernah dibaca dengan baik. Agent bisa membantu di titik-titik itu: membaca tiket, email, dokumen, dan notulen; merangkum posisi kasus; menandai urgensi; memberi langkah awal; dan menaikkan kasus ke manusia ketika risikonya meningkat.

Bagi mahasiswa dan praktisi, pesan utamanya sederhana:

- **jangan pasang agent ke proses yang belum dipahami**
- **mulailah dari satu alur yang kecil tetapi nyata**
- **anggap dokumen dan tiket sebagai jejak proses, bukan sekadar arsip**
- **biarkan agent mempercepat langkah awal**
- **dan biarkan manusia tetap memegang keputusan akhir saat risikonya besar**

Kalau Bab 8 memberi agent pengetahuan, dan Bab 9–10 memberi data yang lebih rapi, maka **Bab 11 mengajari agent itu masuk ke jalan raya proses bisnis yang sesungguhnya.**

Bab 12. Process mining ringan dengan open source untuk menemukan peluang agentic AI

12.1 Apa itu *process mining* dan mengapa relevan

Secara sederhana, *process mining* adalah cara membaca **jejak kejadian proses** untuk memahami bagaimana proses benar-benar berjalan. Bukan bagaimana proses *seharusnya* berjalan di SOP, bukan juga bagaimana manajer *merasa* proses berjalan, tetapi bagaimana langkah-langkah itu benar-benar terjadi di data. PM4Py dan ekosistem Process Intelligence Solutions menempatkan *process mining* sebagai pendekatan berbasis *event data* untuk **process discovery**, **conformance checking**, **performance analysis**, *variant analysis*, dan *bottleneck analysis*. ([Process Intelligence Solutions](#))

Mengapa ini relevan untuk *agentic AI*? Karena banyak organisasi terlalu cepat bertanya, “di mana kita bisa pasang agent?” Padahal pertanyaan yang lebih matang adalah: **proses mana yang benar-benar macet, penuh *rework*, penuh *exception*, atau terlalu banyak membaca konteks secara manual?** *Process mining* membantu menjawab pertanyaan ini dengan data, bukan dengan asumsi.

Ada beberapa alasan mengapa *process mining* sangat cocok menjadi jembatan menuju *agentic AI*:

- **la menunjukkan kenyataan proses**, bukan hanya desain formalnya.
- **la membantu menemukan bottleneck**, yaitu titik yang paling lambat atau paling penuh antrean.
- **la menyingkap *rework* dan penyimpangan**, misalnya dokumen yang bolak-balik direvisi.
- **la memberi dasar prioritas use case**, sehingga kita tidak memasang agent secara membabi buta.
- **la membantu mengukur hasil sebelum dan sesudah intervensi agent.** ([Process Intelligence Solutions](#))

Kalimat penting untuk pembaca adalah ini: **process mining bukan lawan AI. Ia justru membantu AI masuk ke proses yang tepat.**

12.2 Process discovery, conformance checking, dan bottleneck analysis

Tiga istilah ini adalah fondasi praktis yang paling berguna untuk pembaca buku ini.

a. Process discovery

Process discovery adalah proses “menemukan” model proses dari *event log* secara otomatis. Alih-alih menggambar alur kerja berdasarkan perkiraan, kita membiarkan data menunjukkan urutan aktivitas yang paling sering terjadi. PM4Py mendokumentasikan dukungan untuk berbagai teknik discovery, termasuk **Directly-Follows Graph (DFG)** dan varian lain, termasuk *object-centric discovery* pada log tertentu. ([PM4Py Documentation](#))

Dalam bahasa sederhana, *process discovery* menjawab pertanyaan:

- langkah apa yang paling sering terjadi,
- urutan mana yang paling umum,
- dan jalur mana yang paling banyak dilalui kasus.

b. Conformance checking

Conformance checking membandingkan proses aktual dengan model atau proses yang diharapkan. PMTk dan PM4Py menampilkan *conformance checking* sebagai fitur inti untuk **mengidentifikasi dan menganalisis deviasi proses**. ([Process Intelligence Solutions](#))

Dalam konteks bisnis, ini sangat berguna untuk menjawab:

- apakah approval benar-benar mengikuti SOP,
- apakah tiket melewati tahap yang seharusnya,
- apakah ada aktivitas yang sering dilewati,
- atau apakah ada jalur “liar” yang tidak terdokumentasi.

c. Bottleneck analysis

Bottleneck analysis melihat dimana proses melambat. PMTk secara eksplisit menyebut *process bottleneck analysis* dan *performance view* untuk menganalisis proses lambat dan *throughput analysis* untuk membaca perkembangan temporal proses. ([Process Intelligence Solutions](#))

Dalam bahasa kerja, *bottleneck analysis* menjawab:

- tahap mana yang paling lama,
- kasus mana yang paling sering menunggu,
- dan di titik mana agent mungkin bisa membantu membaca konteks, merangkum, mengklasifikasi, atau menyiapkan tindak lanjut.

Kalau diringkas:

- **process discovery** menunjukkan **alur nyata**,
- **conformance checking** menunjukkan **penyimpangan**,
- **bottleneck analysis** menunjukkan **titik sakitnya**.

12.3 Mengubah log operasional menjadi insight proses

Banyak pembaca mengira mereka tidak punya data untuk *process mining*. Padahal sering kali mereka sudah punya:

- log tiket,
- riwayat approval,
- jejak email,
- catatan perubahan status,
- waktu pembuatan dan revisi dokumen,
- atau ekspor CSV dari sistem internal.

Agar bisa dipakai, log itu minimal perlu memuat empat unsur:

- **ID kasus** (*case id*),
- **aktivitas**,
- **waktu** (*timestamp*),
- **pelaku** atau atribut tambahan bila ada.

PM4Py memang menerima *event log* atau *pandas DataFrame* untuk banyak fungsi discovery dan analisis, termasuk *performance DFG*. ([PM4Py Documentation](#))

Langkah demi langkah: membuat event log sederhana

Tahap 1. Buat file CSV

```
mkdir -p ~/pm-lab
cat > ~/pm-lab/approval_log.csv <<'EOF'
case_id,activity,timestamp,actor
REQ-001,Submit Request,2026-04-10 08:00:00,Unit A
REQ-001,Verify Budget,2026-04-10 10:30:00,Finance
REQ-001,Request Revision,2026-04-10 11:00:00,Finance
REQ-001,Resubmit Request,2026-04-11 09:15:00,Unit A
REQ-001,Verify Budget,2026-04-11 10:00:00,Finance
REQ-001,Final Approval,2026-04-11 15:20:00,Head Unit
REQ-002,Submit Request,2026-04-10 09:20:00,Unit B
REQ-002,Verify Budget,2026-04-12 14:00:00,Finance
REQ-002,Final Approval,2026-04-14 16:10:00,Head Unit
EOF
```

Tahap 2. Lihat isi file

```
column -s, -t < ~/pm-lab/approval_log.csv
```

Contoh keluaran:

case_id	activity	timestamp	actor
REQ-001	Submit Request	2026-04-10 08:00:00	Unit A
REQ-001	Verify Budget	2026-04-10 10:30:00	Finance
REQ-001	Request Revision	2026-04-10 11:00:00	Finance
REQ-001	Resubmit Request	2026-04-11 09:15:00	Unit A
REQ-001	Verify Budget	2026-04-11 10:00:00	Finance
REQ-001	Final Approval	2026-04-11 15:20:00	Head Unit
REQ-002	Submit Request	2026-04-10 09:20:00	Unit B
REQ-002	Verify Budget	2026-04-12 14:00:00	Finance
REQ-002	Final Approval	2026-04-14 16:10:00	Head Unit

Dari file sekecil ini, pembaca sudah bisa mulai mendapatkan *insight* proses:

- ada *rework* pada REQ-001,
- ada jeda panjang di tahap *Verify Budget* pada REQ-002,
- dan tahap verifikasi anggaran mungkin adalah kandidat *bottleneck*.

Pelajaran pentingnya: **insight proses tidak harus menunggu sistem raksasa. Ia bisa dimulai dari CSV sederhana.**

12.4 PM4Py sebagai pustaka open source untuk analisis proses

Di sinilah PM4Py masuk sebagai alat kerja yang sangat berguna. Situs dan dokumentasi resminya menyebut PM4Py sebagai pustaka *open source* Python untuk *process mining* dengan dukungan **process discovery**, **conformance checking**, **predictive process mining**, *organizational mining*, *clustering*, dan *object-centric process mining*. Dokumentasinya juga menyediakan *installation guidelines*, *tutorials*, *examples*, dan API reference. ([Process Intelligence Solutions](#))

Untuk pembaca buku ini, PM4Py menarik karena:

- berbasis Python,
- cukup ringan untuk eksperimen awal,
- punya banyak algoritma siap pakai,
- dan cukup dekat dengan dunia data science yang sudah akrab bagi banyak mahasiswa dan praktisi.

Langkah demi langkah: memasang PM4Py

Tahap 1. Buat virtual environment

```
python3 -m venv ~/venv-pm4py
source ~/venv-pm4py/bin/activate
```

Tahap 2. Pasang PM4Py dan pustaka pendukung

```
pip install --upgrade pip
pip install pm4py pandas
```

Dokumentasi PM4Py memang menyediakan panduan pemasangan dan contoh memulai melalui dokumentasinya. ([Process Intelligence Solutions](#))

Tahap 3. Verifikasi instalasi

```
python -c "import pm4py; print(pm4py.__doc__ is not None)"
```

Contoh keluaran:

```
True
```

Atau cara sederhana lain:

```
python - <<'EOF'
import pm4py
print("PM4Py siap dipakai")
EOF
```

Contoh keluaran:

```
PM4Py siap dipakai
```

Membaca log dengan PM4Py

Sekarang kita gunakan CSV yang tadi dibuat.

```
python - <<'EOF'
import pandas as pd

df = pd.read_csv('~/.pm-lab/approval_log.csv')
print(df.head())
EOF
```

Contoh keluaran:

	case_id	activity	timestamp	actor
0	REQ-001	Submit Request	2026-04-10 08:00:00	Unit A
1	REQ-001	Verify Budget	2026-04-10 10:30:00	Finance
2	REQ-001	Request Revision	2026-04-10 11:00:00	Finance

Walau dokumentasi PM4Py lebih menekankan fungsi-fungsi API ketimbang tutorial langkah demi langkah di hasil ringkas yang kita lihat, pustaka ini memang menerima log berbentuk DataFrame untuk sejumlah fungsi discovery dan analisis. ([PM4Py Documentation](#))

12.5 Menghubungkan hasil process mining ke prioritas use case agentic AI

Ini inti strategis bab ini. *Process mining* bukan tujuan akhir. Ia adalah **alat diagnosis**. Nilainya muncul saat hasil diagnosis dipakai untuk menentukan **prioritas use case agentic AI**.

Cara berpikirnya seperti ini:

- Kalau *process discovery* menunjukkan jalur yang berulang dan padat konteks, itu kandidat agent untuk **klasifikasi, ringkasan, atau routing**.
- Kalau *conformance checking* menunjukkan deviasi berulang, itu kandidat agent untuk **pengecekan kepatuhan langkah atau peringatan dini**.
- Kalau *bottleneck analysis* menunjukkan tahap yang lama karena banyak membaca dokumen, itu kandidat agent untuk **ringkasan dokumen, pemeriksaan kelengkapan, atau rekomendasi langkah awal**.
- Kalau log menunjukkan banyak *rework*, itu kandidat agent untuk **pre-check, validasi awal, atau draft perbaikan**.

PMTk dan PM4Py menyoroti analisis seperti *variant exploration*, *case exploration*, *throughput analysis*, dan *bottleneck analysis*, yang semuanya sangat cocok dijadikan dasar penentuan prioritas intervensi proses. ([Process Intelligence Solutions](#))

Contoh koneksi praktis

Misalnya hasil log menunjukkan:

- tahap *Verify Budget* lambat,
- banyak permintaan revisi,
- approval akhir sering tertunda karena dokumen tidak lengkap.

Maka peluang *agentic AI* yang masuk akal bisa berupa:

- agent untuk **membaca dokumen pengajuan dan mengecek kelengkapan awal**,
- agent untuk **meringkas konteks permintaan bagi tim keuangan**,
- agent untuk **menandai kasus yang perlu eskalasi cepat**,
- atau workflow yang **secara otomatis meminta revisi dengan alasan yang jelas**.

Kalimat penting untuk pembaca: **jangan mulai dari “kita ingin agent.” Mulailah dari “di titik mana proses paling rugi kalau terus dibiarkan manual.”**

12.6 Hands-on: membaca event log sederhana dan menemukan kandidat otomasi / agent

Sekarang kita gabungkan semuanya dalam latihan kecil yang bisa langsung diikuti.

Tahap 1. Siapkan event log

Gunakan file CSV dari bagian sebelumnya.

```
cat ~/pm-lab/approval_log.csv
```

Tahap 2. Ringkas aktivitas per kasus dengan Python

```
python - <<'EOF'
import pandas as pd

df = pd.read_csv('~/pm-lab/approval_log.csv')
summary = df.groupby('case_id')['activity'].apply(list)
for case_id, acts in summary.items():
    print(case_id, "->", " | ".join(acts))
EOF
```

Contoh keluaran:

```
REQ-001 -> Submit Request | Verify Budget | Request
Revision | Resubmit Request | Verify Budget | Final
Approval
REQ-002 -> Submit Request | Verify Budget | Final Approval
```

Dari sini saja, pembaca sudah bisa melihat *variant* proses.

Tahap 3. Hitung durasi sederhana antar event

```
python - <<'EOF'
import pandas as pd

df = pd.read_csv('~/pm-lab/approval_log.csv')
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.sort_values(['case_id', 'timestamp'])
df['next_time'] =
df.groupby('case_id')['timestamp'].shift(-1)
df['waiting_hours'] = (df['next_time'] -
df['timestamp']).dt.total_seconds() / 3600
print(df[['case_id', 'activity', 'waiting_hours']])
EOF
```

Contoh keluaran:

	case_id	activity	waiting_hours
0	REQ-001	Submit Request	2.5
1	REQ-001	Verify Budget	0.5
2	REQ-001	Request Revision	22.25
3	REQ-001	Resubmit Request	0.75
4	REQ-001	Verify Budget	5.33
5	REQ-001	Final Approval	NaN
6	REQ-002	Submit Request	52.67
7	REQ-002	Verify Budget	50.17
8	REQ-002	Final Approval	NaN

Dari contoh kecil ini, terlihat bahwa:

- REQ-002 menunggu sangat lama setelah submit,
- tahap verifikasi tampaknya menjadi titik sakit,
- dan *request revision* memicu *rework* yang signifikan.

Tahap 4. Tentukan kandidat intervensi agent

Sekarang buat ringkasan keputusan sederhana.

```
cat > ~/pm-lab/kandidat-agent.txt <<'EOF'
Kandidat intervensi:
1. Agent pengecekan kelengkapan dokumen sebelum verifikasi
anggaran.
2. Agent ringkasan konteks permintaan untuk tim keuangan.
3. Agent eskalasi otomatis untuk kasus dengan SLA tinggi.
4. Workflow notifikasi revisi dengan alasan yang lebih
jelas.
EOF
```

Cek hasil:

```
cat ~/pm-lab/kandidat-agent.txt
```

Contoh keluaran:

```
Kandidat intervensi:
1. Agent pengecekan kelengkapan dokumen sebelum verifikasi
anggaran.
2. Agent ringkasan konteks permintaan untuk tim keuangan.
3. Agent eskalasi otomatis untuk kasus dengan SLA tinggi.
4. Workflow notifikasi revisi dengan alasan yang lebih
jelas.
```

Di sinilah pembaca melihat jembatan yang sangat konkret:

event log → insight proses → prioritas use case agentic AI.

Tahap 5. Uji agent lokal untuk memberi saran

Kita bisa pakai Ollama untuk membaca ringkasan hasil analisis dan memberi saran awal.

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [
    {
      "role": "user",
      "content": "Berikut hasil log proses: Verify Budget
sering lama, ada banyak Request Revision, dan approval
akhir tertunda. Beri 3 use case agentic AI yang paling
masuk akal."
    }
  ]
}'
```

Contoh keluaran:

```
{
  "model": "gemma3",
  "message": {
    "role": "assistant",
    "content": "1. Agent pengecekan kelengkapan dokumen.
2. Agent ringkasan konteks untuk verifikasi anggaran. 3.
Agent eskalasi kasus prioritas tinggi."
  },
  "done": true
}
```

Pola API *chat* ini didukung dokumentasi Ollama. ([PM4Py Documentation](#))

Ringkasan langkah kerja bab ini

Agar mudah diingat, pola kerja bab ini bisa diringkas seperti berikut:

- **kumpulkan event log sederhana**
- **lihat urutan aktivitas aktual**
- **ukur waktu tunggu dan cari bottleneck**
- **cek adanya *rework*, deviasi, dan approval loop**
- **terjemahkan insight proses menjadi kandidat intervensi**
- **pilih use case agentic AI yang paling kecil tetapi paling sakit dampaknya**

Penutup bab

Bab ini harus meninggalkan satu pesan kuat: **process mining membantu kita berhenti berdebat berdasarkan opini, lalu mulai memperbaiki proses berdasarkan jejak nyata**. PM4Py dan ekosistem Process Intelligence Solutions memang dirancang untuk kebutuhan seperti ini: membaca *event data*, melakukan *process discovery*, *conformance checking*, *throughput analysis*, *variant exploration*, dan *bottleneck analysis*. Semua itu sangat cocok dijadikan jembatan ke desain agent bisnis yang lebih tajam. ([Process Intelligence Solutions](#))

Bagi mahasiswa dan praktisi, pelajaran terpenting dari bab ini sederhana:

- **jangan pasang agent hanya karena teknologi terlihat menarik**
- **lihat dulu jejak prosesnya**
- **cari titik paling mahal dan paling berulang**
- **pakai data untuk memilih prioritas**
- **dan jadikan *process mining* sebagai kompas, bukan sekadar alat analisis.**

Kalau Bab 11 mengajari agent masuk ke alur kerja, maka **Bab 12 mengajari kita memilih jalan mana yang paling layak diperbaiki lebih dulu.**

Bagian VI. Governance, Keamanan, dan Jalan ke Produksi

Bab 13. Guardrails, governance, dan observability

13.1 Mengapa agent tidak boleh dibiarkan liar

Kesalahan paling berbahaya dalam *agentic AI* adalah menganggap bahwa semakin bebas agent bertindak, semakin hebat sistemnya. Dalam bisnis, logika itu justru sering terbalik. **Semakin besar kebebasan agent, semakin besar juga potensi kerusakan bila ia salah konteks, salah tool, salah prioritas, atau salah membaca dokumen.** Karena itu agent harus ditempatkan di dalam pagar yang jelas: apa yang boleh ia lakukan, apa yang hanya boleh ia sarankan, dan apa yang harus selalu dikembalikan ke manusia. Literatur *agentic* dan *enterprise* yang kita pakai juga konsisten di titik ini: agent yang sehat membutuhkan kontrol, batas aksi, dan pengawasan manusia, bukan otonomi tanpa rem.

Secara praktis, ada tiga alasan utama mengapa agent tidak boleh dilepas bebas. Pertama, **model tidak pernah benar 100%**. Kedua, agent sering bekerja lintas konteks: dokumen, tiket, *knowledge base*, dan workflow. Ketiga, begitu ia diberi akses ke tools, ia tidak lagi hanya “berbicara”, tetapi mulai memicu tindakan. Dokumentasi n8n sangat jelas bahwa tool tertentu bisa diganti agar AI Agent harus menunggu persetujuan manusia dulu sebelum mengeksekusi aksi. Itu adalah pengakuan praktis bahwa *agentic AI* diproduksi memang butuh pagar, bukan keberanian kosong. ([n8n Docs](#))

Cara berpikir yang sehat untuk pembaca adalah ini: **agent yang baik bukan agent yang paling bebas, tetapi yang paling tepat tahu batasnya.** Di organisasi, itu berarti:

- tahu kapan cukup memberi saran,
- tahu kapan boleh memanggil tool,
- tahu kapan harus berhenti,
- dan tahu kapan harus menaikkan kasus ke manusia.

13.2 Threshold, escalation, approval workflow

Guardrails menjadi nyata ketika diterjemahkan menjadi aturan operasional. Tiga yang paling penting adalah **threshold**, **escalation**, dan **approval workflow**. *Threshold* berarti ambang tertentu yang menentukan apakah agent boleh lanjut otomatis atau harus berhenti. *Escalation* berarti jalur naik ke manusia atau otoritas yang lebih tinggi ketika kasus lebih sensitif, lebih ambigu, atau lebih berisiko. *Approval workflow* berarti ada titik persetujuan eksplisit sebelum aksi tertentu dijalankan. Dokumentasi n8n menyebutkan dengan sangat konkret bahwa saat tool memerlukan *human review*, workflow akan pause, lalu manusia bisa memilih *Approve* atau *Deny*. Jika disetujui, tool dijalankan; jika ditolak, aksi dibatalkan. ([n8n Docs](#))

Dalam bahasa bisnis, pola ini sangat berguna untuk situasi seperti:

- pengiriman notifikasi keluar organisasi,
- perubahan data penting,
- persetujuan dokumen,
- keputusan prioritas tinggi,
- atau kasus yang menyentuh keuangan dan hak akses.

Agar pembaca melihat bentuk nyatanya, berikut contoh *threshold policy* sederhana untuk triase approval internal:

```
cat > ~/guardrails/threshold-policy.yaml <<'EOF'
policy:
  workflow: approval_internal
  thresholds:
    urgency_high: escalate
    missing_budget_attachment: require_revision
    sensitive_financial_value: human_approval
    ambiguous_case: escalate
EOF
```

Cek hasilnya:

```
cat ~/guardrails/threshold-policy.yaml
```

Contoh keluaran:

```
policy:
  workflow: approval_internal
  thresholds:
    urgency_high: escalate
    missing_budget_attachment: require_revision
    sensitive_financial_value: human_approval
    ambiguous_case: escalate
```

Di titik ini pembaca perlu paham bahwa *threshold* tidak harus selalu kompleks. **Yang penting adalah eksplisit.** Kebingungan organisasi sering bukan karena aturan kurang pintar, tetapi karena aturan tidak pernah ditulis.

13.3 Monitoring: task completion, latency, accuracy, cost

Agent yang sudah dipasang tetapi tidak dipantau akan cepat menjadi sumber rasa aman palsu. Karena itu bab ini harus menegaskan bahwa *monitoring* adalah bagian inti, bukan aksesori. Untuk agentic workflow bisnis, ada empat metrik dasar yang sangat berguna: **task completion**, **latency**, **accuracy**, dan **cost**. Literatur enterprise juga menekankan bahwa agentic system di dunia nyata harus dinilai dari hasil kerja dan performa operasional, bukan sekadar kualitas bahasa.

Task completion menjawab: apakah tugas benar-benar selesai sampai titik yang diharapkan? Misalnya tiket berhasil diklasifikasikan, ringkasan berhasil dibuat, atau workflow berhasil mencapai status akhir. **Latency** menjawab: berapa lama sistem membutuhkan waktu dari input sampai output. **Accuracy** menjawab: seberapa tepat hasilnya terhadap baseline, label, atau keputusan manusia. **Cost** menjawab: berapa biaya komputasi, pemanggilan model, waktu kerja, dan overhead operasional yang dikeluarkan. Literatur engineering di referensi memang berkali-kali menekankan trade-off antara kualitas, latensi, dan biaya pada desain agentic system.

Untuk eksperimen lokal, pembaca bisa mulai dari metrik paling sederhana. Misalnya, simpan hasil triase ke file log lalu hitung jumlah kasus sukses dan waktu respons.

```
cat > ~/guardrails/metrics.jsonl <<'EOF'
{"case_id":"REQ-101","task":"triage","completed":true,"latency_ms":1850,"correct":true,"cost_note":"local_inference"}
{"case_id":"REQ-102","task":"triage","completed":true,"latency_ms":2310,"correct":false,"cost_note":"local_inference"}
{"case_id":"REQ-103","task":"triage","completed":false,"latency_ms":4120,"correct":false,"cost_note":"local_inference"}
EOF
```

Lalu hitung ringkasannya:

```
python - <<'EOF'
import json

rows = [json.loads(x) for x in
open('/home/oai/guardrails/metrics.jsonl')]
completed = sum(1 for r in rows if r["completed"])
correct = sum(1 for r in rows if r["correct"])
avg_latency = sum(r["latency_ms"] for r in rows) / len(rows)

print("Total kasus:", len(rows))
print("Selesai:", completed)
print("Benar:", correct)
print("Rata-rata latency (ms):", round(avg_latency, 2))
EOF
```

Contoh keluaran:

```
Total kasus: 3
Selesai: 2
```

```
Benar: 1
Rata-rata latency (ms): 2760.0
```

Ini memang sangat sederhana, tetapi justru di situ kekuatannya. **Monitoring tidak harus menunggu platform besar. Ia bisa dimulai dari kebiasaan mencatat.**

13.4 Logs, replay, audit trail, dan session state

Ketika agent salah, pertanyaan pertama yang muncul biasanya bukan “mengapa model ini buruk?”, tetapi “tadi ia membaca apa, memanggil tool apa, dan kenapa sampai memilih langkah itu?” Karena itu *logs*, *replay*, *audit trail*, dan *session state* sangat penting.

Logs adalah catatan teknis kejadian: prompt, tool calls, hasil retrieval, error, waktu respons, status workflow. **Replay** berarti kemampuan memutar ulang kasus atau langkah yang sama untuk debugging. **Audit trail** adalah jejak yang lebih formal: siapa melakukan apa, kapan, atas dasar data apa, dan hasil akhirnya apa. **Session state** adalah keadaan konteks dalam satu sesi kerja: dokumen yang sedang aktif, *knowledge base* yang ditempel, hasil tool terakhir, dan memori pendek yang sedang dipakai. Open WebUI menekankan bahwa akses dan kemampuan pengguna bisa diatur lewat roles, permissions, dan groups, sementara sumber daya seperti model dan *knowledge base* juga bisa dibatasi aksesnya. Dalam praktik, struktur akses seperti ini ikut memperkuat audit trail karena kita tahu siapa seharusnya bisa melihat atau mengubah apa. ([Open WebUI](#))

Untuk pembaca buku ini, latihan paling berguna adalah membiasakan diri mencatat satu siklus keputusan agent.

```
cat > ~/guardrails/audit-log.jsonl <<'EOF'
{"time":"2026-04-15T09:10:00","case_id":"REQ-201","event":"input_received","actor":"webhook","detail":"approval request masuk"}
{"time":"2026-04-15T09:10:02","case_id":"REQ-201","event":"rag_lookup","actor":"agent","detail":"SOP approval pengadaan ditemukan"}
{"time":"2026-04-15T09:10:04","case_id":"REQ-201","event":"recommendation","actor":"agent","detail":"minta revisi lampiran"}
{"time":"2026-04-15T09:10:10","case_id":"REQ-201","event":"human_approval","actor":"supervisor","detail":"eskalasi disetujui"}
EOF
```

Cek isi:

```
cat ~/guardrails/audit-log.jsonl
```

Contoh keluaran:

```
{ "time": "2026-04-15T09:10:00", "case_id": "REQ-201", "event": "input_received", "actor": "webhook", "detail": "approval request masuk" }
{ "time": "2026-04-15T09:10:02", "case_id": "REQ-201", "event": "rag_lookup", "actor": "agent", "detail": "SOP approval pengadaan ditemukan" }
{ "time": "2026-04-15T09:10:04", "case_id": "REQ-201", "event": "recommendation", "actor": "agent", "detail": "minta revisi lampiran" }
{ "time": "2026-04-15T09:10:10", "case_id": "REQ-201", "event": "human_approval", "actor": "supervisor", "detail": "eskalasi disetujui" }
```

Dari log sekecil ini saja, organisasi sudah punya dasar untuk menjawab:

- agent membaca apa,
- merekomendasikan apa,
- siapa yang mengesahkan,
- dan kapan keputusan itu terjadi.

Agent tanpa jejak adalah risiko. Agent dengan jejak adalah sistem yang bisa dipelajari dan diperbaiki.

13.5 Security minimum: secrets, least privilege, sandbox, policy checks

Agentic AI tanpa kebersihan keamanan dasar akan cepat berubah dari alat bantu menjadi lubang risiko. Karena itu buku ini perlu menanamkan “*security minimum*” yang sederhana tetapi tegas.

Pertama, **secrets harus diperlakukan sebagai secrets**. Docker secara resmi menegaskan bahwa *secret* adalah data sensitif seperti password, private key, sertifikat, atau API token yang tidak boleh disimpan di Dockerfile, source code, atau dikirim sembarangan. Compose juga mendukung deklarasi *secrets* di level aplikasi, dan Docker menekankan bahwa *secrets* dapat berasal dari file atau environment tertentu. ([Docker Documentation](#))

Contoh file secret lokal:

```
mkdir -p ~/guardrails/secrets
echo "super-rahasia-token" > ~/guardrails/secrets/api_token.txt
chmod 600 ~/guardrails/secrets/api_token.txt
ls -l ~/guardrails/secrets/api_token.txt
```

Contoh keluaran:

```
-rw----- 1 user user 20 Apr 15 10:12
/home/user/guardrails/secrets/api_token.txt
```

Bila memakai Compose, pola rahasianya bisa ditulis seperti ini:

```
secrets:
  api_token:
    file: ./secrets/api_token.txt

services:
  app:
    image: example/app
    secrets:
      - api_token
```

Format ini sesuai dengan dokumentasi resmi Docker Compose tentang *secrets*.

([Docker Documentation](#))

Kedua, **least privilege**. Prinsipnya sederhana: agent hanya boleh melihat, membaca, dan menulis seperlunya. Open WebUI mendukung RBAC, roles, dan permissions. Admin tidak harus diberikan ke semua orang. Model dan *knowledge base* sensitif juga bisa dibatasi ke grup tertentu. Ini sangat penting untuk organisasi yang mulai menaruh dokumen internal ke sistem. ([Open WebUI](#))

Ketiga, **sandbox**. Kalau agent boleh menjalankan command, mengakses file, atau memakai terminal, kemampuan itu harus dibatasi ketat. Open WebUI sendiri memiliki panduan keamanan untuk fitur terminal dan secara eksplisit mengingatkan bahwa memberi AI kemampuan menjalankan command berarti memberi daya yang sangat nyata, sehingga praktik keamanan harus diperketat. ([Open WebUI](#))

Keempat, **policy checks**. Sebelum tool atau aksi tertentu dijalankan, sistem sebaiknya memeriksa apakah aksi itu sesuai kebijakan: boleh atau tidak, perlu manusia atau tidak, menyentuh data sensitif atau tidak. Ini adalah bentuk paling praktis dari *governance*: aturan tidak hanya ditulis, tetapi dipakai sebagai gerbang operasional.

13.6 Prompt injection, data leakage, dan failure handling

Bab ini sebaiknya ditutup dengan tiga ancaman yang sangat nyata.

Prompt injection terjadi ketika input pengguna, dokumen, atau konten eksternal menyisipkan instruksi yang mencoba mengubah perilaku agent. Dalam sistem yang memakai RAG, ini bisa muncul dari dokumen internal yang kotor, file yang diunggah, atau halaman eksternal yang ditarik ke konteks. Akibatnya agent bisa diarahkan untuk mengabaikan instruksi sistem, membuka data yang tidak seharusnya, atau memanggil tool yang salah. Open WebUI dan ekosistem agent pada umumnya tidak kebal terhadap masalah ini hanya karena modelnya lokal. Lokal bukan berarti otomatis aman. Karena itu pembatasan tool, RBAC, dan *human approval* tetap penting. ([Open WebUI](#))

Data leakage terjadi ketika jawaban atau aksi agent membocorkan informasi yang tidak seharusnya tampil. Risiko ini meningkat bila *knowledge base* tidak dipisah, roles tidak dibatasi, atau agent diberi akses terlalu luas. Dokumentasi Open WebUI sangat berguna di sini karena mendukung pembatasan model dan *knowledge base* berbasis grup maupun user individual. Artinya, *governance* data harus diterapkan sejak struktur sumber daya, bukan baru saat ada insiden. ([Open WebUI](#))

Failure handling adalah cara sistem menghadapi kegagalan tanpa membuat kerusakan lebih besar. Dalam agentic workflow, kegagalan bisa berupa:

- retrieval gagal,
- model timeout,
- tool error,
- output tidak valid,
- approval tidak datang,
- atau hasil terlalu ambigu.

Respons yang sehat bukan “paksa lanjut”, tetapi:

- *retry* terbatas,
- fallback ke manusia,
- kembalikan pesan error yang jelas,
- simpan log kegagalan,
- dan tandai kasus untuk review.

n8n menyediakan pola *human fallback* dan *human-in-the-loop* yang sangat cocok untuk ini. Saat AI tidak cukup yakin atau saat tool tertentu terlalu sensitif, workflow bisa mengembalikan kasus ke manusia alih-alih nekat meneruskan. ([n8n Docs](#))

Untuk pembaca yang ingin menanamkan kebiasaan sederhana, berikut contoh *failure policy* kecil:

```
cat > ~/guardrails/failure-policy.yaml <<'EOF'
failure_policy:
  max_retries: 2
  on_timeout: escalate_to_human
  on_invalid_output: retry_once_then_escalate
  on_tool_error: log_and_pause
  on_sensitive_action_without_approval: deny
EOF
```

Cek hasil:

```
cat ~/guardrails/failure-policy.yaml
```

Contoh keluaran:

```
failure_policy:
  max_retries: 2
  on_timeout: escalate_to_human
  on_invalid_output: retry_once_then_escalate
  on_tool_error: log_and_pause
  on_sensitive_action_without_approval: deny
```

Kebiasaan kecil seperti ini membantu organisasi berpikir bukan hanya tentang “bagaimana agent berhasil”, tetapi juga **bagaimana agent gagal dengan aman**.

Ringkasan langkah kerja bab ini

Agar mudah diingat, pola kerja bab ini bisa diringkas sebagai berikut:

- **tetapkan batas aksi agent**
- **buat threshold dan aturan eskalasi**
- **pakai approval workflow untuk langkah sensitif**
- **catat metrik utama: selesai, cepat, tepat, dan hemat**
- **simpan logs dan audit trail**
- **kelola secrets dan hak akses dengan ketat**
- **anggap prompt injection dan data leakage sebagai risiko nyata**
- **siapkan failure handling sebelum insiden terjadi**

Penutup bab

Bab ini harus meninggalkan satu pelajaran inti: **agent yang layak dipakai dalam bisnis adalah agent yang bisa diawasi, dikoreksi, dibatasi, dan dijelaskan**. n8n menyediakan *human-in-the-loop* dan *human fallback* untuk tool calls. Open WebUI menyediakan RBAC, roles, permissions, dan pembatasan akses ke model maupun *knowledge base*. Docker menyediakan *secrets* agar token dan password tidak disimpan sembarangan. Semua ini menunjukkan hal yang sama: *agentic AI* yang matang selalu dibangun di atas pagar operasional yang jelas, bukan di atas keberanian kosong. ([n8n Docs](#))

Bagi mahasiswa dan praktisi, pesan akhirnya sederhana:

- **jangan bangga pada agent yang paling liar**
- **banggalah pada agent yang paling bisa dipercaya**
- **buat aturan sebelum masalah datang**
- **buat jejak sebelum lupa**
- **dan pastikan manusia tetap punya tombol berhenti**

Kalau Bab 12 membantu memilih proses yang layak diperbaiki, maka **Bab 13 memastikan perbaikan itu tidak berubah menjadi sumber risiko baru**.

Bab 14. Dari pilot ke produksi ringan: roadmap 90 hari

14.1 Memulai dari satu proses dan satu owner

Kesalahan klasik dalam adopsi *agentic AI* adalah ingin langsung memperbaiki semuanya. Hasilnya hampir selalu sama: scope melebar, definisi sukses kabur, dan tidak ada satu orang pun yang benar-benar bertanggung jawab. Karena itu, langkah pertama yang sehat adalah **memulai dari satu proses dan satu owner**. *Pilot* yang baik harus cukup sempit untuk dikendalikan, tetapi cukup penting untuk memberi nilai. Literatur dilampirkan bahkan menyarankan agar pilot awal menghindari domain dengan regulasi sangat berat, dampak keselamatan tinggi, atau konflik pemangku kepentingan yang kompleks, karena faktor-faktor itu akan mengaburkan penilaian performa inti sistem.

Secara praktis, pilih satu proses yang memenuhi empat syarat:

- **masalahnya nyata** dan terasa setiap hari,
- **alur kerjanya cukup jelas**,
- **dokumennya tersedia**,
- dan **ada pemilik proses** yang bersedia menjadi sponsor sekaligus penguji utama.

Contoh proses yang sehat untuk memulai:

- triase tiket internal,
- ringkasan notulen dan tindak lanjut,
- pengecekan kelengkapan dokumen approval,
- atau asisten SOP internal.

Cara berpikir yang sehat bagi pembaca adalah ini: **jangan mulai dari “teknologi apa yang ingin saya pakai?”**, tetapi dari **“proses kecil apa yang paling layak diselamatkan lebih dulu?”** Begitu proses dipilih, tetapkan **satu owner** yang jelas. Owner ini bukan sekadar nama formal. Ia bertanggung jawab atas:

- tujuan pilot,
- definisi sukses,
- validasi hasil,
- dan keputusan apakah pilot layak dilanjutkan atau dihentikan.

14.2 Metrik keberhasilan: waktu, biaya, kualitas, trust

Bab penutup seperti ini harus tegas pada satu hal: **pilot tanpa metrik hanyalah eksperimen yang mudah disalahpahami**. Literatur enterprise yang dilampirkan menekankan bahwa pilot *agentic AI* harus memiliki tujuan yang terukur, misalnya efisiensi operasional, pengurangan biaya, atau peningkatan kapabilitas. Sumber lain juga menekankan bahwa pengukuran perlu mencakup aspek kuantitatif dan kualitatif, bukan hanya akurasi teknis.

Untuk buku ini, empat metrik yang paling membumi adalah:

- **Waktu**
Apakah proses menjadi lebih cepat?
Contoh: waktu triase tiket, waktu menyiapkan ringkasan, waktu approval awal.
- **Biaya**
Apakah ada jam kerja manual yang berkurang, beban baca yang turun, atau biaya komputasi yang masih masuk akal? Literatur dilampiran menegaskan pentingnya *FinOps* dan cost optimization agar beban komputasi tidak membengkak tanpa nilai bisnis yang jelas.
- **Kualitas**
Apakah hasil agent cukup benar, cukup relevan, dan cukup bisa dipakai? Metrik ini bisa berupa akurasi klasifikasi, kualitas ringkasan, atau tingkat *rework*.
- **Trust**
Apakah pengguna percaya pada sistem? Apakah mereka merasa terbantu? Apakah mereka masih terlalu sering mengoreksi hasil? Dalam agentic AI, *trust* bukan bonus; ia adalah syarat agar pilot bisa naik ke tahap berikutnya. Literatur referensi juga menekankan bahwa observability, audit trail, dan pengawasan manusia membantu membangun kepercayaan itu.

Cara paling sederhana untuk memulai adalah membuat tabel evaluasi mingguan. Bukan rumit, tetapi konsisten. Misalnya:

- berapa kasus diproses,
- berapa yang selesai,
- berapa yang salah,
- berapa yang perlu eskalasi,
- dan bagaimana penilaian pengguna.

Pesan penting untuk pembaca: **AI tidak perlu tampak ajaib. Ia cukup harus membuktikan bahwa proses menjadi lebih cepat, lebih rapi, atau lebih bisa dipertanggungjawabkan.**

14.3 Runbook operasi dan checklist evaluasi

Begitu pilot mulai berjalan, organisasi membutuhkan satu dokumen yang sangat membosankan tetapi sangat penting: **runbook**. *Runbook* adalah panduan operasi harian: apa yang harus dicek, siapa yang dihubungi saat gagal, kapan agent boleh lanjut, kapan harus dihentikan, dan bagaimana rollback dilakukan. Literatur dilampiran menekankan pentingnya logging komprehensif, observability, rollback mechanism, review cycles, dan monitoring protocol sejak fase pilot.

Runbook minimum untuk pilot agentic AI sebaiknya memuat:

- **Tujuan workflow**
Proses apa yang dibantu agent, dan output apa yang diharapkan.
- **Ruang lingkup**
Dokumen, tools, user, dan kasus apa yang termasuk atau tidak termasuk.
- **Ambang tindakan**
Kapan agent boleh lanjut otomatis, kapan harus minta persetujuan.
- **Jalur eskalasi**
Siapa yang dihubungi jika model gagal, retrieval tidak relevan, atau output ambigu.
- **Langkah rollback**
Bagaimana kembali ke proses manual jika sistem tidak sehat. Literatur *Agentic AI For Dummies* bahkan menekankan pentingnya rollback yang kuat untuk mengurangi kecemasan organisasi selama fase pilot.
- **Checklist evaluasi berkala**
Apakah latency masih sehat? Apakah kualitas jawaban menurun? Apakah ada kasus salah yang berulang? Apakah pengguna masih percaya?

Secara praktis, pola evaluasi mingguan bisa sangat sederhana:

- Minggu ini berapa kasus diproses?
- Berapa yang selesai otomatis?
- Berapa yang dieskalasi?
- Apakah ada kegagalan tool atau retrieval?
- Apakah ada keluaran yang berisiko?
- Perubahan apa yang harus dilakukan minggu depan?

Pendekatan ini juga cocok dengan AI RMF Playbook milik NIST yang menekankan bahwa tindakan pada fungsi *Govern*, *Map*, *Measure*, dan *Manage* harus dioperasionalkan sebagai kebiasaan berulang, bukan kebijakan yang hanya indah di kertas. ([NIST](#))

14.4 Kapan memperluas agent, kapan menahan diri

Salah satu keahlian manajerial yang paling penting dalam adopsi AI adalah **tahu kapan menambah ruang gerak, dan tahu kapan berhenti**. Literatur referensi menekankan *progressive rollout*, *supervised mode*, *sandbox*, dan peningkatan otonomi secara bertahap. Itu berarti ekspansi harus didasarkan pada bukti, bukan rasa percaya diri yang berlebihan.

Perluas agent bila:

- metrik waktu, kualitas, dan *trust* membaik stabil,
- jalur eskalasi berjalan baik,
- owner proses merasa beban kerjanya benar-benar turun,
- audit trail cukup jelas,
- dan tidak ada pola kegagalan besar yang terus berulang.

Tahan diri bila:

- hasil agent masih sering perlu koreksi besar,
- dokumen dan knowledge base masih berantakan,
- biaya komputasi naik lebih cepat daripada nilainya,
- tim pengguna belum percaya,
- atau organisasi mulai tergoda menambahkan terlalu banyak use case sekaligus.

Prinsip yang sangat sehat dari literatur dilampiran adalah **progressive intelligence**: mulai dari otomasi yang sederhana dan andal, lalu secara bertahap menambah reasoning yang lebih kompleks sambil tetap menjaga human oversight dan mengukur hasilnya.

Bagi pembaca, kalimat praktisnya bisa dibuat begini: **kalaupun sistem belum stabil pada satu proses kecil, tidak ada alasan rasional untuk memperluasnya ke lima proses lain.**

14.5 Struktur tim kecil untuk menjalankan agentic AI

Buku ini ditujukan untuk mahasiswa, praktisi, dan organisasi yang tidak selalu punya tim AI besar. Karena itu struktur tim yang ditawarkan harus realistis. Kabar baiknya: **pilot agentic AI yang sehat tidak memerlukan tentara besar.** Yang diperlukan adalah tim kecil dengan peran yang jelas. Literatur enterprise dan engineering yang dilampirkan menekankan pentingnya kolaborasi lintas fungsi, executive buy-in, business user alignment, serta penggabungan engineering, finance, dan operasi.

Untuk banyak organisasi kecil–menengah, struktur inti bisa cukup seperti ini:

- **Owner proses**
Orang yang paham sakitnya proses dan menentukan apakah agent benar-benar membantu.
- **Operator / analis bisnis**
Orang yang menguji hasil harian, membaca error, dan memberi umpan balik nyata.
- **Teknis / admin sistem**
Orang yang menjaga host, model, workflow, dan log tetap sehat.
- **Penjaga governance**
Tidak harus jabatan baru, tetapi harus ada yang memastikan akses, dokumen, dan approval gate tetap dijaga.

Dalam fase awal, satu orang bisa memegang dua peran. Yang penting bukan banyaknya orang, melainkan **kejelasan tanggung jawab**. Struktur kecil juga lebih sehat untuk 90 hari pertama karena:

- komunikasi lebih cepat,
- umpan balik lebih langsung,
- perubahan lebih mudah,
- dan konflik prioritas lebih sedikit.

Literatur referensi juga menekankan bahwa perubahan budaya penting. Pengguna perlu merasa menjadi co-designer, bukan korban eksperimen. Itu sebabnya peran operator atau user champion sangat penting.

14.6 Roadmap 90 hari: pilot → ukur → revisi → scale

Untuk pembaca buku ini, 90 hari adalah jangka yang pas: cukup pendek untuk tetap fokus, cukup panjang untuk menghasilkan bukti nyata. Roadmap yang sehat bukan “hari ke-90 semua otomatis”, tetapi **hari ke-90 kita tahu apa yang bekerja, apa yang harus diperbaiki, dan apa yang layak diperluas**. Literatur referensi sangat konsisten dengan pola ini: *start small, measure, and scale*; jalankan pilot terkontrol; ukur tidak hanya efisiensi tetapi juga trust dan usability; lalu perbesar secara bertahap.

Fase 1: Hari 1–30 — Pilot terkendali

Fokus pada satu proses, satu owner, satu workflow, dan batas yang jelas.

Yang dikerjakan:

- pilih satu proses yang cukup sakit tetapi tidak terlalu berisiko,
- siapkan dokumen dan knowledge base minimum,
- tetapkan KPI dasar,
- pasang logging, approval gate, dan jalur rollback,
- jalankan agent dalam mode terbatas atau *supervised*.

Yang dicari di fase ini:

- apakah agent benar-benar bisa membantu,
- apakah user mengerti cara memakainya,
- dan apakah proses dasar bisa berjalan tanpa membuat kegaduhan.

Fase 2: Hari 31–60 — Ukur dan revisi

Di fase ini, fokus bukan menambah fitur, tetapi membaca kenyataan.

Yang dikerjakan:

- review metrik waktu, biaya, kualitas, dan trust,
- analisis log dan audit trail,
- identifikasi kasus gagal yang berulang,
- rapikan prompt, workflow, dan knowledge,
- revisi threshold dan jalur eskalasi.

Yang dicari:

- apakah kesalahan mulai turun,
- apakah pengguna semakin percaya,
- apakah biaya komputasi masih sehat,
- dan apakah manfaat bisnis mulai terlihat.

Fase 3: Hari 61–90 — Scale ringan

Bila fase sebelumnya sehat, baru naikan ruang lingkup secara terbatas.

Yang dikerjakan:

- tambah variasi kasus dalam proses yang sama,
- tambah satu user group atau satu unit kecil,
- aktifkan lebih banyak otomatisasi pada langkah yang sudah terbukti aman,
- dokumentasikan runbook final,
- putuskan apakah agent layak diperluas atau tetap dipertahankan pada scope sekarang.

Yang dicari:

- apakah sistem tetap sehat saat volume naik,
- apakah human oversight masih cukup,
- apakah owner proses masih merasa sistem membantu,
- dan apakah ekspansi menghasilkan nilai tambahan, bukan hanya kompleksitas baru.

Kerangka NIST AI RMF sangat cocok dibaca sebagai pendamping 90 hari ini:

- **Govern:** tetapkan kebijakan, peran, dan akuntabilitas,
- **Map:** pahami proses, konteks, dan dampak,
- **Measure:** ukur performa dan risiko,
- **Manage:** revisi, mitigasi, dan terus awasi. ([NIST](#))

Ringkasan langkah kerja bab ini

Agar mudah diingat, pola kerja bab penutup ini bisa diringkas seperti berikut:

- **pilih satu proses**
- **tetapkan satu owner**
- **ukur dengan empat metrik: waktu, biaya, kualitas, trust**
- **jalankan pilot kecil dengan logging dan approval gate**
- **revisi berdasarkan bukti, bukan ego**
- **naikkan scope hanya bila sistem stabil**
- **bangun tim kecil dengan peran jelas**
- **anggap 90 hari sebagai siklus belajar, bukan panggung kemenangan instan**

Penutup bab

Bab ini harus meninggalkan satu pesan besar: **jalan dari pilot ke produksi ringan bukan lompatan, tetapi serangkaian keputusan kecil yang disiplin.** Literatur enterprise yang dilampirkan sangat konsisten disini: agentic AI yang berhasil tumbuh dari pilot yang jelas, metrik yang jujur, observability yang cukup, rollback yang siap, dan perubahan yang bertahap. NIST juga menegaskan bahwa risiko AI harus dikelola secara iteratif, dengan dokumentasi, oversight, dan pengukuran yang terus berjalan. ([NIST](#))

Untuk mahasiswa dan praktisi, pelajaran akhirnya sederhana:

- **mulailah kecil,**
- **ukur dengan jujur,**
- **perbaiki tanpa drama,**
- **dan naikkan otonomi hanya kalau sistem sudah pantas dipercaya.**

Kalau seluruh buku ini adalah perjalanan dari ide ke sistem kerja, maka **Bab 14 adalah pengingat bahwa sistem yang baik bukan yang paling cepat besar, tetapi yang paling kuat tumbuh tanpa kehilangan kendali.**

Lampiran A. CLI instalasi cepat Ubuntu 26.04 + NVIDIA + Docker + Ollama + Open WebUI + OpenClaw / n8n

Lampiran ini dibuat sebagai **catatan singkat, cepat, dan siap-tempel**. Tujuannya bukan menjelaskan semua teori, tetapi memberi jalur praktis dari host kosong sampai fondasi *agentic AI* lokal siap dipakai. Untuk buku ini, **Ubuntu 26.04 LTS** dipakai sebagai platform sasaran, Docker sudah mendukung Ubuntu 26.04, Ollama menyediakan instalasi Linux dan service lokal, Open WebUI merekomendasikan Docker untuk banyak pengguna, OpenClaw menyediakan *installer script* resmi, dan n8n menyediakan jalur *Docker Compose* untuk *self-hosting*. ([Dokumentasi Ubuntu](#))

A.1 Persiapan awal host

Perbarui sistem lebih dulu.

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

Cek versi sistem setelah hidup kembali.

```
lsb_release -a
uname -r
hostnamectl
```

Contoh keluaran:

```
Distributor ID: Ubuntu
Description:    Ubuntu 26.04 LTS
Release:        26.04
Codename:       resolute
```

A.2 Instal driver NVIDIA dan verifikasi GPU

Ubuntu Server menyediakan panduan resmi pemasangan driver NVIDIA melalui paket distro. Jalur paling aman untuk server adalah memakai paket resmi Ubuntu, lalu memverifikasi dengan `nvidia-smi`. ([Ubuntu](#))

Lihat kandidat driver:

```
ubuntu-drivers list
ubuntu-drivers devices
```

Pasang driver yang sesuai, misalnya:

```
sudo apt install -y nvidia-driver-570-server
sudo reboot
```

Verifikasi:

```
nvidia-smi
```

Contoh keluaran:

```
+-----+
| NVIDIA-SMI 570.xx                Driver Version: 570.xx                CUDA Version: 12.x                |
| GPU   Name                               Bus-Id                  Memory-Usage                       |
|  0   NVIDIA GeForce RTX ...        00000000:01:00.0            512MiB / 12288MiB                 |
+-----+
```

A.3 Instal Docker Engine + Compose Plugin

Docker resmi mendukung Ubuntu 26.04. Jalur yang disarankan adalah memakai repo resmi Docker, bukan paket lama bawaan distro. Compose modern dipasang sebagai **plugin docker-compose-plugin**. ([Docker Documentation](#))

Bersihkan paket konflik lama:

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2
podman-docker containerd runc; do
    sudo apt remove -y $pkg
done
```

Pasang prasyarat:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
```

Tambahkan repo Docker resmi:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
    "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
```

Pasang Docker dan Compose plugin:

```
sudo apt install -y docker-ce docker-ce-cli
containerd.io docker-buildx-plugin docker-compose-plugin
```

Verifikasi:

```
docker --version
docker compose version
sudo docker run hello-world
```

Contoh keluaran:

```
Docker version 28.x.x, build xxxxx
Docker Compose version v2.x.x
Hello from Docker!
```

A.4 Instal Ollama

Dokumentasi resmi Ollama untuk Linux menyediakan dua jalur utama: *install script* cepat dan *manual install*. Untuk lampiran ini, pakai jalur cepat. Ollama juga mendokumentasikan cara membuatnya menjadi *startup service*. API lokalnya berjalan di **localhost:11434**. ([Ollama Docs](#))

Instal Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Cek versi:

```
ollama -v
```

Contoh keluaran:

```
ollama version is 0.x.x
```

Jalankan service manual untuk uji cepat:

```
ollama serve
```

Di terminal lain, tarik model awal:

```
ollama pull gemma3
```

Jalankan model:

```
ollama run gemma3
```

Contoh interaksi:

```
>>> Buat ringkasan singkat memo approval pengadaan
Model akan memberi jawaban singkat sesuai prompt
```

Uji API lokal:

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3",
  "messages": [
    { "role": "user", "content": "Buat 3 use case agentic AI
untuk helpdesk internal." }
  ]
}'
```

Contoh keluaran:

```
{
  "model": "gemma3",
  "message": {
    "role": "assistant",
    "content": "1. Triase tiket. 2. Ringkasan kasus. 3. Eskalasi
prioritas tinggi."
  },
  "done": true
}
```

A.5 Jadikan Ollama sebagai service

Dokumentasi Linux Ollama juga memberi contoh *systemd service* agar Ollama otomatis hidup saat boot. ([Ollama Docs](#))

Buat user dan group:

```
sudo useradd -r -s /bin/false -U -m -d /usr/share/ollama ollama
sudo usermod -a -G ollama $(whoami)
```

Buat file service:

```
sudo tee /etc/systemd/system/ollama.service > /dev/null <<'EOF'
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/
bin:/sbin:/bin"

[Install]
WantedBy=multi-user.target
EOF
```

Aktifkan:

```
sudo systemctl daemon-reload
sudo systemctl enable --now ollama
sudo systemctl status ollama --no-pager
```

Contoh keluaran:

- ollama.service - Ollama Service
Loaded: loaded
Active: active (running)

A.6 Instal Open WebUI dengan Docker

Open WebUI merekomendasikan Docker untuk banyak pengguna. Jalur cepat resminya adalah *pull image*, lalu **docker run** dengan volume persisten dan port **3000:8080**. Ia juga menyediakan varian **:ollama** dan **:cuda**, tetapi untuk arsitektur buku ini lebih sehat memisahkan Ollama dan Open WebUI. ([Open WebUI](#))

Pull image:

```
docker pull ghcr.io/open-webui/open-webui:main
```

Jalankan container:

```
docker run -d \
  -p 3000:8080 \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:main
```

Cek status:

```
docker ps
docker logs --tail=30 open-webui
```

Contoh keluaran:

CONTAINER ID	IMAGE	PORTS	NAMES
abcd1234efgh	open-webui:main	0.0.0.0:3000->8080/tcp	open-webui

Buka:

```
http://localhost:3000
```

A.7 Instal OpenClaw via installer resmi

OpenClaw menyediakan *installer script* resmi untuk Linux, macOS, dan WSL. Dokumentasi resminya juga menyarankan verifikasi melalui `openclaw doctor` dan `openclaw gateway status`. Open WebUI mendokumentasikan koneksi ke OpenClaw melalui endpoint *OpenAI-compatible* di port **18789**. ([OpenClaw](#))

Instal OpenClaw:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Verifikasi:

```
openclaw --version
openclaw doctor
openclaw gateway status
```

Contoh keluaran:

```
openclaw 0.x.x
Doctor: OK
Gateway: not running
```

Jalankan gateway:

```
openclaw gateway
```

Cek lagi:

```
openclaw gateway status
```

Contoh keluaran:

```
Gateway running on http://localhost:18789
```

Kalau ingin dihubungkan ke Open WebUI, endpoint umumnya:

```
http://localhost:18789/v1
```


A.8 Instal n8n via Docker Compose

n8n mendokumentasikan *Docker Compose* sebagai jalur *self-hosting* yang rapi. Untuk workflow bisnis kecil–menengah, ini cara yang sangat sehat karena mudah dirawat dan mudah direplikasi. Dokumentasi resminya juga menyarankan timezone seperti **TZ** dan **GENERIC_TIMEZONE**. ([n8n Docs](#))

Buat direktori kerja:

```
mkdir -p ~/n8n-compose/local-files
cd ~/n8n-compose
```

Buat **.env**:

```
cat > .env <<'EOF'
GENERIC_TIMEZONE=Asia/Jakarta
TZ=Asia/Jakarta
EOF
```

Buat **compose.yaml**:

```
cat > compose.yaml <<'EOF'
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    restart: always
    ports:
      - "5678:5678"
    env_file:
      - .env
    environment:
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
      - N8N_RUNNERS_ENABLED=true
    volumes:
      - n8n_data:/home/node/.n8n
      - ./local-files:/files

volumes:
  n8n_data:
EOF
```

Jalankan:

```
sudo docker compose up -d
```

Verifikasi:

```
docker ps
docker logs --tail=30 n8n
```

Contoh keluaran:

CONTAINER ID	IMAGE	PORTS	NAMES
ijkl5678mnop	n8nio/n8n	0.0.0.0:5678->5678/tcp	n8n

Buka:

`http://localhost:5678`

A.9 Uji cepat seluruh stack

Setelah semua hidup, urutan uji cepat yang sehat adalah:

- **cek GPU**

```
nvidia-smi
```

- **cek Docker**

```
docker ps
```

- **cek Ollama**

```
curl http://localhost:11434/api/tags
```

Contoh keluaran:

```
{"models":[{"name":"gemma3","modified_at":"..."}]}
```

- **cek Open WebUI**

```
docker logs --tail=20 open-webui
```

- **cek OpenClaw**

```
openclaw gateway status
```

- **cek n8n**

```
docker logs --tail=20 n8n
```

A.10 Urutan kerja yang disarankan

Agar pembaca tidak tersesat, urutan paling aman adalah:

- **Ubuntu 26.04**
- **driver NVIDIA**
- **Docker + Compose**
- **Ollama**
- **Open WebUI**
- **OpenClaw atau n8n**
- lalu baru **RAG, workflow, dan agent**

Kalimat yang paling penting untuk ditanamkan pada lampiran ini adalah: **jangan buru-buru ke agent kalau host, GPU, Docker, dan model lokal belum sehat.** Di dunia nyata, fondasi yang rapi jauh lebih berharga daripada demo yang terburu-buru.

Lampiran B. Contoh `docker-compose.yml` minimum

Simpan sebagai `docker-compose.yml`.

```
version: "3.9"

services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    environment:
      - OLLAMA_HOST=0.0.0.0:11434
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    restart: unless-stopped
    depends_on:
      - ollama
    ports:
      - "3000:8080"
    volumes:
      - openwebui_data:/app/backend/data
    environment:
      - OLLAMA_BASE_URL=http://ollama:11434

  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    restart: unless-stopped
    depends_on:
      - ollama
    ports:
      - "5678:5678"
    environment:
      - TZ=Asia/Jakarta
      - GENERIC_TIMEZONE=Asia/Jakarta
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
```

```

    - N8N_RUNNERS_ENABLED=true
volumes:
  - n8n_data:/home/node/.n8n
  - ./shared:/data/shared

pm4py:
  image: python:3.11-slim
  container_name: pm4py
  restart: unless-stopped
  working_dir: /workspace
  volumes:
    - ./pm4py:/workspace
    - ./shared:/data/shared
  command: >
    bash -c "pip install --no-cache-dir pm4py pandas jupyterlab &&
      tail -f /dev/null"

volumes:
  ollama_data:
  openwebui_data:
  n8n_data:

```

Keterangan singkat

Struktur minimum ini sengaja dibuat sesederhana mungkin.

- **ollama**
Menjalankan model lokal dan membuka API di port **11434**.
- **open-webui**
Menjadi antarmuka web untuk pengguna, terhubung ke Ollama melalui jaringan internal Docker.
- **n8n**
Menjadi orkestrator *workflow* bisnis, misalnya untuk *webhook*, approval, notifikasi, dan integrasi.
- **pm4py**
Menjadi ruang kerja Python untuk analisis *event log* dan *process mining*.
- **./shared**
Folder bersama agar n8n dan PM4Py bisa membaca file yang sama, misalnya CSV log, notulen, atau hasil ekspor workflow.

Cara menjalankan

Masuk ke folder yang berisi file `docker-compose.yml`, lalu jalankan:

```
docker compose up -d
```

Contoh keluaran:

```
[+] Running 4/4
✓ Container ollama      Started
✓ Container open-webui  Started
✓ Container n8n         Started
✓ Container pm4py       Started
```

Cek status container:

```
docker ps
```

Contoh keluaran:

CONTAINER ID	IMAGE	PORTS	NAMES
xxxxxxx	ollama:latest	0.0.0.0:11434->11434/tcp	ollama
xxxxxxx	open-webui:main	0.0.0.0:3000->8080/tcp	open-webui
xxxxxxx	n8nio/n8n	0.0.0.0:5678->5678/tcp	n8n
xxxxxxx	python:3.11-slim	-	pm4py

Cara mengakses

- **Open WebUI:** <http://localhost:3000>
- **n8n:** <http://localhost:5678>
- **Ollama API:** <http://localhost:11434>

Uji cepat

Uji Ollama

```
curl http://localhost:11434/api/tags
```

Contoh keluaran:

```
{"models": []}
```

Kalau masih kosong, tarik model dulu:

```
docker exec -it ollama ollama pull gemma3
```

Contoh keluaran:

```
pulling manifest
pulling layers
verifying sha256 digest
```

```
success
```

Uji PM4Py

Masuk ke container:

```
docker exec -it pm4py bash
```

Lalu uji import:

```
python -c "import pm4py; import pandas as pd; print('PM4Py  
siap')"
```

Contoh keluaran:

```
PM4Py siap
```

Struktur folder yang disarankan

Agar rapi, folder proyek bisa dibuat seperti ini:

```
project/  
├── docker-compose.yml  
├── shared/  
│   └── event-log.csv  
└── pm4py/  
    └── analisis.py
```

Catatan praktis

- Untuk host tanpa GPU NVIDIA, hapus blok `deploy.resources.reservations.devices` pada service `ollama`.
- Untuk eksperimen awal, konfigurasi ini sudah cukup.
- Untuk produksi ringan, biasanya kita akan menambah:
 - file `.env`
 - *reverse proxy*
 - *backup volume*
 - *secrets management*
 - logging dan monitoring yang lebih rapi

Versi lebih minimum tanpa GPU

Kalau ingin lebih pendek lagi, service **ollama** bisa dibuat seperti ini:

```
ollama:
  image: ollama/ollama:latest
  container_name: ollama
  restart: unless-stopped
  ports:
    - "11434:11434"
  volumes:
    - ollama_data:/root/.ollama
  environment:
    - OLLAMA_HOST=0.0.0.0:11434
```

Penutup singkat

Contoh ini sengaja dibuat **minimum, mudah dipahami, dan mudah direplikasi**.

Inti desainnya sederhana:

- **Ollama** untuk model lokal
- **Open WebUI** untuk antarmuka manusia
- **n8n** untuk *workflow*
- **PM4Py** untuk analisis proses

Lampiran C. Contoh .env minimum yang direkomendasikan

Lampiran ini berisi **contoh .env minimum** yang praktis untuk stack lokal:

Ollama + Open WebUI + n8n + PM4Py.

Tujuannya bukan membuat konfigurasi paling lengkap, tetapi memberi **fondasi yang cukup aman, mudah dipahami, dan mudah diubah** saat pembaca mulai bereksperimen.

Prinsip utamanya sederhana:

- simpan nilai yang sering berubah di **.env**,
- jangan menanam semua konfigurasi langsung di **docker-compose.yml**,
- dan **pisahkan rahasia penting** bila nanti sistem mulai dipakai lebih serius.

C.1 Contoh .env minimum

Simpan file ini sebagai:

`.env`

Isi yang direkomendasikan:

```
# =====
# Timezone
# =====
TZ=Asia/Jakarta
GENERIC_TIMEZONE=Asia/Jakarta

# =====
# Ports
# =====
OLLAMA_PORT=11434
OPENWEBUI_PORT=3000
N8N_PORT=5678

# =====
# Container names
# =====
OLLAMA_CONTAINER=ollama
OPENWEBUI_CONTAINER=open-webui
N8N_CONTAINER=n8n
PM4PY_CONTAINER=pm4py

# =====
# Ollama
# =====
OLLAMA_HOST=0.0.0.0:11434
OLLAMA_BASE_URL=http://ollama:11434
```

```
# =====
# Open WebUI
# =====
WEBUI_AUTH=True
WEBUI_NAME=Agentic AI Bisnis Lab

# =====
# n8n
# =====
N8N_HOST=localhost
N8N_PROTOCOL=http
N8N_PORT=5678
N8N_EDITOR_BASE_URL=http://localhost:5678
N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
N8N_RUNNERS_ENABLED=true

# =====
# Paths / shared workspace
# =====
SHARED_DIR=./shared
PM4PY_WORKDIR=./pm4py
```

C.2 Fungsi tiap bagian

Agar pembaca tidak sekadar menyalin, berikut arti singkat tiap kelompok variabel.

1. Timezone

```
TZ=Asia/Jakarta
GENERIC_TIMEZONE=Asia/Jakarta
```

Dipakai agar:

- waktu log konsisten,
- jadwal di n8n tidak bergeser,
- dan analisis proses atau SLA lebih mudah dibaca.

TZ adalah timezone sistem umum di container.

GENERIC_TIMEZONE penting terutama untuk node terjadwal di n8n.

2. Port

```
OLLAMA_PORT=11434
OPENWEBUI_PORT=3000
N8N_PORT=5678
```

Ini memudahkan bila nanti port perlu diubah tanpa membongkar `docker-compose.yml`.

Contoh:

- Ollama di **11434**
- Open WebUI di **3000**
- n8n di **5678**

3. Nama container

```
OLLAMA_CONTAINER=ollama
OPENWEBUI_CONTAINER=open-webui
N8N_CONTAINER=n8n
PM4PY_CONTAINER=pm4py
```

Secara teknis tidak wajib, tetapi berguna agar file Compose lebih rapi dan mudah dibaca.

4. Variabel Ollama

```
OLLAMA_HOST=0.0.0.0:11434
OLLAMA_BASE_URL=http://ollama:11434
```

- **OLLAMA_HOST** dipakai agar Ollama mendengar di seluruh interface container.
- **OLLAMA_BASE_URL** dipakai Open WebUI atau layanan lain untuk mengakses Ollama dari jaringan Docker.

5. Variabel Open WebUI

```
WEBUI_AUTH=True
WEBUI_NAME=Agentic AI Bisnis Lab
```

- **WEBUI_AUTH=True** berarti autentikasi tetap aktif. Ini lebih aman daripada mematikan login dari awal.
- **WEBUI_NAME** hanya label tampilan agar workspace lebih mudah dikenali.

6. Variabel n8n

```
N8N_HOST=localhost  
N8N_PROTOCOL=http  
N8N_PORT=5678  
N8N_EDITOR_BASE_URL=http://localhost:5678  
N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true  
N8N_RUNNERS_ENABLED=true
```

Bagian ini menjaga n8n tetap:

- mudah diakses,
- cukup aman untuk eksperimen awal,
- dan siap untuk *workflow* yang sedikit lebih serius.

7. Direktori kerja

```
SHARED_DIR=./shared  
PM4PY_WORKDIR=./pm4py
```

Berguna agar:

- file log,
- event log,
- hasil ekspor,
- dan skrip PM4Py

bisa ditata konsisten dari awal,

C.3 Contoh pemakaian di docker-compose.yml

Agar `.env` ini benar-benar berguna, berikut contoh pemakaiannya di `docker-compose.yml`.

```
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ${OLLAMA_CONTAINER}
    restart: unless-stopped
    ports:
      - "${OLLAMA_PORT}:11434"
    environment:
      - OLLAMA_HOST=${OLLAMA_HOST}
    volumes:
      - ollama_data:/root/.ollama

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: ${OPENWEBUI_CONTAINER}
    restart: unless-stopped
    depends_on:
      - ollama
    ports:
      - "${OPENWEBUI_PORT}:8080"
    environment:
      - OLLAMA_BASE_URL=${OLLAMA_BASE_URL}
      - WEBUI_AUTH=${WEBUI_AUTH}
      - WEBUI_NAME=${WEBUI_NAME}
    volumes:
      - openwebui_data:/app/backend/data

  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: ${N8N_CONTAINER}
    restart: unless-stopped
    ports:
      - "${N8N_PORT}:5678"
    environment:
      - TZ=${TZ}
      - GENERIC_TIMEZONE=${GENERIC_TIMEZONE}
      - N8N_HOST=${N8N_HOST}
      - N8N_PROTOCOL=${N8N_PROTOCOL}
      - N8N_PORT=${N8N_PORT}
      - N8N_EDITOR_BASE_URL=${N8N_EDITOR_BASE_URL}
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=${N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS}
      - N8N_RUNNERS_ENABLED=${N8N_RUNNERS_ENABLED}
    volumes:
      - n8n_data:/home/node/.n8n
      - ${SHARED_DIR}:/data/shared
```

C.4 Cara memeriksa .env sudah terbaca

Setelah file `.env` dibuat, jalankan:

```
docker compose config
```

Perintah ini akan menampilkan konfigurasi final setelah variabel `.env` diekspansi.

Contoh keluaran singkat:

```
services:
  ollama:
    container_name: ollama
    ports:
      - mode: ingress
        target: 11434
        published: "11434"
  open-webui:
    container_name: open-webui
    ports:
      - mode: ingress
        target: 8080
        published: "3000"
```

Kalau nilai-nilainya muncul sesuai harapan, berarti `.env` sudah terbaca dengan benar.

C.5 Versi lebih pendek untuk pembaca yang ingin super-minimum

Kalau pembaca ingin versi paling singkat, ini cukup:

```
TZ=Asia/Jakarta
GENERIC_TIMEZONE=Asia/Jakarta

OLLAMA_PORT=11434
OPENWEBUI_PORT=3000
N8N_PORT=5678

OLLAMA_BASE_URL=http://ollama:11434
OLLAMA_HOST=0.0.0.0:11434

WEBUI_AUTH=True
N8N_EDITOR_BASE_URL=http://localhost:5678
N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
N8N_RUNNERS_ENABLED=true
```

C.6 Catatan penting

Ada beberapa kebiasaan baik yang sebaiknya ditanamkan sejak awal:

- **Jangan simpan password penting langsung di `.env`** bila sistem mulai dipakai tim.
- Untuk tahap belajar, `.env` boleh sederhana.
- Untuk produksi ringan, pertimbangkan memakai:
 - Docker secrets
 - file khusus yang tidak ikut ke Git
 - pemisahan `.env.dev`, `.env.stage`, `.env.prod`
- **`WEBUI_AUTH=True`** lebih aman untuk awal daripada mematikan login.
- **Timezone harus konsisten** sejak awal, terutama untuk n8n dan analisis log.
- **Nama dan port yang jelas** akan menghemat banyak waktu debugging.

Penutup singkat

File `.env` yang baik tidak harus panjang. Yang penting:

- **jelas**
- **mudah dibaca**
- **mudah diubah**
- dan **cukup aman untuk tahap eksperimen**

Untuk stack buku ini, `.env` minimum yang baik akan membantu pembaca menjaga satu hal penting:

konfigurasi tetap rapi walau sistem mulai bertambah kompleks.

Lampiran D. 30 Prompt Sistem Siap Pakai untuk Agent Bisnis

Lampiran ini berisi **30 *prompt sistem*** yang bisa langsung dipakai untuk membangun agent bisnis berbasis **Ollama + Open WebUI + OpenClaw / n8n**. Tujuannya bukan membuat agent terdengar “wah”, tetapi membuat agent **lebih tertib, lebih fokus, lebih aman, dan lebih berguna** untuk pekerjaan nyata.

Agar mudah dipakai, semua *prompt* di bawah ini ditulis dengan gaya yang:

- **jelas**
- **langsung ke tugas**
- **mudah dimodifikasi**
- dan **cukup aman** untuk konteks bisnis umum

Cara pakai singkat:

- Tempelkan *prompt sistem* ini pada **model preset, agent, atau workflow AI node**.
- Tambahkan **knowledge base** bila perlu.
- Sesuaikan bagian:
 - nama organisasi
 - jenis dokumen
 - SLA
 - peran pengguna
 - batas aksi agent
 - aturan eskalasi

A. Prompt untuk pengetahuan, SOP, dan kebijakan

1. Asisten SOP internal

Anda adalah asisten SOP internal. Tugas Anda adalah membantu pengguna memahami prosedur kerja berdasarkan dokumen yang tersedia. Jawab singkat, jelas, dan praktis. Jika jawaban berasal dari dokumen, sebutkan sumber atau bagian yang relevan. Jangan mengarang aturan yang tidak ada. Jika informasi tidak ditemukan, katakan bahwa informasi belum tersedia dan sarankan pemeriksaan manual.

2. Asisten kebijakan internal

Anda adalah asisten kebijakan internal organisasi. Tugas Anda menjelaskan kebijakan dengan bahasa sederhana, sopan, dan akurat. Utamakan aturan yang paling baru. Jika ada kemungkinan konflik antar kebijakan, tandai konflik tersebut dan sarankan eskalasi ke manusia. Jangan membuat interpretasi hukum yang tidak didukung dokumen.

3. Asisten FAQ operasional

Anda adalah asisten FAQ operasional. Tugas Anda menjawab pertanyaan rutin dengan cepat dan ringkas berdasarkan basis pengetahuan yang tersedia. Prioritaskan jawaban yang langsung dapat dipraktikkan. Jika pertanyaan di luar cakupan FAQ, katakan dengan jujur bahwa kasus perlu ditangani manusia.

4. Asisten onboarding pegawai baru

Anda adalah asisten onboarding untuk pegawai baru. Jelaskan langkah, istilah, dan aturan kerja dengan bahasa yang ramah, sederhana, dan praktis. Pecah jawaban menjadi langkah-langkah kecil bila perlu. Jangan memberi jawaban yang bertentangan dengan SOP atau kebijakan resmi.

5. Asisten dokumen proyek

Anda adalah asisten dokumen proyek. Tugas Anda membantu pengguna mencari, merangkum, dan menjelaskan isi dokumen proyek. Jawaban harus fokus pada konteks proyek, keputusan, tenggat, penanggung jawab, dan risiko. Jika dokumen tidak cukup jelas, katakan bagian mana yang belum jelas.

B. Prompt untuk ringkasan, memo, dan laporan

6. Ringkasan rapat

Anda adalah asisten ringkasan rapat. Ubah isi rapat menjadi empat bagian: ringkasan utama, keputusan, tindak lanjut, dan penanggung jawab. Jangan menambahkan keputusan yang tidak muncul dalam materi. Jika ada bagian ambigu, tandai sebagai perlu klarifikasi.

7. Ringkasan notulen menjadi aksi

Anda adalah pengubah notulen menjadi aksi. Tugas Anda adalah mengubah notulen menjadi daftar tindakan yang jelas, terukur, dan bisa dikerjakan. Untuk setiap tindakan, tulis: apa yang harus dilakukan, siapa PIC, dan urgensinya. Jangan menulis narasi panjang.

8. Penyusun memo internal

Anda adalah asisten penyusun memo internal. Tulis memo dengan gaya profesional, singkat, dan langsung ke inti. Strukturkan menjadi: latar belakang, isu utama, dampak, dan tindakan yang disarankan. Hindari kalimat bertele-tele.

9. Ringkasan laporan manajerial

Anda adalah asisten ringkasan laporan manajerial. Fokus pada empat hal: capaian, masalah utama, risiko, dan rekomendasi. Tulis dengan bahasa yang mudah dipahami manajer non-teknis. Jangan mengulang isi laporan secara mentah.

10. Pembuat executive summary

Anda adalah pembuat executive summary. Ubah dokumen panjang menjadi ringkasan padat yang menjawab: apa yang terjadi, mengapa penting, apa risikonya, dan apa keputusan yang perlu diambil. Tulis tajam, strategis, dan singkat.

C. Prompt untuk helpdesk, tiket, dan triase

11. Triase tiket helpdesk

Anda adalah agent triase helpdesk internal. Untuk setiap tiket, lakukan empat hal: klasifikasikan kategori, tentukan prioritas awal, jelaskan alasan singkat, dan usulkan tindakan awal. Jika kasus ambigu atau berisiko tinggi, tandai untuk eskalasi manusia.

12. Klasifikasi insiden operasional

Anda adalah asisten klasifikasi insiden operasional. Bacalah laporan insiden dan tentukan jenis masalah, dampak potensial, tingkat urgensi, dan pihak yang paling tepat menangani. Hindari asumsi yang tidak didukung data.

13. Asisten respons awal helpdesk

Anda adalah asisten respons awal helpdesk. Tugas Anda menyusun jawaban awal yang sopan, jelas, dan membantu. Jika tersedia SOP, gunakan sebagai dasar. Jangan menjanjikan penyelesaian akhir bila kasus belum jelas.

14. Penentu eskalasi tiket

Anda adalah agent penentu eskalasi tiket. Tentukan apakah tiket dapat ditangani di level awal atau harus dinaikkan. Naikkan jika ada dampak besar, urgensi tinggi, data sensitif, atau ketidakjelasan konteks. Jelaskan alasan eskalasi secara singkat.

15. Pemeriksa kelengkapan tiket

Anda adalah pemeriksa kelengkapan tiket. Periksa apakah informasi dalam tiket cukup untuk diproses. Jika belum cukup, tulis daftar informasi yang masih kurang dengan bahasa yang sopan dan spesifik.

D. Prompt untuk approval, pemeriksaan dokumen, dan workflow

16. Pemeriksa kelengkapan dokumen approval

Anda adalah asisten pemeriksa kelengkapan dokumen approval. Tugas Anda menilai apakah dokumen sudah cukup lengkap untuk masuk ke tahap berikutnya. Jika belum lengkap, sebutkan item yang kurang dengan jelas. Jangan memberi approval final; hanya beri evaluasi awal.

17. Ringkasan konteks approval

Anda adalah asisten ringkasan konteks approval. Bacalah dokumen pengajuan lalu buat ringkasan singkat yang membantu approver memahami isi, tujuan, risiko, dan hal yang perlu diperhatikan sebelum menyetujui.

18. Agent routing approval

Anda adalah agent routing approval. Tugas Anda menentukan jalur approval yang paling tepat berdasarkan jenis dokumen, unit pengaju, nilai transaksi, dan risiko. Jika jalur tidak jelas, tandai untuk review manusia.

19. Agent pengecekan risiko dokumen

Anda adalah asisten pengecekan risiko dokumen. Baca dokumen lalu soroti risiko administratif, operasional, atau kepatuhan yang tampak dari isi dokumen. Jangan membuat keputusan final; hanya berikan peringatan dan alasan.

20. Agent tindak lanjut revisi

Anda adalah asisten tindak lanjut revisi dokumen. Jika dokumen belum layak diproses, buat daftar revisi yang jelas, spesifik, dan mudah dipahami pengaju. Hindari bahasa menyalahkan.

E. Prompt untuk keputusan operasional dan manajerial

21. Asisten keputusan operasional

Anda adalah asisten keputusan operasional. Tugas Anda merangkum situasi, mengidentifikasi masalah utama, dan menyusun dua sampai tiga opsi tindakan yang realistis. Jangan memilih keputusan final; biarkan manusia memutuskan.

22. Asisten briefing manajer

Anda adalah asisten briefing untuk manajer. Tulis ringkasan singkat yang menjelaskan konteks, angka penting, isu utama, dan keputusan yang kemungkinan perlu diambil hari ini. Prioritaskan kejelasan dan relevansi.

23. Penanda anomali proses

Anda adalah asisten penanda anomali proses. Tugas Anda membaca data, log, atau laporan proses dan menyoroti pola yang tidak biasa, keterlambatan, rework, atau penyimpangan. Jelaskan secara singkat kenapa pola itu penting.

24. Pemberi rekomendasi langkah awal

Anda adalah asisten rekomendasi langkah awal. Untuk setiap kasus, berikut langkah awal paling aman dan paling praktis. Jika informasi belum cukup, sarankan klarifikasi atau eskalasi. Jangan bertindak diluar informasi yang tersedia.

25. Agent eskalasi manajerial

Anda adalah agent eskalasi manajerial. Tugas Anda menentukan kapan kasus perlu dinaikkan ke pimpinan. Naikkan bila ada dampak bisnis tinggi, keterlambatan kritis, risiko reputasi, atau potensi pelanggaran kebijakan.

F. Prompt untuk quality control, keamanan, dan governance

26. Pemeriksa kesetiaan terhadap sumber

Anda adalah pemeriksa kesetiaan jawaban terhadap sumber. Tugas Anda memastikan bahwa jawaban hanya berdasarkan dokumen atau konteks yang tersedia. Jika ada bagian yang tidak didukung sumber, tandai dengan jelas.

27. Pemeriksa data sensitif

Anda adalah pemeriksa data sensitif. Tugas Anda mendeteksi informasi yang berpotensi sensitif seperti data pribadi, data pelanggan, informasi keuangan, atau akses internal. Jika ditemukan, beri peringatan dan sarankan penanganan yang lebih aman.

28. Agent dengan batas aksi ketat

Anda adalah agent bisnis dengan batas aksi ketat. Anda boleh membaca, merangkum, mengklasifikasikan, dan memberi rekomendasi awal. Anda tidak boleh mengambil keputusan final, mengubah data penting, atau mengirim komunikasi keluar tanpa persetujuan manusia.

29. Pemeriksa kualitas output

Anda adalah pemeriksa kualitas output agent. Nilai hasil berdasarkan empat hal: relevansi, kejelasan, kelengkapan, dan risiko salah tafsir. Jika kualitas belum cukup, jelaskan apa yang harus diperbaiki.

30. Agent fallback ke manusia

Anda adalah agent yang harus selalu mengutamakan keselamatan operasional. Jika konteks tidak cukup, sumber tidak jelas, atau konsekuensi tindakan terlalu tinggi, jangan menebak. Berhenti, jelaskan keterbatasan, dan sarankan eskalasi ke manusia.

Cara memilih prompt yang tepat

Jangan tempel semua *prompt* sekaligus. Itu justru akan membuat agent kabur. Pilih satu sesuai fungsi utama.

Kalau tugasnya mencari pengetahuan, pakai:

- Asisten SOP internal
- Asisten kebijakan internal
- Asisten FAQ operasional

Kalau tugasnya merangkum dokumen, pakai:

- Ringkasan rapat
- Ringkasan laporan manajerial
- Executive summary

Kalau tugasnya triase dan helpdesk, pakai:

- Triase tiket helpdesk
- Penentu eskalasi tiket
- Pemeriksa kelengkapan tiket

Kalau tugasnya approval dan workflow, pakai:

- Pemeriksa kelengkapan dokumen approval
- Ringkasan konteks approval
- Agent routing approval

Kalau tugasnya lebih sensitif, tambahkan pagar seperti:

- Agent dengan batas aksi ketat
- Pemeriksa data sensitif
- Agent fallback ke manusia

Cara memodifikasi agar lebih sesuai organisasi

Agar *prompt sistem* benar-benar terasa milik organisasi, sesuaikan beberapa bagian berikut:

- ganti istilah umum menjadi istilah internal organisasi
- tambahkan nama proses, unit, atau jenis dokumen
- tambahkan SLA bila ada
- tambahkan aturan eskalasi
- tambahkan larangan yang jelas, misalnya:
 - tidak boleh memberi keputusan final
 - tidak boleh membuka data sensitif
 - wajib menyebut sumber bila ada

Contoh modifikasi sederhana:

Dari:

```
Anda adalah agent triase helpdesk internal.
```

Menjadi:

```
Anda adalah agent triase helpdesk internal untuk unit layanan digital organisasi. Fokus Anda adalah tiket akses sistem, approval workflow, dan masalah dokumen internal.
```

Pola *prompt sistem* yang sehat

Ada pola sederhana yang hampir selalu berguna:

- **Peran**
Anda adalah siapa?
- **Tugas**
Apa yang harus dilakukan?
- **Batas**
Apa yang tidak boleh dilakukan?
- **Format output**
Jawaban harus dalam bentuk apa?
- **Aturan eskalasi**
Kapan harus berhenti dan naik ke manusia?

Contoh template umum:

```
Anda adalah [peran agent].  
Tugas Anda adalah [fungsi utama].  
Utamakan [tujuan utama].  
Jangan [larangan utama].  
Jika informasi tidak cukup, [aturan fallback].  
Format jawaban: [format yang diinginkan].
```

Penutup singkat

Lampiran ini sengaja dibuat **praktis, siap tempel, dan mudah diubah.**

Yang paling penting untuk pembaca adalah memahami satu hal:

Prompt sistem yang baik bukan yang paling puitis, tetapi yang paling jelas peran, tugas, batas, dan jalur eskalasinya.

Di dunia bisnis, agent tidak perlu terdengar seperti manusia super.

Ia cukup harus:

- **tahu tugasnya**
- **tahu batasnya**
- **tahu kapan berhenti**
- **dan tahu kapan menyerahkan keputusan ke manusia**

Lampiran E. Checklist RAG dan *Governance*

Lampiran ini dibuat sebagai **checklist praktis** untuk memastikan sistem RAG bisnis tidak hanya “bisa menjawab”, tetapi juga **bisa dipercaya, bisa diaudit, dan aman dipakai**. Ini sangat selaras dengan menekankan **auditability, source transparency, governance, dan penggunaan nyata di bisnis**. Open WebUI sendiri menempatkan **Knowledge** sebagai tempat file dan koleksi yang bisa dicari, dibaca, dan dinalar AI; mendukung **Focused Retrieval** dan **Full Context**; serta menekankan sitasi sumber, *hybrid search*, dan akses yang bisa dibatasi. Sementara NIST AI RMF menekankan bahwa pengelolaan risiko AI harus dilakukan secara iteratif melalui fungsi *Govern, Map, Measure, dan Manage*. ([Open WebUI](#))

Di bawah ini, checklist dibagi ke dalam beberapa bagian. Pembaca tidak harus sempurna di hari pertama. Yang penting, **setiap bagian diperiksa dengan sadar sebelum sistem diperluas**. Open WebUI juga mendukung RBAC melalui **roles, permissions, dan groups**, sedangkan Docker Compose mendukung penggunaan **secrets** agar token dan kredensial tidak diletakkan sembarangan di source code atau file konfigurasi biasa. ([Open WebUI](#))

E.1 Checklist tujuan dan ruang lingkup

Sebelum membahas dokumen dan retrieval, pastikan pertanyaan paling dasar sudah jelas.

- **Use case RAG sudah jelas**
Misalnya: asisten SOP, triase tiket, ringkasan memo, atau pencarian kebijakan.
- **Batas domain pengetahuan sudah jelas**
Apa saja yang masuk ke basis pengetahuan, dan apa yang tidak.
- **Pemilik proses atau owner sudah ditetapkan**
Harus ada satu pihak yang bertanggung jawab atas kualitas isi dan pemakaian sistem.
- **Pengguna utama sudah diidentifikasi**
Apakah untuk staf, supervisor, analis, atau manajer.
- **Tingkat risiko sudah dipahami**
Apakah sistem hanya memberi jawaban, atau juga memicu tindak lanjut dan eskalasi.

Checklist ini penting karena RAG yang baik harus dibangun untuk **masalah yang nyata dan terbatas**, bukan untuk semua hal sekaligus. Itu juga sejalan dengan kerangka NIST yang menekankan pemetaan konteks, tujuan, pemangku kepentingan, dan risiko sejak awal. ([NIST](#))

E.2 Checklist kualitas dokumen sumber

RAG yang baik sangat bergantung pada dokumen yang dimasukkan. Open WebUI menjelaskan bahwa *Knowledge* bekerja dengan retrieval berbasis chunk, dan bahkan membedakan antara **Focused Retrieval** untuk koleksi besar dan **Full Context** untuk dokumen pendek yang perlu dimasukkan utuh. Itu berarti dokumen yang kacau akan menghasilkan retrieval yang kacau juga. ([Open WebUI](#))

- **Dokumen memiliki judul yang jelas**
- **Versi dokumen diketahui**
- **Tanggal berlaku atau tanggal revisi tersedia**
- **Pemilik dokumen atau pemilik proses diketahui**
- **Isi dokumen tidak mencampur terlalu banyak topik yang tidak berkaitan**
- **Dokumen lama, usang, atau duplikat sudah dibersihkan**
- **PDF hasil scan sudah dicek kualitas ekstraksi teksnya**
- **Tabel, gambar, dan lampiran kritis punya teks penjelas yang memadai**
- **Bahasa istilah cukup konsisten antar dokumen**
- **Dokumen sensitif sudah dipisahkan dari dokumen umum**

Kalimat sederhananya: **jangan berharap RAG memperbaiki dokumen yang memang buruk.**

E.3 Checklist struktur knowledge base

Open WebUI menjelaskan bahwa *knowledge bases* dapat dibuat per proyek, tim, atau topik, lalu ditempelkan ke model secara terarah. Akses juga dapat dibuat *scoped* agar model hanya mencari pada basis pengetahuan yang relevan. ([Open WebUI](#))

- **Knowledge base dipisah per domain**
 - SOP
 - kebijakan
 - FAQ
 - kontrak
 - dokumen proyek
 - laporan
- **Nama knowledge base jelas dan tidak ambigu**
- **Dokumen dimasukkan ke koleksi yang benar**
- **Tidak semua dokumen ditempel ke semua model**
- **Knowledge base untuk data sensitif dipisah**
- **Model atau agent hanya diberi akses ke knowledge yang relevan**
- **Ada aturan siapa yang boleh menambah, mengubah, atau mengeksport knowledge**

Checklist ini penting karena Open WebUI memang mendukung *scoped access* dan pengelolaan berbasis koleksi. Itu berarti organisasi bisa dan sebaiknya **membatasi ruang cari model** agar retrieval lebih fokus dan risiko kebocoran turun. ([Open WebUI](#))

E.4 Checklist retrieval dan mode konteks

Open WebUI secara resmi membedakan dua mode utama: **Focused Retrieval** yang memakai RAG untuk menarik chunk relevan, dan **Full Context** yang menyuntikkan seluruh isi file ke setiap pesan. Dokumentasi juga menyebut *hybrid search* dengan gabungan BM25, vector search, dan *reranking* untuk akurasi yang lebih baik. ([Open WebUI](#))

- **Sudah diputuskan kapan memakai Focused Retrieval**
 - cocok untuk koleksi besar
 - cocok saat hanya sebagian dokumen yang relevan
- **Sudah diputuskan kapan memakai Full Context**
 - cocok untuk dokumen pendek
 - cocok untuk style guide atau instruksi tetap
- **Model yang dipakai cukup mampu melakukan retrieval**
- **Chunking cukup masuk akal untuk jenis dokumen**
- **Metadata penting seperti versi, unit, tanggal, dan kategori tersedia**
- **Hybrid search diaktifkan bila koleksi cukup besar**
- **Reranking dipertimbangkan bila hasil retrieval sering kurang tepat**
- **Native function calling diuji dengan benar**
- **Kalau native function calling aktif, model sudah diberi instruksi untuk memakai tool knowledge**
- **Jika retrieval tidak konsisten, sudah dicoba ulang dengan Full Context untuk dokumen pendek**

Checklist ini penting karena Open WebUI juga mengingatkan bahwa saat **native function calling** aktif, *Knowledge* tidak otomatis disuntikkan; model harus memanggil tool knowledge atau kita perlu mematikan mode itu pada model tertentu. ([Open WebUI](#))

E.5 Checklist evaluasi kualitas RAG

Sistem RAG harus diuji bukan hanya dari “jawaban terasa bagus”, tetapi dari apakah ia **mengambil sumber yang benar, menjawab dengan setia, cukup cepat, dan bisa ditelusuri**. Open WebUI menekankan sitasi dan batas dokumen yang dijaga, sedangkan NIST mendorong pengukuran berkala dan pengelolaan risiko secara terus-menerus. ([Open WebUI](#))

- **Ada daftar pertanyaan uji yang nyata**
- **Pertanyaan uji mewakili kebutuhan harian pengguna**
- **Ada jawaban referensi atau baseline**
- **Ada dokumen sumber yang seharusnya muncul**
- **Relevance diuji**
 - apakah dokumen/chunk yang diambil memang tepat
- **Faithfulness diuji**
 - apakah jawaban setia pada sumber
 - apakah ada bagian yang mengarang
- **Latency diuji**
 - apakah waktu respons masih nyaman
- **Source traceability diuji**
 - apakah sumber bisa ditunjukkan dengan jelas
- **Kasus gagal dicatat**
- **Perbaikan dilakukan berdasarkan hasil uji, bukan perasaan**

Prinsip sederhananya: **jawaban yang lancar belum tentu jawaban yang benar.**

E.6 Checklist governance akses dan peran

Open WebUI menyediakan RBAC dengan tiga lapisan yang saling berkaitan: **roles**, **permissions**, dan **groups**. Model keamanannya bersifat *additive*, sehingga akses pengguna adalah gabungan dari hak default dan hak tambahan dari grup. Ini berarti governance akses harus didesain dengan sadar, bukan dibuka lebar untuk semua user. ([Open WebUI](#))

- **Admin dibatasi ke sedikit orang**
- **User biasa tidak otomatis bisa melihat semua model**
- **Knowledge base sensitif dibatasi per grup**
- **Permissions untuk fitur tertentu sudah dipikirkan**
 - web search
 - upload file
 - delete chat
 - workspace edit
- **Grup pengguna mencerminkan struktur kerja yang nyata**
- **Akses individual hanya diberikan bila benar-benar perlu**
- **Ada proses review akses berkala**
- **Akses pengguna yang keluar tim atau proyek dicabut**

Ini penting karena sistem RAG bisnis sering memegang dokumen internal yang jauh lebih sensitif daripada chatbot biasa.

E.7 Checklist security minimum

Docker menekankan bahwa *secrets* seperti password, API token, atau private key **tidak boleh** disimpan di image, source code, atau file konfigurasi biasa jika bisa dihindari. Compose juga mendukung deklarasi *secrets* dari file atau environment tertentu. ([Docker Documentation](#))

- Token dan password tidak ditulis langsung di **docker-compose.yml**
- Rahasia penting disimpan sebagai Docker secrets atau file terpisah
- File **.env** tidak dipakai untuk rahasia yang sangat sensitif bila sistem sudah dipakai tim
- Hak baca file rahasia dibatasi
- Volume data penting dibackup
- Port yang dibuka ke luar benar-benar perlu
- Host dan container diupdate berkala
- Log tidak membocorkan token atau data pribadi
- Model dan knowledge tidak diberi akses yang tidak perlu
- Ada batas aksi agent untuk tool yang sensitif

Kalimat kuncinya: **rahasia harus diperlakukan sebagai rahasia, bukan sekadar string konfigurasi.**

E.8 Checklist governance dokumen dan data sensitif

- Dokumen sensitif diidentifikasi sejak awal
- Data pribadi sudah di mask atau diminimalkan bila tidak perlu
- Koleksi dokumen umum dan sensitif dipisah
- Ada aturan berapa lama dokumen disimpan
- Ada mekanisme menghapus dokumen yang tidak boleh lagi dipakai
- Tidak semua user boleh mengunggah file ke knowledge base produksi
- Versi lama dokumen tidak dibiarkan aktif tanpa alasan
- Ada pencatatan siapa mengunggah atau mengubah dokumen penting

Bagian ini penting untuk menjaga agar sistem RAG tidak berubah menjadi gudang dokumen yang tak terkendali.

E.9 Checklist observability dan audit

RAG yang dipakai di bisnis perlu jejak yang cukup. Itu berarti bukan hanya tahu jawabannya apa, tetapi juga:

- model mencari ke mana,
- dokumen apa yang diambil,
- user siapa yang bertanya,
- dan apa keluaran akhirnya.
- **Ada log pertanyaan dan waktu respons**
- **Ada catatan dokumen/chunk yang dipakai**
- **Kasus error retrieval dicatat**
- **Kasus jawaban kosong dicatat**
- **Kasus yang perlu manusia dicatat**
- **Audit trail untuk perubahan knowledge tersedia**
- **Sumber jawaban dapat ditunjukkan kembali**
- **Ada review berkala terhadap log dan error yang berulang**

Ini juga sejalan dengan pendekatan NIST yang mendorong pengukuran, dokumentasi, dan pengelolaan risiko secara terus-menerus. ([NIST](#))

E.10 Checklist sebelum go-live

Gunakan bagian ini sebagai pemeriksaan akhir.

- **Use case sudah jelas**
- **Owner sudah jelas**
- **Knowledge base sudah dibersihkan**
- **Mode retrieval sudah dipilih dengan sadar**
- **Pertanyaan uji sudah dijalankan**
- **Jawaban sudah diperiksa relevansi dan faithfulness-nya**
- **Akses user dan grup sudah ditata**
- **Secrets tidak ditaruh sembarangan**
- **Ada audit trail minimum**
- **Ada fallback ke manusia bila sistem gagal**
- **Ada runbook singkat bila hasil retrieval memburuk**
- **Pilot sudah menunjukkan manfaat nyata sebelum diperluas**

Versi checklist super-singkat

Kalau pembaca hanya ingin ringkasan 10 poin, gunakan ini:

- **Dokumen sumber bersih dan relevan**
- **Knowledge base dipisah per domain**
- **Mode retrieval dipilih dengan sadar**
- **Pertanyaan uji nyata sudah disiapkan**
- **Sumber jawaban bisa ditelusuri**
- **User access dibatasi sesuai peran**
- **Rahasia tidak ditulis sembarangan**
- **Data sensitif diperlakukan khusus**
- **Log dan audit trail aktif**
- **Ada fallback ke manusia**

Penutup singkat

Checklist ini sengaja dibuat **praktis, operasional, dan mudah dipakai ulang**. Pesan terpentingnya sederhana: **RAG yang berguna bukan yang paling banyak dokumennya, tetapi yang paling jelas sumbernya, paling rapi aksesnya, dan paling mudah diaudit saat salah**. Open WebUI memberi fondasi kuat untuk itu melalui *Knowledge*, mode retrieval, sitasi, dan RBAC; Docker membantu pengelolaan *secrets*; dan NIST memberi kerangka agar semua ini dipandang sebagai proses manajemen risiko yang berulang, bukan tugas satu kali. ([Open WebUI](#))

Lampiran F. Template *Event Log* Sederhana untuk *Process Mining*

Lampiran ini dibuat untuk membantu pembaca menyiapkan **format log proses yang cukup rapi** agar bisa dipakai untuk analisis *process mining*, baik secara manual, dengan Python, maupun dengan pustaka seperti **PM4Py**. Untuk buku ini, tujuannya bukan membuat sistem pencatatan paling rumit, tetapi membuat **jejak proses yang cukup jelas** agar *bottleneck*, *rework*, *exception*, dan *approval loop* bisa terlihat.

Prinsip paling penting dalam *event log* sederhana adalah: **satu baris = satu kejadian**. Setiap kejadian minimal harus menjawab empat hal:

- **kasus apa** yang sedang terjadi
- **aktivitas apa** yang dilakukan
- **kapan** aktivitas itu terjadi
- **siapa** yang melakukannya

Kalau empat unsur ini sudah ada, pembaca sudah bisa mulai membaca proses secara jauh lebih objektif.

F.1 Struktur kolom minimum

Template minimum yang direkomendasikan:

- **case_id**
ID unik untuk satu kasus atau satu alur proses
- **activity**
Nama aktivitas yang terjadi
- **timestamp**
Waktu kejadian
- **actor**
Orang, unit, atau sistem yang menjalankan aktivitas

Contoh header CSV:

```
case_id,activity,timestamp,actor
```

Itu sudah cukup untuk banyak eksperimen awal.

F.2 Template paling sederhana

Berikut template CSV minimum yang bisa langsung dipakai.

```
case_id,activity,timestamp,actor
REQ-001,Submit Request,2026-04-10 08:00:00,Unit A
REQ-001,Verify Budget,2026-04-10 10:30:00,Finance
REQ-001,Request Revision,2026-04-10 11:00:00,Finance
REQ-001,Resubmit Request,2026-04-11 09:15:00,Unit A
REQ-001,Verify Budget,2026-04-11 10:00:00,Finance
REQ-001,Final Approval,2026-04-11 15:20:00,Head Unit
REQ-002,Submit Request,2026-04-10 09:20:00,Unit B
REQ-002,Verify Budget,2026-04-12 14:00:00,Finance
REQ-002,Final Approval,2026-04-14 16:10:00,Head Unit
```

Template ini sudah cukup untuk melihat:

- jalur proses utama
- *rework*
- keterlambatan
- dan variasi antar kasus

F.3 Template yang lebih kaya

Kalau pembaca ingin log yang sedikit lebih informatif, tambahkan kolom berikut:

- **status**
status setelah aktivitas
- **channel**
asal kejadian, misalnya email, tiket, form, sistem
- **priority**
rendah, sedang, tinggi
- **document_type**
jenis dokumen atau kasus
- **note**
catatan singkat tambahan

Contoh:

```
case_id,activity,timestamp,actor,status,channel,priority,document_type,note
REQ-001,Submit Request,2026-04-10 08:00:00,Unit A,submitted,form,medium,pengadaan,permintaan awal dikirim
REQ-001,Verify Budget,2026-04-10 10:30:00,Finance,in_review,system,medium,pengadaan,verifikasi anggaran dimulai
REQ-001,Request Revision,2026-04-10 11:00:00,Finance,revision_needed,email,medium,pengadaan,lampiran anggaran belum lengkap
REQ-001,Resubmit Request,2026-04-11 09:15:00,Unit A,resubmitted,form,medium,pengadaan,lampiran dilengkapi
```



```
REQ-001,Verify Budget,2026-04-11
10:00:00,Finance,in_review,system,medium,pengadaan,verifikasi
ulang
REQ-001,Final Approval,2026-04-11 15:20:00,Head
Unit,approved,system,medium,pengadaan,approval akhir diberikan
```

Versi ini jauh lebih membantu bila nanti pembaca ingin:

- memfilter kasus prioritas tinggi
- membandingkan proses dari channel berbeda
- atau melihat apakah jenis dokumen tertentu lebih sering macet

F.4 Cara membuat file log sederhana

Buat direktori kerja:

```
mkdir -p ~/process-log-template
```

Buat file CSV minimum:

```
cat > ~/process-log-template/event-log.csv <<'EOF'
case_id,activity,timestamp,actor
REQ-001,Submit Request,2026-04-10 08:00:00,Unit A
REQ-001,Verify Budget,2026-04-10 10:30:00,Finance
REQ-001,Request Revision,2026-04-10 11:00:00,Finance
REQ-001,Resubmit Request,2026-04-11 09:15:00,Unit A
REQ-001,Verify Budget,2026-04-11 10:00:00,Finance
REQ-001,Final Approval,2026-04-11 15:20:00,Head Unit
REQ-002,Submit Request,2026-04-10 09:20:00,Unit B
REQ-002,Verify Budget,2026-04-12 14:00:00,Finance
REQ-002,Final Approval,2026-04-14 16:10:00,Head Unit
EOF
```

Lihat hasilnya:

```
column -s, -t < ~/process-log-template/event-log.csv
```

Contoh keluaran:

case_id	activity	timestamp	actor
REQ-001	Submit Request	2026-04-10 08:00:00	Unit A
REQ-001	Verify Budget	2026-04-10 10:30:00	Finance
REQ-001	Request Revision	2026-04-10 11:00:00	Finance
REQ-001	Resubmit Request	2026-04-11 09:15:00	Unit A
REQ-001	Verify Budget	2026-04-11 10:00:00	Finance
REQ-001	Final Approval	2026-04-11 15:20:00	Head Unit
REQ-002	Submit Request	2026-04-10 09:20:00	Unit B
REQ-002	Verify Budget	2026-04-12 14:00:00	Finance
REQ-002	Final Approval	2026-04-14 16:10:00	Head Unit

F.5 Template untuk beberapa jenis proses

Agar pembaca mudah mengadaptasi, berikut contoh pola untuk beberapa proses umum.

1. Approval dokumen

```
case_id,activity,timestamp,actor
APP-001,Submit Document,2026-04-10 08:00:00,Requester
APP-001,Initial Review,2026-04-10 09:10:00,Admin
APP-001,Request Revision,2026-04-10 09:30:00,Admin
APP-001,Resubmit Document,2026-04-10 13:15:00,Requester
APP-001,Manager Approval,2026-04-10 16:00:00,Manager
```

2. Helpdesk internal

```
case_id,activity,timestamp,actor
HD-001,Ticket Created,2026-04-11 08:10:00,User
HD-001,Triage,2026-04-11 08:20:00,Helpdesk
HD-001,Assign Technician,2026-04-11 08:30:00,Helpdesk
HD-001,Resolve Issue,2026-04-11 10:45:00,Technician
HD-001,Close Ticket,2026-04-11 11:00:00,Helpdesk
```

3. Ringkasan rapat dan tindak lanjut

```
case_id,activity,timestamp,actor
MEET-001,Meeting Held,2026-04-12 09:00:00,Team
MEET-001,Notes Written,2026-04-12 10:30:00,Secretary
MEET-001,Action Items Assigned,2026-04-12 11:00:00,Manager
MEET-001,Follow-up Reminder Sent,2026-04-13 09:00:00,System
MEET-001,Task Confirmed,2026-04-13 13:00:00,PIC
```

F.6 Aturan penamaan aktivitas

Agar *event log* mudah dianalisis, gunakan nama aktivitas yang:

- **konsisten**
- **pendek**
- **jelas**
- **dan tidak berubah-ubah tanpa alasan**

Contoh yang baik:

- **Submit Request**
- **Verify Budget**
- **Request Revision**
- **Final Approval**

Contoh yang kurang baik:

- kirim
- cek
- review lagi
- lanjut proses
- done

Semakin konsisten penamaan, semakin mudah nanti proses dibaca.

F.7 Aturan timestamp

Gunakan format waktu yang konsisten.

Paling aman untuk latihan awal adalah:

```
YYYY-MM-DD HH:MM:SS
```

Contoh:

```
2026-04-10 08:00:00
```

Kalau memungkinkan, tetap gunakan zona waktu yang sama di seluruh sistem.

Untuk buku ini, **Asia/Jakarta** adalah pilihan paling wajar.

F.8 Template log dengan kolom durasi hasil proses

Kadang pembaca ingin menyimpan hasil akhir per kasus agar lebih mudah dibaca manajer. Itu boleh, tetapi **jangan menggantikan event log utama**. Jadikan ini tabel tambahan.

```
case_id,start_time,end_time,total_hours,status,variant
REQ-001,2026-04-10 08:00:00,2026-04-11
15:20:00,31.33,approved,with_revision
REQ-002,2026-04-10 09:20:00,2026-04-14
16:10:00,102.83,approved,straight_through
```

Tabel ini berguna untuk:

- membaca SLA
- membandingkan kasus cepat dan lambat
- melihat varian proses

Tetapi ingat: **ini ringkasan kasus, bukan event log rinci**.

F.9 Pemeriksaan cepat kualitas event log

Sebelum dipakai untuk *process mining*, cek hal-hal berikut:

- setiap baris punya **case_id**
- setiap baris punya **activity**
- setiap baris punya **timestamp**
- urutan waktu masuk akal
- tidak ada nama aktivitas yang campur aduk
- tidak ada timestamp kosong
- satu kasus punya lebih dari satu event
- format file konsisten CSV atau JSONL
- kalau ada kolom tambahan, isinya tidak liar

F.10 Contoh cek cepat dengan Python

Cek jumlah kasus, jumlah event, dan aktivitas unik:

```
python - <<'EOF'
import pandas as pd

df = pd.read_csv('/home/oai/process-log-template/event-log.csv')
print("Jumlah event :", len(df))
print("Jumlah kasus :", df['case_id'].nunique())
print("Aktivitas unik :", sorted(df['activity'].unique()))
EOF
```

Contoh keluaran:

```
Jumlah event : 9
Jumlah kasus : 2
Aktivitas unik : ['Final Approval', 'Request Revision',
'Resubmit Request', 'Submit Request', 'Verify Budget']
```

Cek jumlah event per kasus:

```
python - <<'EOF'
import pandas as pd

df = pd.read_csv('/home/oai/process-log-template/event-log.csv')
print(df.groupby('case_id').size())
EOF
```

Contoh keluaran:

```
case_id
REQ-001    6
REQ-002    3
dtype: int64
```

Cek apakah timestamp bisa dibaca dengan benar:

```
python - <<'EOF'
import pandas as pd

df = pd.read_csv('/home/oai/process-log-template/event-log.csv')
df['timestamp'] = pd.to_datetime(df['timestamp'])
print(df.dtypes)
EOF
```

Contoh keluaran:

```
case_id          object
activity         object
timestamp    datetime64[ns]
actor           object
dtype: object
```

F.11 Template JSONL alternatif

Kalau pembaca lebih suka format JSONL, ini contoh sederhananya:

```
{"case_id":"REQ-001","activity":"Submit Request","timestamp":"2026-04-10
08:00:00","actor":"Unit A"}
{"case_id":"REQ-001","activity":"Verify Budget","timestamp":"2026-04-10
10:30:00","actor":"Finance"}
{"case_id":"REQ-001","activity":"Request Revision","timestamp":"2026-04-10
11:00:00","actor":"Finance"}
{"case_id":"REQ-001","activity":"Resubmit Request","timestamp":"2026-04-11
09:15:00","actor":"Unit A"}
{"case_id":"REQ-001","activity":"Final Approval","timestamp":"2026-04-11
15:20:00","actor":"Head Unit"}
```

Format ini berguna bila log nanti dihasilkan dari workflow atau agent secara otomatis.

F.12 Penutup singkat

Template ini sengaja dibuat **sederhana, praktis, dan cukup kuat** untuk latihan awal *process mining*. Pesan paling penting untuk pembaca adalah:

Jangan menunggu sistem log yang sempurna. Mulailah dari event log yang cukup rapi.

Kalau empat unsur inti sudah ada:

- **case_id**
- **activity**
- **timestamp**
- **actor**

maka pembaca sudah bisa mulai:

- melihat alur nyata
- menemukan bottleneck
- membaca rework
- dan memilih peluang agentic AI yang lebih tajam

Lampiran G. Checklist evaluasi agent sebelum *go-live*

Lampiran ini dibuat untuk membantu pembaca melakukan **pemeriksaan akhir** sebelum agent dipakai di lingkungan kerja yang lebih nyata. Tujuannya sederhana: **jangan biarkan agent masuk ke operasi hanya karena demonya terlihat bagus**. Agent yang layak *go-live* harus cukup baik di empat sisi sekaligus:

- **berguna**
- **aman**
- **bisa diaudit**
- **bisa dihentikan bila perlu**

Ini sangat selaras dengan menempatkan *guardrails*, *governance*, *observability*, dan adopsi bertahap sebagai fondasi sistem agentic yang matang.

Di bawah ini, checklist dibagi menjadi beberapa bagian. Pembaca tidak harus sempurna di semua hal pada hari pertama, tetapi **semakin banyak kotak yang belum tercentang, semakin besar alasan untuk menahan diri dan belum *go-live***.

G.1 Checklist tujuan dan ruang lingkup

Sebelum melihat performa teknis, pastikan pertanyaan paling dasar sudah dijawab.

- **Tujuan agent sudah jelas**
- **Use case utama sudah spesifik**
- **Ruang lingkup tugas agent sudah tertulis**
- **Batas yang tidak boleh dilakukan agent sudah eksplisit**
- **Owner proses sudah jelas**
- **Pengguna utama sudah jelas**
- **Kriteria sukses *go-live* sudah ditentukan**
- **Ada keputusan jelas: agent ini hanya memberi saran, membantu keputusan, atau boleh mengeksekusi langkah tertentu**

Kalau bagian ini belum jelas, biasanya masalah berikutnya akan kabur juga.

G.2 Checklist kualitas fungsi utama agent

Ini inti evaluasi: apakah agent benar-benar membantu tugas yang dijanjikan?

- **Agent berhasil menjalankan fungsi utamanya**
- **Output agent relevan dengan konteks kasus**
- **Jawaban atau rekomendasi cukup akurat**
- **Format output konsisten**
- **Agent tidak sering keluar topik**
- **Agent tidak terlalu sering mengarang**
- **Agent bisa mengatakan “tidak tahu” atau “perlu eskalasi”**
- **Agent cukup stabil pada kasus yang mirip**
- **Agent cukup masuk akal pada kasus yang sedikit berbeda**
- **Agent membantu mengurangi kerja manual, bukan menambah kebingungan**

Prinsip penting: jangan nilai agent dari satu demo terbaiknya. Nilai dari **rangkaian kasus nyata**.

G.3 Checklist kualitas RAG dan basis pengetahuan

Kalau agent memakai *knowledge base* atau RAG, maka kualitas retrieval harus diperiksa terpisah.

- **Dokumen sumber yang dipakai masih berlaku**
- **Versi dokumen jelas**
- **Dokumen duplikat atau usang sudah dibersihkan**
- **Knowledge base dipisah per domain**
- **Mode retrieval sudah dipilih dengan sadar**
- **Jawaban agent mengacu ke sumber yang benar**
- **Sumber jawaban bisa ditelusuri**
- **Kasus retrieval salah sudah diuji**
- **Agent tidak terlalu sering mengambil dokumen yang tidak relevan**
- **Latency retrieval masih masuk akal**

Kalau sistem memakai RAG tetapi tidak bisa menunjukkan sumber dengan cukup jelas, maka untuk banyak konteks bisnis itu sudah menjadi tanda bahaya.

G.4 Checklist guardrails dan batas aksi

Sebelum *go-live*, pastikan agent tidak dibiarkan liar.

- **Batas aksi agent tertulis**
- **Aksi sensitif memerlukan approval**
- **Kasus ambigu diarahkan ke manusia**
- **Kasus berisiko tinggi otomatis eskalasi**
- **Agent tidak boleh mengubah data penting tanpa izin**
- **Agent tidak boleh mengirim komunikasi keluar tanpa persetujuan bila risikonya tinggi**
- **Ada aturan *fallback* bila konteks kurang**
- **Ada batas retry**
- **Ada aturan saat output tidak valid**
- **Ada aturan jelas: kapan agent harus berhenti**

Kalau agent punya akses tools tetapi tidak punya batas aksi yang jelas, berarti sistem belum siap *go-live*.

G.5 Checklist keamanan minimum

Ini pemeriksaan dasar agar agent tidak berubah menjadi sumber risiko baru.

- **Password dan token tidak ditaruh sembarangan**
- **Rahasia penting tidak ditulis langsung di konfigurasi publik**
- **Hak akses user dibatasi sesuai kebutuhan**
- **Knowledge base sensitif dibatasi ke user/grup yang tepat**
- **Model sensitif tidak dibuka ke semua orang**
- **Port yang tidak perlu tidak dibuka**
- **Host dan container sudah diperbarui**
- **File sensitif sudah di mask atau diminimalkan bila perlu**
- **Agent tidak punya akses lebih dari yang dibutuhkan**
- **Ada mekanisme mencabut akses dengan cepat bila ada masalah**

Untuk tahap produksi ringan, keamanan minimum lebih penting daripada fitur tambahan yang belum matang.

G.6 Checklist logging, audit, dan observability

Agent yang masuk operasi tanpa jejak adalah risiko.

- **Ada log input dan output utama**
- **Ada catatan waktu respons**
- **Ada catatan tool call**
- **Ada catatan retrieval sumber**
- **Ada log kasus gagal**
- **Ada audit trail untuk approval manusia**
- **Ada cara membaca ulang kasus setelah kejadian**
- **Ada penanda siapa user atau role yang menjalankan agent**
- **Ada cara membedakan kasus sukses dan gagal**
- **Log cukup aman dan tidak membocorkan data sensitif**

Kalau sistem salah, organisasi harus bisa menjawab:

- apa input-nya,
- apa yang dibaca,
- apa yang diputuskan,
- siapa yang menyetujui,
- dan bagaimana memperbaikinya.

G.7 Checklist metrik operasi

Sebelum *go-live*, pastikan metrik dasar sudah diukur minimal di lingkungan pilot.

- **Waktu respons sudah diukur**
- **Tingkat penyelesaian tugas sudah diukur**
- **Akurasi atau kualitas hasil sudah diukur**
- **Tingkat eskalasi sudah dicatat**
- **Kasus koreksi manual sudah dicatat**
- **Biaya komputasi atau beban operasi dipahami**
- **Pengguna merasa terbantu**
- **Trust pengguna mulai terbentuk**
- **Tidak ada penurunan kualitas yang jelas saat volume naik**
- **Ada baseline sebelum agent dipasang**

Kalau tidak ada baseline, organisasi akan mudah tertipu oleh rasa “sepertinya lebih baik”.

G.8 Checklist kesiapan pengguna dan proses

Kadang masalah bukan pada agent, tetapi pada manusia dan proses di sekelilingnya.

- **Pengguna tahu fungsi agent**
- **Pengguna tahu batas agent**
- **Pengguna tahu kapan harus percaya, kapan harus cek ulang**
- **Owner proses siap menerima umpan balik**
- **Ada jalur dukungan bila pengguna bingung**
- **Perubahan kerja akibat agent sudah dijelaskan**
- **Pengguna tidak menganggap agent sebagai pengganti total penilaian manusia**
- **Ada dokumentasi singkat cara memakai agent**
- **Ada contoh kasus yang benar dan salah**
- **Tim tidak dipaksa percaya sebelum sistem membuktikan diri**

Trust dibangun dari pengalaman yang konsisten, bukan dari slogan.

G.9 Checklist runbook dan penanganan kegagalan

Sistem yang sehat bukan yang tidak pernah gagal, tetapi yang **gagal dengan aman**.

- **Ada runbook operasi singkat**
- **Ada langkah jika model gagal merespons**
- **Ada langkah jika retrieval salah**
- **Ada langkah jika tool error**
- **Ada langkah jika output tidak valid**
- **Ada langkah rollback ke proses manual**
- **Ada orang yang dihubungi saat insiden**
- **Ada batas waktu kapan masalah harus eskalasi**
- **Ada prosedur mematikan agent sementara**
- **Ada prosedur memulihkan layanan**

Kalau agent tidak punya jalur mati yang jelas, berarti agent belum layak masuk produksi.

G.10 Checklist evaluasi kasus uji sebelum go-live

Sebelum *go-live*, jalankan uji akhir pada beberapa kategori kasus.

Kasus normal

- Agent menangani kasus normal dengan baik
- Output cukup cepat dan cukup benar

Kasus ambigu

- Agent tidak terlalu percaya diri
- Agent meminta klarifikasi atau eskalasi

Kasus sensitif

- Agent tidak membocorkan data
- Agent tidak melangkah otomatis tanpa izin

Kasus gagal

- Agent memberi pesan yang jelas
- Sistem tetap stabil
- Ada fallback ke manusia

Kasus volume lebih tinggi

- Agent tetap cukup stabil
- Latency masih dapat diterima
- Tidak ada degradasi besar yang mengejutkan

G.11 Skor keputusan sederhana sebelum go-live

Untuk pembaca yang ingin keputusan cepat, gunakan skala sederhana berikut.

Beri nilai:

- **2** = siap
- **1** = cukup, tetapi perlu perhatian
- **0** = belum siap

Kategori yang dinilai:

- tujuan & scope
- kualitas fungsi utama
- kualitas RAG
- guardrails
- keamanan
- logs & audit
- metrik operasi
- kesiapan pengguna
- runbook & failure handling

Contoh tabel ringkas:

Kategori	Nilai
Tujuan & scope	2
Fungsi utama	2
RAG	1
Guardrails	2
Keamanan	1
Logs & audit	1
Metrik operasi	2
Kesiapan pengguna	1
Runbook	1

Interpretasi sederhana:

- **15–18** → layak untuk *go-live* ringan dengan pengawasan
- **10–14** → tunda ekspansi, perbaiki dulu
- **<10** → belum layak *go-live*

Ini bukan metode ilmiah mutlak, tetapi cukup berguna untuk mencegah keputusan yang terlalu emosional.

G.12 Versi checklist super-singkat

Kalau pembaca hanya ingin ringkasan 10 poin, gunakan ini:

- **Use case jelas**
- **Owner jelas**
- **Agent cukup akurat**
- **Sumber jawaban bisa ditelusuri**
- **Aksi sensitif dibatasi**
- **Hak akses rapi**
- **Log dan audit trail aktif**
- **Ada fallback ke manusia**
- **Pengguna paham cara memakai agent**
- **Ada runbook dan tombol berhenti**

Penutup singkat

Checklist ini sengaja dibuat **praktis, operasional, dan tidak muluk-muluk**.
Pesan utamanya sederhana:

Agent layak *go-live* bukan ketika ia terlihat paling pintar, tetapi ketika ia sudah cukup berguna, cukup aman, cukup bisa dijelaskan, dan cukup mudah dihentikan bila salah.

Untuk buku ini, itu adalah standar yang sehat.
Bukan standar sempurna, tetapi **standar yang bertanggung jawab**.

Glossary

Istilah	Penjelasan singkat
Agent	Entitas perangkat lunak yang tidak hanya menjawab, tetapi juga memahami tujuan, memilih langkah, memakai alat, dan bergerak menuju hasil yang diinginkan.
Agentic AI	Pendekatan AI yang menekankan kerja berbasis tujuan , penggunaan <i>tools</i> , konteks, dan alur tindakan, bukan sekadar menghasilkan teks.
Agentic system	Keseluruhan sistem tempat agent bekerja, mencakup model, memori, <i>tools</i> , workflow, evaluasi, dan pengawasan manusia.
Approval gate	Titik persetujuan yang mewajibkan campur tangan manusia sebelum agent melanjutkan aksi tertentu.
Approval workflow	Alur kerja persetujuan dokumen, permintaan, atau keputusan yang melibatkan tahapan review, verifikasi, dan approval.
Audit trail	Jejak yang dapat ditelusuri untuk melihat siapa melakukan apa, kapan, dengan dasar apa, dan hasil akhirnya apa.
Autonomy theater	Kondisi ketika sistem AI tampak canggih dan otonom di demo, tetapi sebenarnya rapuh, tidak dapat diaudit, dan sulit diandalkan.
Bias	Penyimpangan atau kecenderungan tidak adil dalam data atau model yang dapat menghasilkan keluaran yang menyesatkan atau timpang.
Bottleneck	Titik macet dalam proses yang menyebabkan antrian, keterlambatan, atau penurunan kecepatan kerja.
Case ID	Identitas unik untuk satu kasus atau satu alur proses dalam <i>event log</i> .
Chunking	Proses memotong dokumen menjadi bagian-bagian yang cukup kecil untuk dicari, tetapi tetap cukup utuh untuk bermakna.
Conformance checking	Analisis untuk membandingkan proses aktual dengan proses yang seharusnya atau dengan SOP/model proses yang diharapkan.
Data governance	Aturan dan praktik untuk mengelola kualitas, akses, sensitivitas, penggunaan, dan siklus hidup data.

Dataset evaluasi	Kumpulan data yang dipakai untuk menilai performa model atau agent secara jujur, terpisah dari data training.
Dataset training	Kumpulan data yang dipakai untuk membentuk perilaku model atau agent selama proses pelatihan atau penyesuaian.
Drift	Perubahan data, proses, atau konteks yang membuat model atau sistem yang tadinya baik menjadi semakin kurang relevan.
Embedding	Representasi teks dalam bentuk vektor angka agar bisa dicari berdasarkan kemiripan makna.
Escalation	Proses menaikkan kasus ke manusia atau tingkat otoritas yang lebih tinggi ketika agent tidak cukup aman atau yakin untuk melanjutkan.
Event log	Catatan kejadian proses yang minimal memuat <i>case_id</i> , aktivitas, waktu, dan pelaku.
Exception	Penyimpangan atau kasus khusus yang tidak mengikuti jalur proses normal.
Faithfulness	Tingkat kesetiaan jawaban model terhadap sumber atau dokumen yang dipakai.
Fine-tuning	Penyesuaian model dengan dataset khusus agar lebih konsisten atau lebih fasih pada domain tertentu.
Focused Retrieval	Mode RAG yang hanya mengambil potongan dokumen yang paling relevan terhadap pertanyaan pengguna.
Full Context	Mode yang menyuntikkan seluruh isi dokumen ke konteks model, biasanya cocok untuk dokumen pendek dan sangat relevan.
Generative AI	AI yang mampu menghasilkan konten baru, seperti teks, ringkasan, jawaban, atau kode.
Guardrails	Pagar atau batas operasional yang menjaga agent tetap aman, terkontrol, dan tidak bertindak di luar aturan.
Hallucination / halusinasi	Keluaran model yang terdengar meyakinkan tetapi tidak didukung data atau sumber yang benar.
Human-in-the-loop	Pola kerja di mana manusia tetap ikut memeriksa, mengarahkan, atau menyetujui langkah tertentu dari agent.
Hybrid	Pendekatan gabungan, misalnya memakai RAG sekaligus <i>fine-tuning</i> untuk mendapatkan pengetahuan dinamis dan perilaku yang lebih spesifik.

Instruction dataset	Kumpulan contoh instruksi, input, dan output yang dipakai untuk mengajari model cara merespons tugas tertentu.
Knowledge base	Kumpulan dokumen, file, atau sumber pengetahuan yang dipakai agent untuk menjawab atau bernalar di atas informasi yang relevan.
Latency	Waktu yang dibutuhkan sistem dari menerima input sampai menghasilkan output.
Least privilege	Prinsip keamanan bahwa agent atau pengguna hanya diberi akses minimum yang benar-benar diperlukan.
Logging	Pencatatan kejadian sistem seperti input, output, <i>tool call</i> , error, dan waktu respons.
Masking	Penyembunyian atau penyamaran data sensitif agar tidak tampil utuh di dataset atau sistem.
Memory	Konteks kerja yang dipertahankan agent, misalnya riwayat percakapan, status tugas, atau informasi proses sebelumnya.
Metadata	Label tambahan pada dokumen atau chunk, seperti judul, versi, kategori, tanggal berlaku, atau tingkat sensitivitas.
Monitoring	Pemantauan terus-menerus terhadap performa agent, termasuk kualitas, kecepatan, error, dan biaya.
Multi-agent	Arsitektur dengan beberapa agent yang berbagi peran atau bekerja sama dalam satu sistem.
n8n	Orchestrator <i>workflow</i> bisnis berbasis <i>low-code</i> yang dapat memadukan otomatisasi proses dengan AI agent dan integrasi tools.
Observability	Kemampuan untuk melihat, memahami, dan menelusuri perilaku sistem secara utuh selama berjalan.
Ollama	Runtime model lokal yang dipakai untuk menjalankan model AI secara <i>self-hosted</i> dan menyediakan API lokal.
Open WebUI	Antarmuka <i>self-hosted</i> untuk berinteraksi dengan model lokal, mengelola <i>knowledge base</i> , dan membangun workspace AI.
OpenClaw	<i>Agent runtime</i> yang dirancang agar agent dapat memakai tools dan <i>skills</i> untuk melakukan tindakan nyata.
Overconfidence	Kondisi ketika model atau sistem terlalu percaya diri meskipun dasar datanya lemah atau konteksnya tidak cukup.

PM4Py	Pustaka <i>open source</i> Python untuk <i>process mining</i> , termasuk <i>process discovery</i> , <i>conformance checking</i> , dan analisis bottleneck.
Process discovery	Teknik untuk menemukan pola atau model proses aktual dari <i>event log</i> .
Process mining	Pendekatan analitis untuk memahami proses nyata berdasarkan jejak kejadian di data.
Prompt injection	Upaya menyisipkan instruksi menyesatkan ke dalam input atau dokumen agar agent bertindak di luar maksud semula.
RAG (Retrieval-Augmented Generation)	Teknik yang menggabungkan model generatif dengan pencarian dokumen atau pengetahuan eksternal agar jawaban lebih faktual dan dapat ditelusuri.
Relevance	Tingkat kecocokan dokumen atau chunk yang diambil terhadap pertanyaan pengguna.
Reranking	Penyusunan ulang kandidat hasil retrieval agar dokumen yang paling relevan naik ke urutan teratas.
Rework	Kerja ulang yang terjadi karena revisi, pengulangan langkah, atau kegagalan pada tahap sebelumnya.
RPA (Robotic Process Automation)	Otomasi yang meniru langkah-langkah manusia di aplikasi, biasanya cocok untuk tugas yang stabil dan deterministik.
Runbook	Panduan operasi harian dan penanganan masalah untuk sistem agent atau workflow.
Sandbox	Lingkungan terbatas dan terisolasi untuk menjalankan aksi atau kode agar risiko kerusakan lebih kecil.
Scheduling	Mekanisme untuk menjalankan workflow atau agent berdasarkan waktu atau jadwal tertentu.
Secrets	Data rahasia seperti token, password, atau API key yang harus disimpan dan dikelola secara aman.
Session state	Keadaan atau konteks aktif dalam satu sesi kerja agent, termasuk memori pendek dan status proses.
Single-agent	Arsitektur yang memakai satu agent utama untuk menangani satu domain atau alur kerja tertentu.
Synthetic data	Data buatan yang dibuat untuk menambah variasi, melengkapi kekurangan data nyata, atau membantu eksperimen awal.

Threshold	Ambang batas yang menentukan kapan agent boleh lanjut otomatis dan kapan harus berhenti atau meminta review.
Tool / tools	Fungsi, API, atau layanan yang dapat dipanggil agent untuk mengambil data atau melakukan aksi.
Traceability	Kemampuan untuk menelusuri asal jawaban, langkah keputusan, dan sumber data yang dipakai agent.
Use case	Skenario penggunaan nyata yang menjadi sasaran penerapan agentic AI.
Vector database	Basis data yang menyimpan embedding agar pencarian berbasis kemiripan makna bisa dilakukan dengan cepat.
Webhook	Titik masuk HTTP yang memungkinkan sistem lain memicu workflow atau mengirim data ke agent/orkestrator.
Workflow	Urutan langkah kerja yang terstruktur untuk menyelesaikan suatu proses bisnis.

Referensi

ACL Anthology. (2023). *Self-Instruct: Aligning language models with self-generated instructions*. <https://aclanthology.org/2023.acl-long.754/>

Baker, P. (2026). *Agentic AI for dummies*. John Wiley & Sons.

Biswas, A., & Talukdar, W. (2025). *Building agentic AI systems: Create intelligent, autonomous AI agents that can reason, plan, and adapt*. Packt Publishing.

Docker, Inc. (n.d.). *Post-installation steps*. Docker Docs. <https://docs.docker.com/engine/install/linux-postinstall/>

Docker, Inc. (n.d.). *Rootless mode*. Docker Docs. <https://docs.docker.com/engine/security/rootless/>

Docker, Inc. (n.d.). *Secrets*. Docker Docs. <https://docs.docker.com/reference/compose-file/secrets/>

Docker, Inc. (n.d.). *Secrets in Compose*. Docker Docs. <https://docs.docker.com/compose/how-tos/use-secrets/>

Docker, Inc. (n.d.). *Ubuntu*. Docker Docs. <https://docs.docker.com/engine/install/ubuntu/>

Huang, K. (Ed.). (2025). *Agentic AI: Theories and practices*. Springer Nature Switzerland.

Hugging Face. (n.d.). *Dataset cards*. <https://huggingface.co/docs/hub/datasets-cards>

Hugging Face. (n.d.). *Distilabel*. <https://huggingface.co/docs/hub/datasets-distilabel>

Hugging Face. (n.d.). *PEFT*. <https://huggingface.co/docs/peft/index>

Hugging Face. (n.d.). *Splits and slicing — datasets 1.0.1 documentation*. <https://huggingface.co/docs/datasets/v1.0.1/splits.html>

Nagasubramanian, D. (2026). *Agentic AI for engineers: Architecting goal-driven systems*. Apress.

National Institute of Standards and Technology. (n.d.). *AI Risk Management Framework*. <https://www.nist.gov/itl/ai-risk-management-framework>

National Institute of Standards and Technology. (n.d.). *NIST AI RMF Playbook*. <https://www.nist.gov/itl/ai-risk-management-framework/nist-ai-rmf-playbook>

Nayebi, F. (2025). *Foundations of agentic AI for retail: Concepts, technologies, and architectures for autonomous retail systems*. Gradient Divergence.

n8n. (n.d.). *AI Agent node documentation*. n8n Docs. <https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/>

n8n. (n.d.). *Docker*. *n8n Docs*. <https://docs.n8n.io/hosting/installation/docker/>

n8n. (n.d.). *Docker Compose*. *n8n Docs*.
<https://docs.n8n.io/hosting/installation/server-setups/docker-compose/>

n8n. (n.d.). *Human-in-the-loop for AI tool calls*. *n8n Docs*.
<https://docs.n8n.io/advanced-ai/human-in-the-loop-tools/>

NVIDIA. (n.d.). *Installing the NVIDIA Container Toolkit*. *NVIDIA Documentation*.
<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

NVIDIA. (n.d.). *NVIDIA System Management Interface (nvidia-smi)*. *NVIDIA Documentation*.
<https://docs.nvidia.com/deploy/nvidia-smi/>

Ollama. (n.d.). *Authentication*. *Ollama Documentation*.
<https://docs.ollama.com/api/authentication>

Ollama. (n.d.). *Context length*. *Ollama Documentation*.
<https://docs.ollama.com/context-length>

Ollama. (n.d.). *Embeddings*. *Ollama Documentation*.
<https://docs.ollama.com/capabilities/embeddings>

Ollama. (n.d.). *FAQ*. *Ollama Documentation*. <https://docs.ollama.com/faq>

Ollama. (n.d.). *Introduction*. *Ollama Documentation*. <https://docs.ollama.com/api/introduction>

Ollama. (n.d.). *Linux*. *Ollama Documentation*. <https://docs.ollama.com/linux>

Ollama. (n.d.). *Quickstart*. *Ollama Documentation*. <https://docs.ollama.com/quickstart>

Ollama. (n.d.). *Tool calling*. *Ollama Documentation*.
<https://docs.ollama.com/capabilities/tool-calling>

Onno W. Purbo. (2026). *Agentic AI untuk bisnis dengan open source: Membangun agen kerja lokal untuk operasi pengetahuan, manajemen proses, dan workflow bisnis* [Dokumen terlampir / draft naskah].

Onno W. Purbo. (2026). *Buat agentic AI dengan open source* [Dokumen terlampir / draft naskah].

Open WebUI. (n.d.). *Getting started*. *Open WebUI Documentation*.
<https://docs.openwebui.com/getting-started/>

Open WebUI. (n.d.). *Home*. *Open WebUI Documentation*. <https://docs.openwebui.com/>

Open WebUI. (n.d.). *Knowledge*. *Open WebUI Documentation*.
<https://docs.openwebui.com/features/workspace/knowledge/>

Open WebUI. (n.d.). *Models*. *Open WebUI Documentation*.
<https://docs.openwebui.com/features/workspace/models/>

Open WebUI. (n.d.). *OpenClaw. Open WebUI Documentation.*
<https://docs.openwebui.com/getting-started/quick-start/connect-an-agent/openclaw/>

Open WebUI. (n.d.). *Quick start. Open WebUI Documentation.*
<https://docs.openwebui.com/getting-started/quick-start/>

Open WebUI. (n.d.). *Role-Based Access Control (RBAC). Open WebUI Documentation.*
<https://docs.openwebui.com/features/authentication-access/rbac/>

Open WebUI. (n.d.). *Security policy. Open WebUI Documentation.*
<https://docs.openwebui.com/security/>

OpenClaw. (n.d.). *Install. OpenClaw Documentation.* <https://docs.openclaw.ai/install>

OpenClaw. (n.d.). *OpenClaw — Personal AI Assistant.* <https://openclaw.ai/>

OpenClaw. (n.d.). *Skills. OpenClaw Documentation.* <https://docs.openclaw.ai/tools/skills>

PM4Py. (n.d.). *pm4py package — PM4Py documentation.*
<https://pm4py-source.readthedocs.io/en/stable/pm4py.html>

Process Intelligence Solutions GmbH (P.I.S.). (n.d.). *PM4Py features.*
<https://processintelligence.solutions/pm4py/features>

Process Intelligence Solutions GmbH (P.I.S.). (n.d.). *PMTk.*
<https://processintelligence.solutions/pmtk/>

Process Intelligence Solutions GmbH (P.I.S.). (n.d.). *Welcome to PM4Py's documentation!.*
<https://processintelligence.solutions/pm4py/api/index.html>

Ranjan, S., Chembachere, D., & Lobo, L. (2025). *Agentic AI in enterprise: Harnessing agentic AI for business transformation.* Apress.

Ubuntu. (2026). *Ubuntu 26.04 LTS release notes. Ubuntu Documentation.*
<https://documentation.ubuntu.com/release-notes/26.04/>

Ubuntu. (n.d.). *Firewall. Ubuntu Server Documentation.*
<https://ubuntu.com/server/docs/how-to/security/firewalls/>

Ubuntu. (n.d.). *Install and manage packages. Ubuntu Server Documentation.*
<https://ubuntu.com/server/docs/how-to/software/package-management/>

Ubuntu. (n.d.). *NVIDIA drivers installation. Ubuntu Server Documentation.*
<https://ubuntu.com/server/docs/how-to/graphics/install-nvidia-drivers/>

Ubuntu. (n.d.). *Ubuntu Server documentation.* <https://ubuntu.com/server/docs/>

Ubuntu. (n.d.). *User management. Ubuntu Server Documentation.*
<https://ubuntu.com/server/docs/how-to/security/user-management/>

Tentang Penulis



Onno W. Purbo, saat ini bertugas sebagai rektor di Institut Teknologi Tangerang Selatan (ITTS). Onno memperoleh gelar Ph.D bidang Electrical Engineering dari University of Waterloo, Canada, adalah seorang copyleft, educator dan ICT evangelist. Dia sudah mempublikasikan 50++ buku, termasuk free ICT ebook untuk sekolah tahun 2008. Beberapa buku terakhirnya adalah "Internet-TCP/IP: Konsep Dan Implementasi", 2018; "Sistem Operasi, Konsep Dan Membuat Linux OpenWRT Dan ROM Android", 2019; "IPv6 Untuk Mendukung Operasi Jaringan Dan Domain Name System", 2019; "Kubernetes untuk Pemula", 2024 dan "Membuat Operator Seluler 5G Sendiri", 2024.

Dia memimpin sambungan pertama Internet di Institut Teknologi Bandung, tahun 1993-2000, dan menggunakannya untuk membuat jaringan Internet pendidikan yang pertama di Indonesia. Dia membebaskan frekuensi WiFi, memperkenalkan RT/RW-net, antenna Wajanbolic dan jaringan selular OpenBTS dan private 5G sendiri. Dia memimpin jaringan telepon pertama di atas Internet, VoIP Merdeka, yang kemudian hari dikenal sebagai VoIP Rakyat berbasis SIP dan menggunakan kode area +62520 dan +62521. Dia saat ini aktif memperkenalkan e-Learning, dan menjalankan server e-Learning di <http://lms.onnocenter.or.id/moodle/> 66,000++ siswa / mahasiswa dan <https://opencourse.itts.ac.id> 39.000+ mahasiswa secara gratis. Email: onno@indo.net.id